



Docker, Kubernetes & AWS

First steps with docker

Overview: Introduction

ubuntu 



- ▶ **In this chapter:**

- ▶ We will explore the typical problems during the development cycle
- ▶ ... and how docker helps to mitigate them

- ▶ **We will explore basic concepts:**

- ▶ What is a "container"?
- ▶ What is the difference between a virtual machine and a Docker container?
- ▶ How do we install Docker Desktop?

- ▶ **You will even docker to run your first containers:**

- ▶ Ubuntu
- ▶ Python
- ▶ And we will see how easy it is to switch to a different version!

Docker, Kubernetes & AWS

Development & deployment problems

How to align dev environment and production?

- ▶ Classic problem: **"But it works on my system!"**
- ▶ **Let's say:**
 - ▶ You're developing a Node.js application on Windows or macOS
 - ▶ The application needs to read a cached file
 - ▶ Everything works perfectly on your local setup – all tests pass
- ▶ **But then:**
 - ▶ Deployment: We push it to a Linux server
 - ▶ And we get an error:
 - ▶ *no such file or directory, open '/app/Cache/cache-data.json'*
 - ▶ **The reason:**
 - ▶ The default Linux file systems (usually: ext4) are case-sensitive
 - ▶ The default file systems on macOS / Windows are case-insensitive
- ▶ **Solution:**
 - ▶ We need to ensure our dev environment matches production



Docker, Kubernetes & AWS

What is a container?

What is a container?

- ▶ A separate environment on your computer, which is isolated from the rest of your system
- ▶ It contains the whole code base, including all dependencies that are required to execute an application
- ▶ It can be executed on other operating systems (OS)
- ▶ It is isolated from other containers



Docker, Kubernetes & AWS

Let's install  **docker. !**

What is Docker?

- ▶ The company Docker, Inc has developed an industry standard for container
- ▶ Docker is the default software for managing containers
- ▶ Docker itself is free software, and available as open-source
- ▶ **Docker Desktop:**
 - ▶ A software to manage Docker with a graphical user interface (GUI)
 - ▶ Docker Desktop is proprietary software
- ▶ **From my understanding:**
 - ▶ Free for personal use
 - ▶ Free for small companies
 - ▶ And, free for educational use
- ▶ **But I'm not a lawyer, thus I would need to refer you directly to Docker:**
 - ▶ <https://www.docker.com/pricing/>



Docker, Kubernetes & AWS

First Examples

First Examples

- ▶ Let's just see, if docker works properly
- ▶ **For this, we want to run a hello world example:**
 - ▶ `docker run -it hello-world`
- ▶ **But we can do way more:**
 - ▶ **Ubuntu:**
 - ▶ `docker run -it ubuntu`
 - ▶ **We can also just run Node.js:**
 - ▶ `docker run -it node`

Docker, Kubernetes & AWS

Where do the containers come from?

Where do the containers come from?

- ▶ **How did docker know that we want to start an Ubuntu?**

- ▶ `docker run -it ubuntu`
- ▶ Docker is connecting to a container registry
- ▶ This registry provides pre-built images for us
- ▶ **The default one: Docker Hub** (<https://hub.docker.com>)
 - ▶ Provided by Docker, Inc.
 - ▶ Images are free and are provided by Docker or the community
 - ▶ No login is required for downloading images
 - ▶ But if we wanted to publish images, we would have to register
- ▶ **Security implications:**
 - ▶ You're downloading files that will be executed on your system
 - ▶ Be careful to ensure that you can trust the images
 - ▶ Consider using mostly (or only) *official images* or images from a *verified publisher*



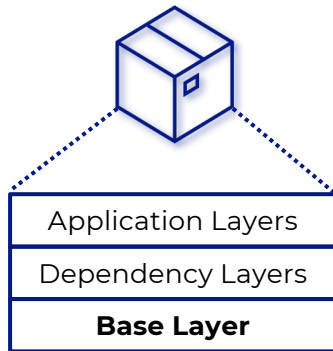
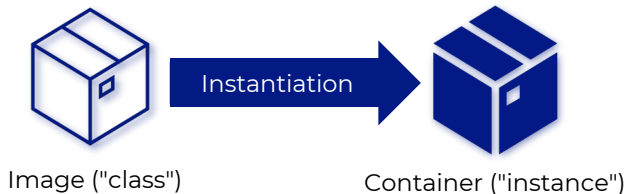
Docker, Kubernetes & AWS

What is an image?

What is an image?

► What is an image?

- Package of optimized layers that includes all the essential data to create a container
- The layers build on top of each other
- An image can even build on top of another image
- Once created, images (and their layers) are immutable (read-only)
- A container can never alter its underlying image!



The command: `docker image`

- ▶ **To download an image from a remote repository:**

- ▶ `docker image pull [name of image]`

- ▶ **To see the history of an image:**

- ▶ `docker image history [name of image]`

- ▶ `docker image history python`

- ▶ `docker image history --no-trunc python`

- ▶ **To show more detailed information about an image:**

- ▶ `docker image inspect [name of image]`

- ▶ **To get a list of all local images, we can use the command:**

- ▶ `docker image ls / docker images`

Docker, Kubernetes & AWS

Tags of images

Tags of images

- ▶ Tags allow us to access different versions of images
- ▶ We can specify the version of an image with a colon, `image:tag`
 - ▶ `node:latest`
- ▶ When not specifying a tag, the tag `:latest` will be used
 - ▶ The image `python:latest` is equivalent to `python`
- ▶ **A different version of an image:**
 - ▶ **For the python image:**
 - ▶ Can just refer to a different node version (`python:2`)
 - ▶ But can also switch the available modules, or the underlying OS
 - ▶ `python:latest`
 - ▶ `python:slim`
 - ▶ For other packages, it can even completely switch the available features
 - ▶ (such as: PHP without webserver, PHP with webserver,...)

Docker, Kubernetes & AWS

What exactly did docker run do?

The command: `docker [container] run`

`docker [container]
run -it`

`docker [image] pull`

Downloads an image
(usually from Docker Hub)

`docker [container] create -it`

Builds a container based on an image
(also downloads an image if needed)

`docker [container] start -i`

Executes an existing container

Docker, Kubernetes & AWS

Overview: Linux crash course

Heads-up

- ▶ **In this course:**

- ▶ We'll be working extensively with the Linux terminal
- ▶ In the next lecture, we'll offer you a quiz to test your terminal skills

- ▶ **If you can answer every question with “yes”:**

- ▶ You can skip the crash course
- ▶ And continue with the next chapter

- ▶ **If you answer any question with “no”:**

- ▶ Be sure to watch the crash course

Docker, Kubernetes & AWS

Crash course: Linux shell

Command	Meaning	Action
pwd	print working directory	Prints the current directory
cd	change directory	Change directory to a different (sub-)directory
cd ..	change directory	Change directory to the parent directory
ls	list	List the contents of a directory
mkdir	make directory	Creates a new folder
rm	remove	Delete files or directories
cat	concatenate	Output a text file to the shell
touch		Create a new file
clear		Clear the current view
exit (or Ctrl+D)		Quit the current shell
apt update		Update package definitions (on Debian / Ubuntu)
apt install nano		Install the editor nano (on Debian / Ubuntu)

Docker, Kubernetes & AWS

How to manage containers / running a server

Managing containers / running a server

- ▶ **In this chapter:**

- ▶ We will start a first web server within a docker container
- ▶ We will run this container in the background (detached mode)

- ▶ **After that, we will explore:**

- ▶ Stopping and starting of an existing container
- ▶ Showing information about all running containers

- ▶ **Debugging:**

- ▶ How to launch a shell in a running a container
- ▶ How to view logs of this container

Docker, Kubernetes & AWS

Heads-up: IP-addresses, ports, ...



115.235.253.146

:80

:443

IP addresses vs. port numbers

- ▶ **IP address:** Refers to a specific device in a network
 - ▶ IPv4, e.g. 115.235.253.146 (4x decimal numbers, 0-255)
 - ▶ IPv6, e.g. 2001:0db8:85a3:0000:0000:8a2e:0370:7334 (8x hexadecimal numbers)
- ▶ **Port:** Refers to a process on a device in a network
 - ▶ 65,535 port numbers in total
 - ▶ Most protocols have default ports:
 - ▶ **Port 80:** HTTP
 - ▶ **Port 443:** HTTPS
 - ▶ **Port 22:** SSH
 - ▶ **Important:**
 - ▶ Each port can only be opened once
 - ▶ If we try to use a port that already exists, a port collision occurs
 - ▶ Certain ports might require additional privileges (usually: Port 1 to 1023)
- ▶ **IP & port:** We can write them in combination (e.g. 115.235.253.146:80)

Accessing our own computer: localhost

- ▶ To access ourselves, we can use the IP address: `127.0.0.1`
- ▶ This IP address is an exception, it always stands for loopback / localhost / ...
- ▶ **Example:**
 - ▶ <http://127.0.0.1>
 - ▶ Would access our own computer, on port 80 (default port of http)
 - ▶ <http://127.0.0.1:8888>
 - ▶ Would access our own computer, on port 8888
 - ▶ <http://localhost:8888>
 - ▶ Instead of writing `127.0.0.1`, we can also just write `localhost` - it will automatically resolve to `127.0.0.1`

Docker, Kubernetes & AWS

What is a webserver?

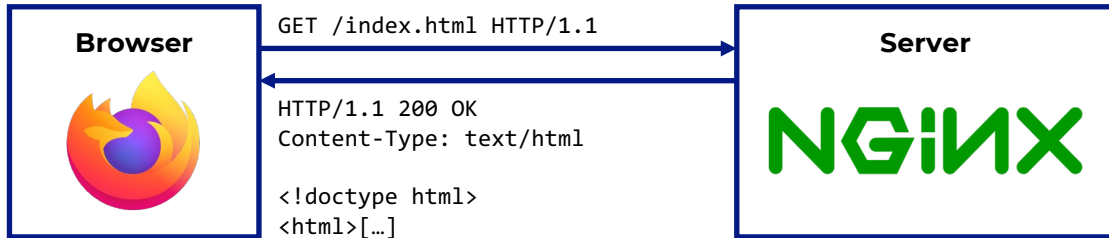
What is a webserver?

- ▶ **A web server:**

- ▶ A software or a hardware system that delivers web content to users over the internet

- ▶ **Key functions:**

- ▶ Handles HTTP requests and sends back responses
- ▶ Example: Serving HTML pages, serving downloadable files, or APIs





Docker, Kubernetes & AWS

Publishing a port / running nginx



Launching an nginx server

- ▶ Nginx is one of the most popular web server software
- ▶ Its market share is about 34% of all websites
- ▶ How can we launch a web server within a container?
- ▶ **Well, we can just launch a container based on the nginx image:**
 - ▶ `docker run -it nginx`
- ▶ **But somehow, we can't access the web server:**
 - ▶ <http://localhost>
- ▶ **Why is that?**
 - ▶ **It's not a bug, it's a feature!**

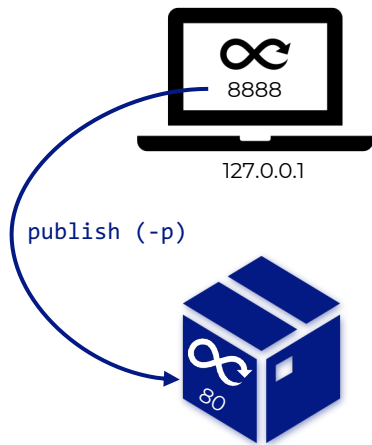
How to enable access to container ports

► Using ports within a container:

- Docker containers are designed to be isolated from each other (and from the host environment)
- Inside their isolated environment, containers can open any port
- By default, these internal ports are not connected to any ports on the host
- This makes them inaccessible from your computer or network

► Publishing a port:

- This allows our host machine to access a port on a container we can use the `-p` flag (or `--publish`)
- For this we map a port on your host to a port on the container
- `docker run -it -p [host-port]:[container-port] [image-name]`



Launching an nginx server



- ▶ **How can we launch a nginx properly?**
 - ▶ `docker run -it -p 8888:80 nginx`
 - ▶ `[host-port]:[container-port]`
 - ▶ nginx is configured to always listen to port 80
 - ▶ Thus, we can choose a port from our host, i.e., 8888
 - ▶ ... and map it to port 80 of the container
- ▶ **After this, we can access nginx through our Browser:**
 - ▶ <http://localhost:8888>

Docker, Kubernetes & AWS

Running a container in the background, showing logs

How to run a container in the background?

- ▶ **First, a heads-up:**

- ▶ A container is only kept alive as long as the main process is running
- ▶ If the main process terminates, the container will be stopped as well

- ▶ **This means:**

- ▶ For a container to keep running in the background, it needs a main process
- ▶ This main process needs to keep running

- ▶ **To start a container in the background:**

- ▶ **When using `docker run`:**

- ▶ We need to add the flag `-d` (`--detach`)
- ▶ In this case, we can omit `-it`

- ▶ **To show the logs:**

- ▶ `docker container logs`
- ▶ `docker logs`

Heads-up

- ▶ **What did -it mean?**

- ▶ **-t:** We want to attach a terminal to the container
- ▶ **-i:** Interactive, meaning the container should be allowed to receive user input

Docker, Kubernetes & AWS

Showing information about containers

How to show info about a container

- ▶ So now that we can launch containers, how can we inspect them?
- ▶ **For this, we can use the docker container command:**
 - ▶ `docker container ls -as`
 - ▶ **The flag -a (or --all):**
 - ▶ Will list all containers
 - ▶ Without it, it would only list the currently active ones
 - ▶ **The flag -s (or --size):**
 - ▶ Will also print the size of each container
- ▶ **We can also show statistics about containers:**
 - ▶ `docker container stats`
- ▶ **We can also inspect an individual container:**
 - ▶ `docker [container] inspect [name]`

Docker, Kubernetes & AWS

Naming a container

How to name a container

- ▶ **Each container can be accessed through either:**

- ▶ The container id (long form, or shortened)
- ▶ The container name
- ▶ If we don't specify a name, a name will be generated automatically
- ▶ The names of containers need to be unique

- ▶ **When launching, we can specify the name:**

- ▶ Use the `--name` flag when executing `docker run`
 - ▶ `--name [new container-name]`
- ▶ `docker run --name ubuntu-test -it ubuntu`

Docker, Kubernetes & AWS

Managing containers

How to stop and start a container

- ▶ If a container is running...
- ▶ ... how can we stop the container?
- ▶ **For this, we can use the following command:**
 - ▶ `docker container stop [container-name or container-id]`
 - ▶ `docker stop [container-name or container-id]`
- ▶ **To start the container again:**
 - ▶ `docker container start [container-name or container-id]`
 - ▶ `docker start [container-name or container-id]`

How to delete a container

- ▶ Once we no longer need a container, we can delete it
- ▶ **The command:**
 - ▶ `docker container rm [container-name or container-id]`
 - ▶ `docker rm [container-name or container-id]`
- ▶ **Important:**
 - ▶ We can only delete containers, that are not currently running
- ▶ **Tip:**
 - ▶ We can even use a shell expansion to delete all inactive containers:
 - ▶ `docker rm $(docker container ls -aq)`

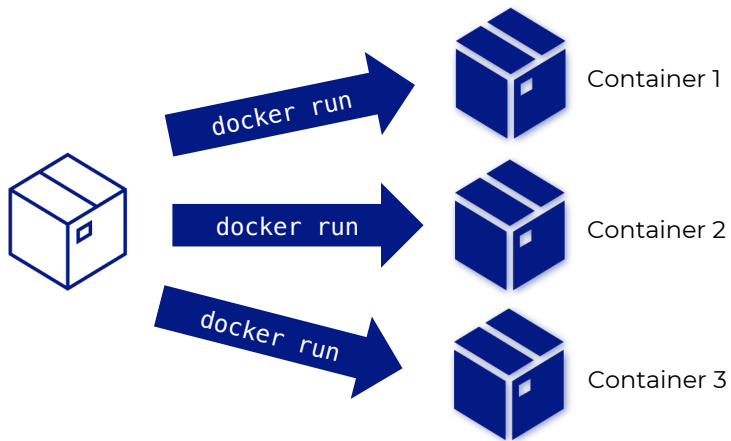
Docker, Kubernetes & AWS

Automatically deleting a container

What docker run does

- ▶ **Important heads-up:**

- ▶ `docker run` always creates and starts a new container



How to delete containers automatically

- ▶ Sometimes, we just want to test a different container
- ▶ **If we just want to test something quick:**
 - ▶ We can add the flag `--rm` to it:
 - ▶ This will ensure that the container will be deleted once it has been stopped
 - ▶ `docker run -it --rm ubuntu`
 - ▶ `docker run -it --rm fedora`

Docker, Kubernetes & AWS

Launching a different program in a container

How to launch programs in containers

- ▶ Most images specify a program that will be launched by default

- ▶ **Example:**

- ▶ For the image ubuntu: `bash`
 - ▶ And for the image python: an interactive python shell

- ▶ **But we can change this default!**

- ▶ `docker container run [options] [image] [command]`

- ▶ **Why would we want to do this?**

- ▶ Let's say we need a tool NumPy in our python container

- ▶ **We could launch a python container with a bash:**

- ▶ `docker container run -it python bash`

- ▶ **Install NumPy in it:**

- ▶ `pip install numpy`
 - ▶ And then launch python: `python`

Docker, Kubernetes & AWS

How to debug a running container

How to launch programs in running containers

- ▶ **If a container is running, we can also launch additional programs in it:**

- ▶ `docker container exec [options] [container] [command]`
- ▶ This can be useful for various things, such as creating files, or downloading additional tools, debugging a running container,...

- ▶ **This would allow us another approach:**

- ▶ **We could just start the python container normally:**

- ▶ `docker container run --name my-python -it python`

- ▶ **And then, in a different shell, launch a bash in this container:**

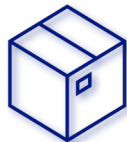
- ▶ `docker container exec -it my-python bash`
- ▶ And in this shell, we would then install NumPy:
 - ▶ `pip install numpy`

Docker Deployment Bootcamp

Why did we need -it during docker container create

The command: docker create

- ▶ **The command docker [container] create:**
 - ▶ Creates a container
 - ▶ If the image doesn't exist locally, it will try to download it
- ▶ **This container needs to be configured during creation:**
 - ▶ If we want the container to have a terminal attached to it, we need to add the flag `-t / --tty`
 - ▶ In addition, we also add the flag `-i` (or `--interactive`), to allow this container to receive user input
 - ▶ We also need to specify port mappings (`-p`), mounts (`-v`), environment variables (`-e`),... in this step
 - ▶ Once a container has been created, these options are immutable



docker create





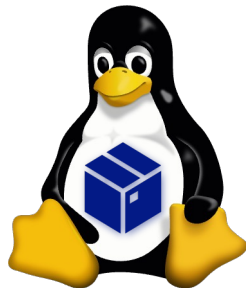
Docker, Kubernetes & AWS

Exercise:

Changing the nginx start page

Exercise

- ▶ **Launch an nginx container (or re-use an existing one)**
- ▶ **Then:** Change the default page displayed when nginx is accessed in a browser
- ▶ **To do this:**
 - ▶ **Navigate to the document root folder:**
 - ▶ `/usr/share/nginx/html`
 - ▶ **In this folder, edit the `index.html` file:**
 - ▶ You might need to install a text editor for the CLI of your choice (such as: nano)
 - ▶ In the next lecture, we'll offer a crash course about nano
 - ▶ **Make the following changes:**
 - ▶ Change the background color by replacing `<body>` with:
 - ▶ `<body style="background-color: lightgray">`
 - ▶ Update the heading by replacing `<h1>Welcome to nginx</h1>` with:
 - ▶ `<h1>Welcome to nginx in docker</h1>`
 - ▶ **Open the page in a browser of your choice and explore the result!**



Docker, Kubernetes & AWS

Day 3:

Docker on a Linux Server

Day 3: Docker on a Linux Server



- ▶ **In this chapter:**

- ▶ We want to explore how to run Docker on a Linux server
- ▶ We will run an Apache web server, through Docker!

- ▶ **What will we explore?**

- ▶ **We will explore the basics of server management (on Ubuntu):**

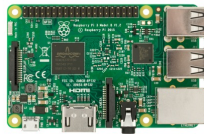
- ▶ How to update software?
- ▶ How to install software?
- ▶ How can we install docker?
- ▶ How can we connect to a server (SSH)?
- ▶ Keeping our server up to date is required...
- ▶ ... even when using docker for the rest!

- ▶ **Docker:**

- ▶ We will explore how to ensure a container is launched on boot

Heads-up

- ▶ In the next chapter, we'll also offer additional guides on how to run docker on other platforms:
 - ▶ Raspberry Pi
 - ▶ Ubuntu in a VirtualBox
- ▶ These may be options if you do not want to follow along with AWS



Docker, Kubernetes & AWS

Before choosing where to run our server: What is AWS & EC2?

Important

- ▶ For this chapter, we do not have to use AWS
- ▶ We could rent a server from any hosting provider
- ▶ **However:**
 - ▶ As we'll be using specific AWS features later in this course, renting a server on AWS allows us start exploring it now already
- ▶ **Additionally, I will provide other guides:**
 - ▶ How to run Docker on a virtual machine
 - ▶ How to run Docker on a Raspberry Pi

What is AWS?

- ▶ **AWS: Amazon Web Services**

- ▶ Cloud computing platform from Amazon
- ▶ Offers over 200 services:
 - ▶ Computing, serverless computing
 - ▶ Storage, long-term storage
 - ▶ Various databases, caches

- ▶ **Global infrastructure:**

- ▶ Regions: Geographic areas around the world
- ▶ Availability zones: Isolated locations within regions

- ▶ **Security:**

- ▶ IAM (Identity and Access Management): Controls access to AWS services (quite granular control possible)

- ▶ **Pricing:**

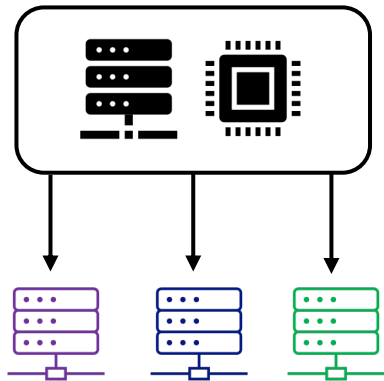
- ▶ Usually, pay-as-you-go, but cheaper if we reserve capacity



What is EC2?

- ▶ **EC2: Elastic Compute Cloud**

- ▶ A core part of AWS
- ▶ You can use it to create and manage virtual servers (instances) in the cloud
- ▶ **Many different instances available, for different workloads:**
 - ▶ Various configurations for CPU, memory and storage
 - ▶ We can choose, which operating system should be installed
- ▶ **As of recording:**
 - ▶ Some servers are available for free (free tier of AWS)



Docker, Kubernetes & AWS

Where to run docker?

Where to run docker?

► Server on Amazon AWS:

- This will be a real server in the cloud
- Additional charges might occur
(but free tier might be available, subject to change)
- This server will then be accessible from the internet



► Raspberry Pi:

- A small computer based on the ARM CPU architecture, that runs Linux
- Perfect for small home projects or home automation
- Disadvantages: Not as fast as a normal computer (but fast enough), additional costs



► Virtual Machine:

- It's a software that we can install on your computer
- It will then provide a virtual computer to our Linux system
- Advantage: No additional costs, you can test a Linux system quite easily

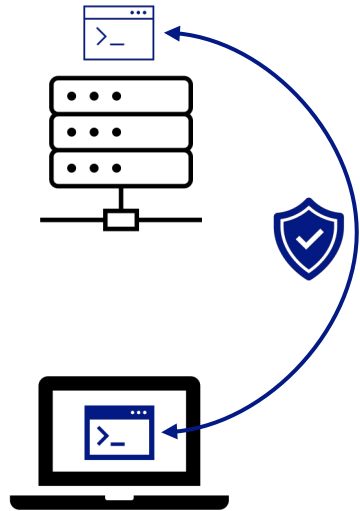


Docker, Kubernetes & AWS

What is SSH?

What is SSH?

- ▶ **secure shell**: a cryptographic network protocol
- ▶ Encrypts all data transmitted between the client and server, ensuring confidentiality and integrity
- ▶ The most common use case for ssh is to **securely** access and manage a remote server through the terminal
- ▶ **For this, we need the following:**
 - ▶ **We need an SSH server:**
 - ▶ This is what needs to be installed on the server
 - ▶ On servers that we rent, it is already pre-installed
 - ▶ It will accept incoming connections and allow us to connect to it
 - ▶ **We also need an SSH client:**
 - ▶ Most operating systems already ship with one
 - ▶ Usually, we can just use the program `ssh` (on the terminal), to connect to a remote server
 - ▶ Even on Windows, we don't need to install anything!



How to access a remote server with SSH

- ▶ When connecting to a remote server, we login as a user of our choice

- ▶ **Example:**

- ▶ `ssh jannisseemann@raspberrypi.local`

- ▶ It might then ask us for the password to continue with the login

- ▶ **Quite often:**

- ▶ The ssh server might be running on a different port
 - ▶ This prevents random attacks from flooding our logfiles
 - ▶ In this case, we need to specify the different port:

- ▶ `ssh -p 222 jannisseemann@raspberrypi.local`

- ▶ **On AWS servers:**

- ▶ It is configured to run on the default port (22)
 - ▶ But it doesn't allow password logins
 - ▶ We need to create a private / public key pair for authentication!

SSH: How to generate a key pair



▶ What is a key pair?

- ▶ We can imagine this as a combination of a key and a lock
- ▶ ... just in a cryptographic way
- ▶ **Key (technical term: private key):**
 - ▶ Secret, we should never share this with anyone
- ▶ **Lock (technical term: public key):**
 - ▶ This can be public, everyone can know about this
 - ▶ It can be used to identify if the private key was used to sign a message / to authenticate a user

▶ We can generate such a key pair:

- ▶ `ssh-keygen -t ed25519`
 - ▶ The flag `-t` specifies the type of key to generate
 - ▶ `ed25519` is a highly secure and efficient cryptographic algorithm
- ▶ This key pair will then be stored in the folder `~/.ssh`

Docker, Kubernetes & AWS

Users on Linux systems

Users on Linux systems



- ▶ In Linux, users can be categorized into three general categories:

- ▶ **Regular users:**

- ▶ They have access to their own files and directories
 - ▶ They cannot perform administrative tasks or access other user's files without permission

- ▶ **Superuser (root):**

- ▶ The superuser (root) has unrestricted access to the entire system (including files in the home directories of regular users)
 - ▶ Can add / remove users, install software
 - ▶ Can change the configuration of the system

- ▶ **System accounts:**

- ▶ They are responsible for running background tasks on your system (such as: webserver, database,...)
 - ▶ They don't have a home directory

Elevating privileges: sudo

- ▶ If we want to temporarily elevate our privileges...
... we can put a `sudo` in front of our command
- ▶ **Example:**
 - ▶ `sudo ls /root`
 - ▶ Usually, we could not access this directory (only `root` could)
 - ▶ But our regular user is allowed to use the `sudo` command to temporarily elevate its privileges
 - ▶ This allows us to access it anyway!
- ▶ **But be careful:**
 - ▶ Always make sure you understand what a command does
 - ▶ Especially when `sudo` is involved
- ▶ **Example:**
 - ▶ **Important: DO NOT EXECUTE!**
 - ▶ `sudo rm -rf /etc`

Docker, Kubernetes & AWS

Server management: Keeping the system up to date

Why is this important?

- ▶ **Even when running Docker on a server:**

- ▶ It is our responsibility to keep the underlying operating system up to date
- ▶ We thus must explore:
 - ▶ How to keep our system up to date
 - ▶ How to install packages

Package management on Linux

► Package management:

- Most Linux distributions offer a centralized way to install software
- This process is called package management
- This is an enormous benefit of many Linux systems
- As it helps us to keep our system up to date

► Also:

- A lot of applications no longer need to include their own updater
(Example: Firefox, Chrome,...)

► How does it work?

- Our system connects to centralized repositories
- They provide a list of available packages
(including available versions, and their dependencies)
- This list can then be used to install updates, or install additional tools



Ubuntu: How to update software with apt

- ▶ On Debian-based distributions (such as Ubuntu), we can use the tool `apt` to keep our system up to date
- ▶ **It provides several commands for us:**
 - ▶ **`apt update`:**
 - ▶ Refreshes the list of available packages
 - ▶ We should run this before doing anything else with `apt`
 - ▶ **`apt upgrade`:**
 - ▶ Runs a small upgrade of our system
 - ▶ Small means: Upgrades existing packages (and when using `apt`, also allows the install of additional dependencies)
 - ▶ **`apt full-upgrade` or `apt dist-upgrade`:**
 - ▶ Runs a large upgrade of our system
 - ▶ Large means: Upgrades existing packages, and removes / installs additional packages (dependencies)

Docker, Kubernetes & AWS

Server management:

Installing software using apt

| Hello from Ubuntu! |

=====

\\

^ ^

(oo)_____
(__)\\)\\/
||-----w||
|| ||

Ubuntu: How to install / remove software

- ▶ **To install / remove software:**

- ▶ **apt install [package-name]**

- ▶ This will install an additional package on our system

- ▶ **Example:** apt install cowsay

- ▶ **apt remove [package-name]**

- ▶ Removes a package from our system

- ▶ **apt autoremove**

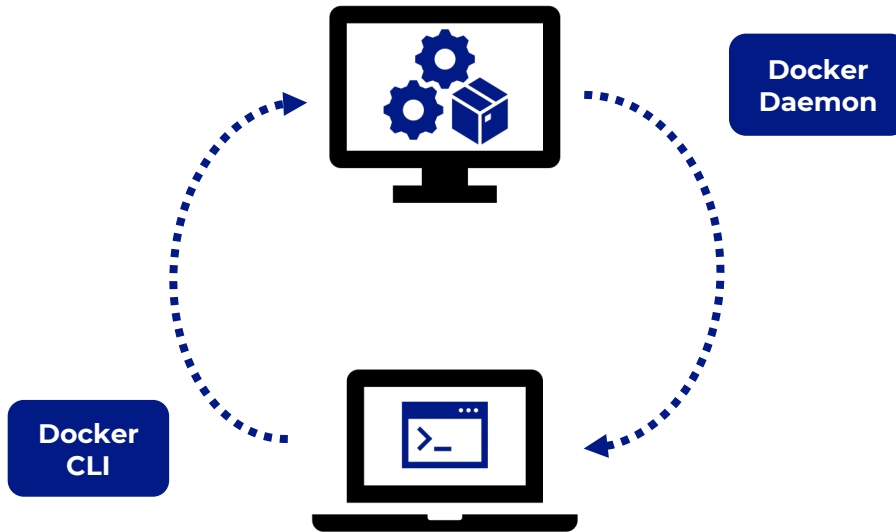
- ▶ Removes packages that are no longer needed

- ▶ You can run this if there're any errors during an upgrade or
full-upgrade

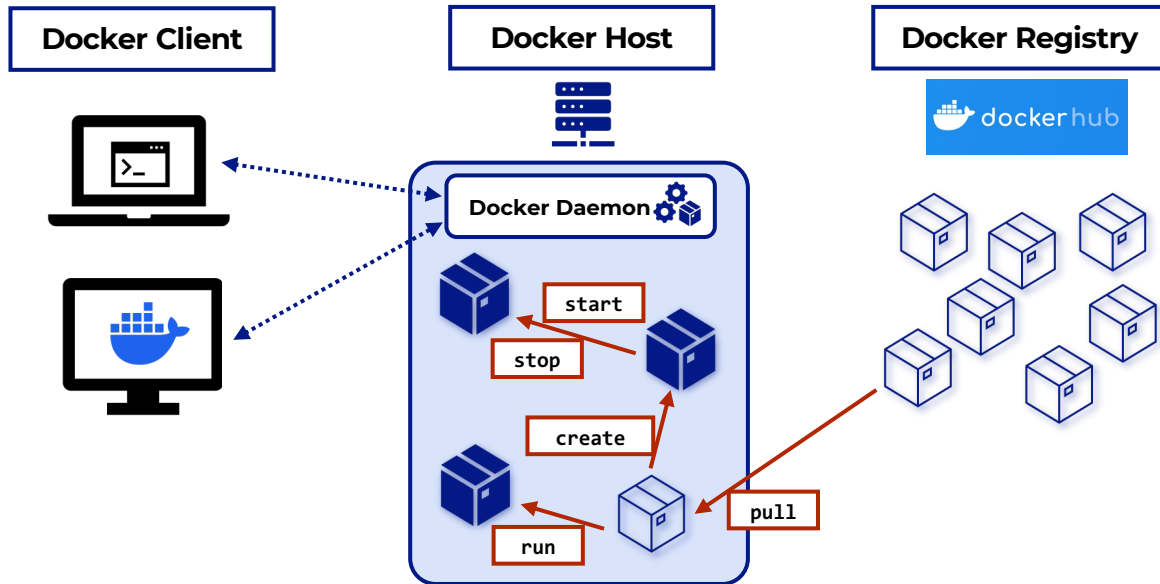
Docker, Kubernetes & AWS

Docker Architecture: Client Host and Registry

Client ↔ Server architecture of Docker



The Docker architecture



Docker Host

- ▶ **Docker Host:**

- ▶ **Runs the Docker Daemon:**

- ▶ When launching Docker Desktop, the Docker Daemon will be started automatically
 - ▶ **Daemon:** *a process that runs in the background and provides services for other processes*
 - ▶ It is responsible for managing the containers - it builds them, launches them and manages them
 - ▶ We can use `docker info` to show additional information about this process



Docker Client

- ▶ **Docker Client:**

- ▶ Communicates with the Docker Daemon
- ▶ Utilizes the Docker API
- ▶ It allows us to access Docker from the command line

- ▶ **Important:**

- ▶ Docker Client and Docker Daemon can be run on the same physical machine - but they don't have to!



Docker, Kubernetes & AWS

Installing Docker

Installing Docker

- ▶ **Unfortunately:**

- ▶ The version of Docker that is included in the default repositories is outdated

- ▶ **We thus can't just run this to use Docker:**

- ▶ (will install an outdated version): `sudo apt install docker`

- ▶ **Thus:**

- ▶ **To install Docker, we either have to follow the install guide:**

- ▶ <https://docs.docker.com/engine/install/>

- ▶ **Or we use an automatic convenience script:**

- ▶ <https://get.docker.com/>

- ▶ (not recommended for production)

- ▶ **Once installed, we can just use docker as we're used to it:**

- ▶ `sudo docker run -it hello-world`

Docker, Kubernetes & AWS

Docker:

Restart policy of a container

Configuring the restart policy

- ▶ Docker will not automatically re-launch our container
- ▶ Once the container stops, it will remain stopped (restart policy: no)
- ▶ For a webserver, we might want to deviate from this behavior:
 - ▶ Docker should try to start it automatically, and restart it on failure
- ▶ **But we can change this by setting the restart policy:**
 - ▶ `on-failure[:max-retries]`
 - ▶ In case of an error, the container should be restarted
 - ▶ `always`
 - ▶ The container should always be restarted
 - ▶ (unless we stop it manually, then the container will only be started at the next launch of Docker Daemon)
 - ▶ `unless-stopped`
 - ▶ The container will always be restarted
 - ▶ If we stop the container, it will remain stopped

How to set the restart policy

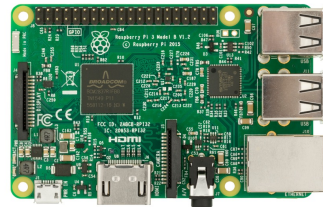
- ▶ **For a new container:**

- ▶ `docker run -d --restart unless-stopped [image-name]`

- ▶ **For an existing container:**

- ▶ `docker container update --restart unless-stopped [container-id / container-name]`

- ▶ `docker update --restart unless-stopped [container-id / container-name]`



Docker, Kubernetes & AWS

Fallback: Self-hosting alternatives to AWS

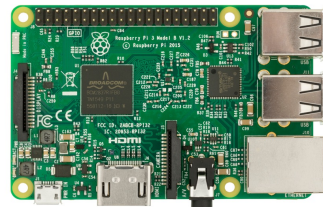
Running Docker somewhere else?

- ▶ **For now, using AWS is technically not required:**

- ▶ We could've also rented a server somewhere else and ran Docker there
- ▶ However, later throughout this course, we'll deploy to specific AWS services - so we might already rent our server from them as well

- ▶ **In this chapter:**

- ▶ We will explore how you can run Docker on a Raspberry Pi, or in a virtual machine
- ▶ This chapter is completely optional, and meant to be skipped by most students
- ▶ However, if you're interested in running docker on your own server – feel free to watch!



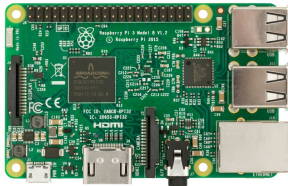
Docker, Kubernetes & AWS

Fallback:

Running Docker on a Raspberry Pi

What is a Raspberry Pi?

- ▶ Single-Board computer: Compact, affordable computer
- ▶ All on a single circuit board
- ▶ **Features:**
 - ▶ Processor: ARM-based CPU
 - ▶ USB, HDMI, Ethernet, Wi-Fi, Bluetooth (varies by model)
 - ▶ GPIO pins: General purpose pins for interacting with hardware (not relevant for us)
- ▶ It's ideal for home automation, DIY projects, or a simple home server
- ▶ **But it is even used in commercial applications:**
 - ▶ Prototyping
 - ▶ IoT devices
 - ▶ Edge computing



How to install an OS on Raspberry Pi?

- ▶ To install a Raspberry Pi OS, we need to have a microSD card
- ▶ Raspberry Pi will boot from this microSD card, and launch the operating system
- ▶ **To install an operating system on the microSD card:**
 - ▶ The easiest way is to use the Raspberry Pi Imager
 - ▶ It will automate everything for us
 - ▶ <https://www.raspberrypi.com/software/>
 - ▶ **Important:**
 - ▶ The microSD card will be overwritten, existing data might get lost
 - ▶ **I highly recommend configuring the OS settings:**
 - ▶ Setting a WiFi password
 - ▶ Enabling public / private key auth through ssh
 - ▶ Since I don't have a display / keyboard to connect to the Raspberry Pi, I won't be able to change these settings later!



Docker, Kubernetes & AWS

Fallback (if AWS doesn't work): Linux in a Virtual Machine (VM)

If you're using macOS

- ▶ **If you have an Intel processor:**

- ▶ You can use VirtualBox
- ▶ You can keep following this guide

- ▶ **If you have an Apple processor:**

- ▶ As of now, VirtualBox might not run properly on your system
- ▶ You might need to use UTM, we'll provide you with a text guide in the next lecture

What is VirtualBox?

- ▶ VirtualBox is a **virtualization software** that allows you to create and run virtual machines (VMs) on your computer
 - ▶ Developed by Oracle Corporation
 - ▶ Free & open-source software
- ▶ **Installation:**
 - ▶ <https://www.virtualbox.org/wiki/Downloads>



Docker, Kubernetes & AWS

Managing files in a container

The goals in this chapter

- ▶ **In the last chapter:**

- ▶ It was incredibly difficult to change the website our webserver was serving

- ▶ **In this chapter, we will answer the following questions:**

- ▶ How are files stored within a docker container?
 - ▶ How can we copy files into / out of a container?
 - ▶ `docker cp`
 - ▶ How can we enable the container to access the filesystem of the host?
 - ▶ Bind mounts
 - ▶ What does the `-v` mean?
 - ▶ `docker run -it -p 3000:80 -v $(pwd):/usr/share/nginx/html nginx`

- ▶ **Last, but not least:**

- ▶ We will apply this knowledge to our server - and serve a real website!

Docker, Kubernetes & AWS

How are files being stored?

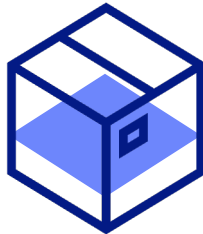
How are files being stored?

► So far:

- We stored files directly in the container
- Files were stored in the writable layer of the container
- This allowed images to be read-only
- If you delete a container, all its data will be lost - but the image can still be used for other containers

► This was quite a feature:

- If we don't deviate much from the image, a container needs almost no (additional) storage!
- This made our containers incredibly efficient!



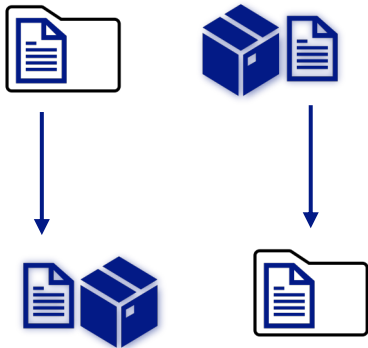
Docker, Kubernetes & AWS

Copying files: docker cp

The command: `docker cp`

- ▶ The command `docker cp` allows us to copy files or folders into a container - or from a container
- ▶ **We need to specify 2 arguments:**
 - ▶ Name or path of the element that should be copied
 - ▶ Directory, in which the element should be copied
- ▶ **To copy data into a container:**
 - ▶ `docker cp [host-file/folder] [container-name/id]:[container-directory]`
- ▶ **To copy data from a container into the host:**
 - ▶ `docker cp [container-name/id]:[container-directory] [host-file/folder]`
- ▶ **Heads-up:**
 - ▶ `docker cp` works even when the container is not running!

1st parameter: File or directory that we want to copy



2nd parameter: Directory where we want to copy it to

Docker, Kubernetes & AWS

(Bind-) mounts in docker

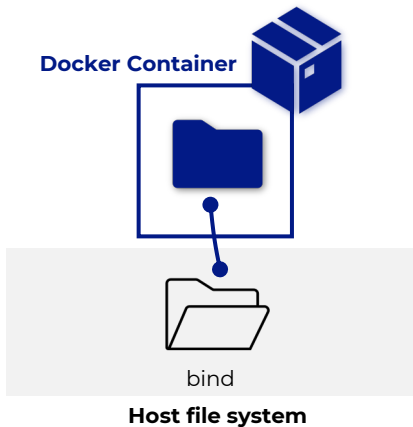
What is a bind mount?

- ▶ **Bind mount:**

- ▶ Maps a directory inside the container to a directory on the host's local file system
- ▶ This allows us to easily access a directory that is stored on the host

- ▶ **Important:**

- ▶ Docker also offers other types of mounts, for example a volume mount - that will be useful when running a database
- ▶ We will get to that at a later point in this course



How do we create a bind mount?

- ▶ We have 2 different ways to define a bind mount
- ▶ **Let's first look at the more complex method:**
 - ▶ `--mount type=bind,source=[host-path],destination=[container-path]`
 - ▶ We want to create a mount (**--mount**)
 - ▶ Specify the type of the mount (**type=bind**)
 - ▶ Provide the host directory path to share with the container (**source**)
 - ▶ Set the target directory path inside the container (**destination**)

The flags: `--mount` vs. `-v`

- ▶ We can also use a shorter way to specify a bind mount
- ▶ **For this, we can use the flag `-v` (or `--volume`):**
 - ▶ `-v [host-path]:[container-path]`
 - ▶ **Important:**
 - ▶ Here, we must provide an absolute path to the local directory
 - ▶ Otherwise, `-v` will understand it as a volume mount (we haven't explored this yet)
- ▶ **But there's also an additional difference:**
 - ▶ `-v` is shorter
 - ▶ If the directory doesn't exist on the host, `-v` will also create a new folder on the host
 - ▶ `--mount` will fail with an error message, if the path does not exist
- ▶ **The syntax with `-v` is more common!**

Docker, Kubernetes & AWS

Read-only bind mounts

Read-only bind mounts

- ▶ We can also create a read-only bind mount
- ▶ **In this case:**
 - ▶ The container is only allowed to read from this mount
 - ▶ If we try to write into this mount (from the container), we will get an error
 - ▶ Changes on the host will still be seen by the container immediately
 - ▶ **How do we create a read-only bind mount?**
 - ▶ `-v [path on host]:[path on container]:ro`
 - ▶ `--mount type=bind,source=[host-path],destination=[container-path],readonly`

Docker, Kubernetes & AWS

Copy vs. bind mount

Copy vs. bind mount: When to use?

- ▶ **If we do have a choice:**

- ▶ **docker cp**

- ▶ Directly copies files/directories into the container's writable layer
 - ▶ Optimized by Docker for speed
 - ▶ We should prefer this in production
 - ▶ But once the setup of a container is done, changes to the data must be copied into the container manually
 - ▶ (usually: better, more explicit)

- ▶ **bind mount:**

- ▶ Very convenient during development
 - ▶ Changes made on the host are automatically visible in the container
 - ▶ This streamlines the update process
 - ▶ But it also creates an overhead: Docker needs to query the host for changes on any access!

Docker, Kubernetes & AWS

Transferring files to the server

How to transfer files to the server

- ▶ **The goal:**

- ▶ Let's say we have finished developing our website locally
- ▶ We want to deploy our website into a docker container that is hosted on our server

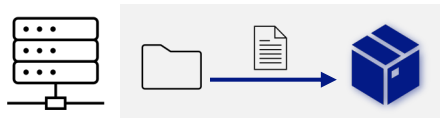
- ▶ **2 steps:**

- ▶ **1.) Transfer files to the server:**

Move the local files to the server



- ▶ **2.) Copy files into the container:** Import the files into the Docker container



Step 1: Copying files to the server

- ▶ **The task:**

- ▶ Copy files to the server
- ▶ How can we do this?

- ▶ **The Secure Copy Protocol (scp)**

- ▶ scp allows us to copy files through an encrypted SSH connection
- ▶ It will use the same username/password as we use for SSH

- ▶ **The syntax:**

- ▶ `scp [source-file/folder] [destination-path]`
- ▶ For changing the port, we can use the `-P` flag
- ▶ Here in our case, we can just use the default SSH port (22)

- ▶ **Example:**

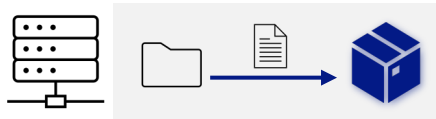
- ▶ `scp -r . ubuntu@13.37.244.236:/home/ubuntu/app`



Step 2: Copying files into the container

- ▶ Now, that the files are on the server, we can just use Docker to copy it into the container:

- ▶ `docker cp`
- ▶ Let's have a look at this!



Docker, Kubernetes & AWS

Overview:

Connecting a Domain

Overview: Connecting a domain

- ▶ **In this chapter:**

- ▶ We will purchase a domain... and connect it to our server

- ▶ **This chapter is required:**

- ▶ If you want to deploy your application on a publicly accessible website, or have it delivered through https
 - ▶ Also, in the later chapters we'll come back to this

- ▶ **But don't worry:**

- ▶ As of recording, a domain can be purchased for as little as 3 dollars for a whole year

Docker, Kubernetes & AWS

Before buying a domain: How does it work? DNS!

DNS protocol

- ▶ **DNS: Domain Name System**

- ▶ Translates domain names to IP addresses
- ▶ Facilitates human-readable access to websites and services

- ▶ **The idea:**

- ▶ Instead of typing in the IP-address, we just want to type in "google.com", and our computer should do the rest

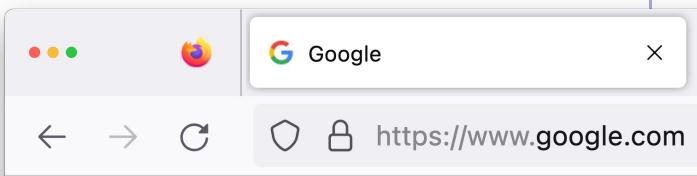
The DNS process

- ▶ **The problem:**

- ▶ How does our Browser know to which IP address it should connect to?

- ▶ **Step 1:**

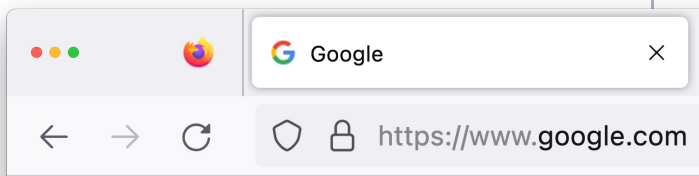
- ▶ Local cache: The browser has its own cache and stores a mapping between Domains and their IP addresses
 - ▶ If the IP can't be found there, the browser asks the operating system to resolve the domain name
 - ▶ The operating system also has a local cache. If the IP can be found in it, we're done!
 - ▶ If not, we need to continue the process



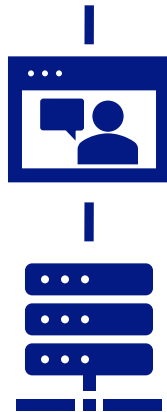
The DNS process

► Step 2: DNS resolver

- They help us resolve the DNS entries from a domain
- Usually provided by our Internet Service Provider (ISP)
- It will check its cache (resolver cache)
- If it can't find the IP address in its cache, it needs to resolve it
- How does it do it?

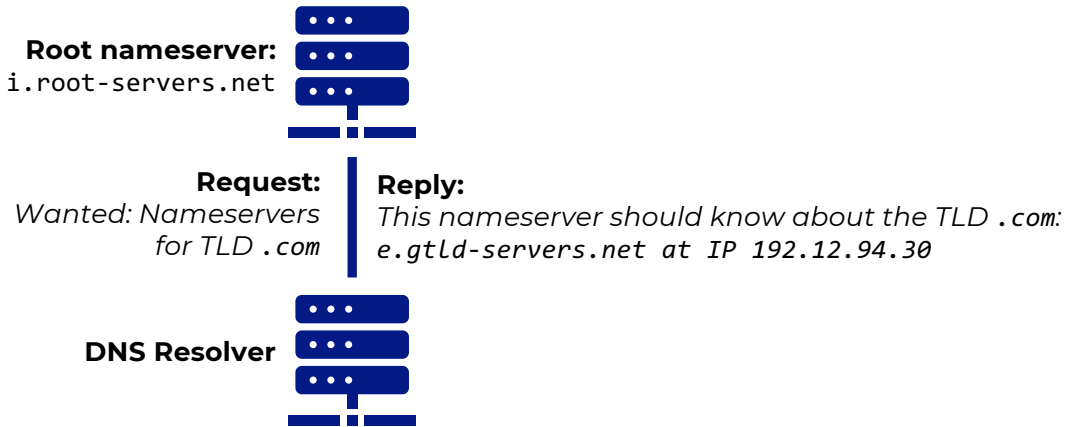


DNS Resolver



DNS: Resolving

- ▶ If the DNS resolver needs to resolve a Domain name, it will reach out to one of 13 root nameserver (at random)
- ▶ They are labeled with letters from A to M



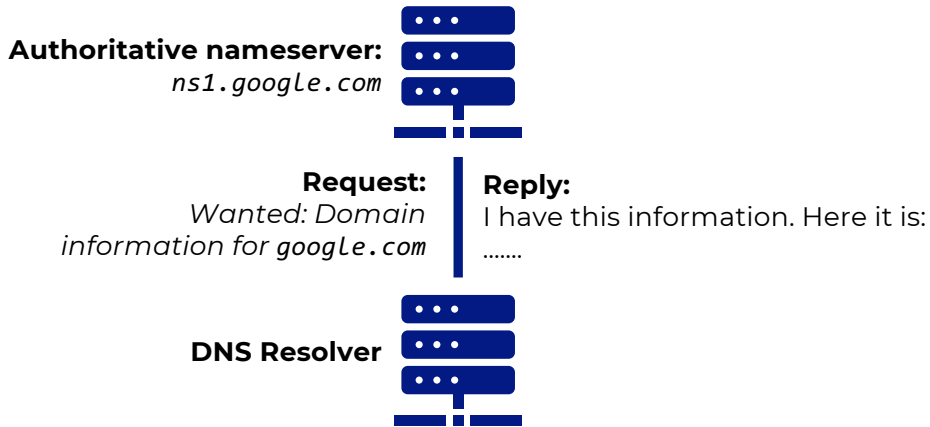
DNS: Resolving

- ▶ So, now we have the TLD nameserver...
- ▶ How does the resolving continue?



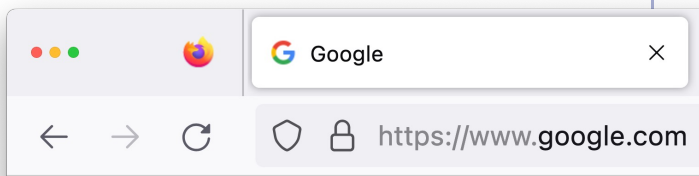
DNS: Resolving

- ▶ Now we got the authoritative nameserver for google.com
- ▶ Let's now continue!



The DNS process

- ▶ Now, the DNS resolver of our ISP has the DNS information
- ▶ And can now send this to our operating system
- ▶ And the operating system will give this information to our browser
- ▶ We can now finally send the request to `google.com` 😊



Docker, Kubernetes & AWS

Type of DNS records

Type of DNS records

▶ Common DNS record types

- ▶ A: Maps a domain name to an IPv4 address
- ▶ AAAA: Maps a domain name to an IPv6 address
- ▶ CNAME: This domain is an alias for another domain name
(which then may need to be resolved after)
- ▶ MX: Specifies mail servers for a domain
- ▶ TXT: Additional, text information
- ▶ NS: Lists authoritative name servers for a domain

▶ We can list the received DNS entries with the following command:

- ▶ *(on ubuntu systems, the package `dnsutils` must be installed)*
- ▶ `host <server>`
- ▶ `host -a <server>`
- ▶ **Example:**
 - ▶ `host -a google.com`

Docker, Kubernetes & AWS

[Optional]: Buying and configuring a domain

Buying and configuring a domain

- ▶ **Goal: We now want to purchase and configure a domain**
 - ▶ At this stage, we could use any domain broker of our choice
- ▶ **However, as we're already using AWS in this course:**
 - ▶ We will use Amazon Route 53 for this
 - ▶ This is the domain broker provided by AWS
 - ▶ It allows us to easily purchase a domain
- ▶ **After purchasing, we need to configure the DNS:**
 - ▶ DNS: Domain Name System

www.example.com



93.184.216.34

Docker, Kubernetes & AWS

[Optional]

Enabling https

Important

- ▶ **This lecture will only work if:**
 - ▶ You have purchased a domain
 - ▶ The server is publicly accessible via this domain
- ▶ **Both of those things are required!**

What is https?

- ▶ **The protocol https**

- ▶ Stands for http over TLS

- ▶ **HTTP**

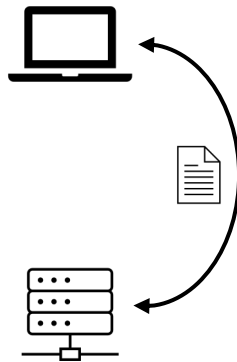
- ▶ **H**yper **T**ext **T**ransfer Protocol
 - ▶ A protocol used to transfer websites and data across the internet

- ▶ **TLS**

- ▶ **T**ransport **L**ayer **S**ecurity
 - ▶ A cryptographic protocol designed to provide secure communications
 - ▶ It first performs a "handshake" to establish an encrypted connection between parties

- ▶ **Thus**

- ▶ **https** enhances HTTP by providing a secure layer of encryption



The webserver: caddy

- ▶ So far, we were using the images `httpd` (or `nginx`) for serving our website
- ▶ **For those webserver:**
 - ▶ We would have to configure `https` manually
 - ▶ To properly implement this, we would have to connect multiple containers together (like: `httpd` + `certbot`,...)
 - ▶ We will get to that later in this course (in an optional bonus)
 - ▶ But for this, we must have explored how to connect multiple containers together (`docker compose`)
- ▶ **But for now:**
 - ▶ We want to use a webserver that does all of this for us
 - ▶ And that allows us to not have to worry about the setup procedure
 - ▶ Luckily, all of this is already implemented in caddy!

Docker, Kubernetes & AWS

Overview:

AppRunner and Images

Overview: Deployment to AppRunner

- ▶ We now want to deploy a docker container to AppRunner
- ▶ This will allow us to run a web server in production
- ▶ **In this chapter (part 1):**
 - ▶ We will explore what AppRunner is
 - ▶ And launch a web server there
- ▶ **In addition (part 2):**
 - ▶ We will start learning how we can our own images
 - ▶ ... that we will eventually deploy to App Runner as well
(in a later chapter)

Docker, Kubernetes & AWS

What is AppRunner (AWS)?

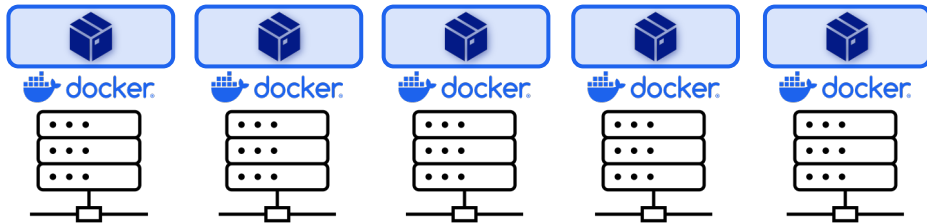
Why do we need AWS App Runner?

- ▶ **So far:**

- ▶ We had to manually create a server to then run docker on it

- ▶ **This had a few limitations:**

- ▶ We had to manage everything manually (https, domain,...)
 - ▶ And if we had to scale, we also would have had to manage it manually (to add additional instances)
 - ▶ How about we try a managed service?



Managed services vs self-hosting

Managed services (AWS)	Self-hosted services
• AWS handles everything	• We run our own server and install all the required services
• Scaling happens automatically: We don't need to scale it manually	• We need to manually manage scaling • This can take additional time
• We most pay per use or per utilization, plus an hourly fee • (+ traffic costs, ...)	• We are charged by the hour for server time • (+ traffic costs, ...)
• Can be significantly cheaper when we're only sporadically using a server	• Tends to be more budget-friendly if we use the service heavily, but we've got to keep it running smoothly

What is AWS App Runner?

- ▶ **AWS App Runner:**

- ▶ A streamlined environment for deployment that allows us to run stateless docker containers

- ▶ **Stateless:**

- ▶ Does not store any data in the container
 - ▶ No database, no other files
 - ▶ All data must be stored outside of the container
 - ▶ Automatically manages load balancing and provisioning for us
 - ▶ Can automatically scale our service
- ▶ **Let's launch an AppRunner container!**



Docker, Kubernetes & AWS

Dockerfiles: Deploying our own images

Dockerfiles: Overview

- ▶ **So far:**

- ▶ We were able to launch a container based on a pre-defined image

- ▶ **But:**

- ▶ How can we create our own custom image?
- ▶ And then derive containers from this image?
- ▶ This would then allow us to easily deploy our service / deploy to AppRunner
- ▶ **For this:** We need to explore Dockerfiles

- ▶ **The roadmap now:**

- ▶ What is a Dockerfile?
- ▶ What are the most important commands?
- ▶ How does the command `docker build` work?
- ▶ We will create an image for a custom NodeJS application
- ▶ ... and publish it to our own repository (next chapter)!

Docker, Kubernetes & AWS

A first Dockerfile, docker build

Building custom images: docker build

- ▶ If we now want to turn this Dockerfile into an image, we need to use the the command:
 - ▶ `docker build [build-context-path]`
- ▶ During the build process, the entire contents of the specified build context directory are sent to the Docker Engine (!!!)
- ▶ **Example:**
 - ▶ Usually, we first navigate into the correct folder in the terminal
 - ▶ And then we can just execute in our current folder:
 - ▶ `docker build .`
 - ▶ We can also use `-t` (or `--tag`) to give our image a tag:
 - ▶ `docker build -t my-app .`



`docker build`



Container Creation Pipeline



Docker, Kubernetes & AWS

Copying files into the image:

COPY & ADD

The COPY & ADD commands

► COPY

- Copy files and directories from a specified source into the filesystem of the Docker image

► **Syntax:** COPY [source-path] [destination-path]

► **Example:** COPY . /app

► ADD

- Provides all capabilities of COPY, but also adds additional features:

- Fetches data from remote URLs

- Extract archives like gzip, tar, bzip2 (but not: zip) automatically

► **Syntax:** ADD [source-path] [destination-path]

► **Example:** ADD <https://example.com/example.tar.gz> /tmp

► Best practice:

- Prefer COPY over ADD, unless you need the features of ADD
- Both will add a new layer to your image

Docker, Kubernetes & AWS

Configuring the container: The CMD command

The CMD command

- ▶ **CMD**

- ▶ Specifies the default command to execute when the container starts
- ▶ Only the last CMD will be taken into effect
- ▶ If no CMD is provided, Docker uses the CMD from the parent image

- ▶ **Syntax**

- ▶ **Exec form (preferred):**

- ▶ CMD ["executable", "param1", "param2"]

- ▶ **Shell form (if we need a shell):**

- ▶ CMD command param1 param2
 - ▶ Executes the command within a shell

- ▶ **Examples:**

- ▶ CMD ["python3", "-m", "http.server"]
- ▶ CMD ls *
- ▶ CMD echo "Hello Container!"

Docker, Kubernetes & AWS

Configuring the container: The ENTRYPOINT command

The ENTRYPOINT command

- ▶ **ENTRYPOINT**

- ▶ Like CMD, it defines the default application that executes when the container starts
- ▶ However, ENTRYPOINT parameters are not overridden by the CLI arguments from `docker run`

- ▶ **Syntax:**

- ▶ **Exec Form:** `ENTRYPOINT ["executable", "param1", "param2"]`
- ▶ **Shell Form:** `ENTRYPOINT command param1 param2`

- ▶ **Example:**

- ▶ `ENTRYPOINT ["node", "--version"]`

- ▶ **Best practices:**

- ▶ We should prefer ENTRYPOINT for containers that operate as stand-alone executables

- ▶ **If both ENTRYPOINT and CMD are present:**

- ▶ CMD will be passed as parameters to the program specified in ENTRYPOINT

Docker, Kubernetes & AWS

Running a webserver / EXPOSE

The EXPOSE command

- ▶ **EXPOSE**

- ▶ Serves as a documentation for the ports that the container will be using
- ▶ It does not automatically publish ports (*usually**)

- ▶ **Syntax:**

- ▶ EXPOSE [PORT]
- ▶ EXPOSE [PORT]/[PROTOCOL]

- ▶ **Examples:**

- ▶ EXPOSE 80/tcp

- ▶ *Automatically publishing ports:

- ▶ Using the -P flag with `docker run` will publish all ports specified through EXPOSE instructions to random ports on the host machine
- ▶ However, it's usually best to manually publish ports (-p)

Docker, Kubernetes & AWS

Ignoring files / folders: `.dockerignore`

The .dockerignore file

► Problem

- When copying folders into an image (COPY or ADD), it's easy to accidentally copy unnecessary files

► Solution: .dockerignore

- A (hidden) text file with a list of files and folders that should be excluded
- It must be placed in the top-level folder of the build context

► We can also use wild card patterns:

- data* - Excludes all files and folders that start with "data" in the root directory
(e.g., data.txt, database/)
- *data - Excludes all files and folders that end with "data" in the root directory
(e.g., collected_data, more-data/)
- **/data - Excludes all files and folders named "data" in any subdirectory
(e.g., archive/data, users/data/)

► Advantages:

- Reduces image size, enhances security, and streamlines the build process

Docker, Kubernetes & AWS

Running commands: RUN

The RUN command

▶ RUN

- ▶ Executes commands inside your Docker image layers during the build process
- ▶ Typically used for configuration tasks like
 - ▶ installing packages, creating folders, modifying files,...
- ▶ Each RUN command creates a new layer to the image

▶ Syntax:

- ▶ RUN [command]
- ▶ You can chain multiple commands using &&
 - ▶ This also reduces the number of layers and thus the final image size

▶ Example:

- ▶ RUN apt-get update && apt-get install -y python3
- ▶ RUN apt update && apt install -y python3

▶ Important:

- ▶ A RUN command cannot access user input

Docker, Kubernetes & AWS

Simplifying paths:

WORKDIR

The WORKDIR command

- ▶ **WORKDIR**

- ▶ Sets the working directory for following instructions:

- ▶ ADD, COPY, CMD, RUN, ...

- ▶ If the specified directory does not exist, **WORKDIR** will create it

- ▶ **Syntax:**

- ▶ **WORKDIR** [path]

- ▶ **Example**

- ▶ **WORKDIR** /app

- ▶ **Best practice:**

- ▶ Do include a **WORKDIR** in your Dockerfile

- ▶ Otherwise, the **WORKDIR** might be set from the image that we are deriving from

- ▶ Use absolute paths to avoid confusion

Docker, Kubernetes & AWS

Deploying our image: Running a custom registry

Running our own Registry

► In this chapter:

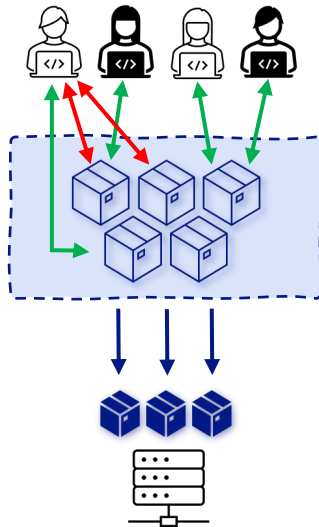
- We want to establish our own registry
- This will allow us to share our docker images...
- ... and to deploy them to multiple servers
- (almost like a private DockerHub for just ourselves)

► The goal:

- We can use our registry to help us deploying an image
- We can use our own registry to transfer an image from our computer to another computer / our server!

► Important:

- Many cloud providers offer their built-in registries
- If you're deploying images to a cloud service, their native registries could be a practical choice... we will get to this in the next chapter!
- But now, we want to focus on how to run our own registry!

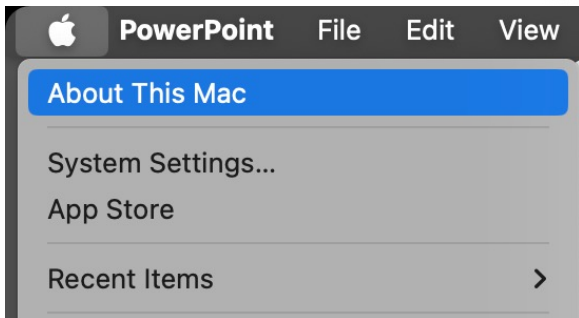


Docker, Kubernetes & AWS

Additional: CPU architectures

Why is this important?

- ▶ You are running docker on a Raspberry Pi
- ▶ You want to deploy to AWS (to an ARM-based server)
 - ▶ Usually, they might be a bit cheaper
- ▶ You are using a modern mac with an Apple CPU
 - ▶ (M1, M2, M3, M4,...)



CPU architectures: ARM vs. x86

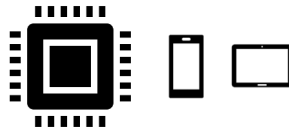
► x86:

- A microprocessor/instruction set architecture that is currently used as standard for CPUs in laptops and desktop computers
- In particular: Intel and AMD produce x86-compatible CPUs



► ARM:

- An alternative microprocessor/instruction set architecture that is mainly used in smartphones and tablets
- But Apple chips for MacBooks are also based on ARM
- And some laptops use ARM chips already
- There seems to be a trend towards ARM-based CPUs



CPU architectures and docker

- ▶ Docker uses the kernel of the host operating system
- ▶ **This means:**
 - ▶ Running a container under a different CPU architecture is not easily possible
 - ▶ An ARM host usually runs container built for the ARM architecture (`linux/arm64`)
 - ▶ A x86_64 host runs the container for the x86_64 architecture (`linux/amd64`)
 - ▶ **However:** Some systems might support running a different platform
- ▶ **Example:**
 - ▶ `docker run -it --platform=linux/amd64 ubuntu`
 - ▶ This will enforce `linux/amd64` as a CPU architecture
 - ▶ If the host can't run this architecture, an error will occur
 - ▶ **Important:** Emulating a different CPU architecture causes a significant overhead
- ▶ **Thus, you need to be aware:**
 - ▶ An image needs to be created for each CPU architecture separately
 - ▶ Additionally, the code might run slightly differently

Docker, Kubernetes & AWS

Let's launch a registry!

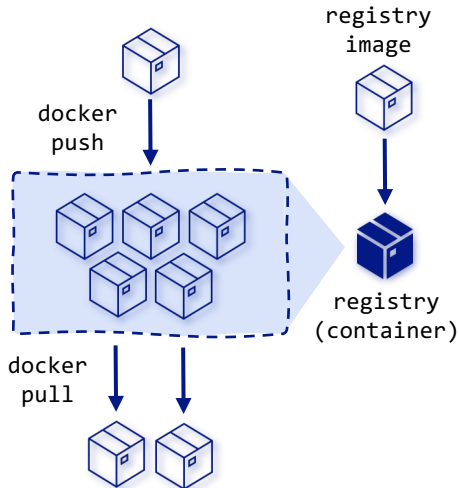
Running our own registry

► Start a new registry:

- We need to launch a container based on the registry image:
 - https://hub.docker.com/_/registry
 - `docker run -d -p 5000:5000 --name registry registry:2`
- This will launch our personal registry

► How do we store an image into this registry?

- 1.) Tag the image for your local registry
 - Here, my server for the registry is called `localhost:5000`
 - `docker image tag my-node-app localhost:5000/my-node-app`
- 2.) Push the tagged image to your registry
 - `docker push localhost:5000/my-node-app`
- 3.) Now you can retrieve the image from your own registry as needed
 - `docker pull localhost:5000/my-node-app`



Docker, Kubernetes & AWS

Enabling password protection

Protecting our registry with a password

- ▶ The easiest way to enable authentication is to use the HTTP Basic Authentication

- ▶ **What is HTTP Basic Authentication?**

- ▶ A method for authentication that is built into the HTTP standard
- ▶ **On the server:**
 - ▶ We can configure it using a `.htpasswd` file
 - ▶ It contains records of usernames and their encrypted passwords

- ▶ **How do we create a `.htpasswd`?**

- ▶ 1.) We can use a tool like `htpasswd` to create and manage the `.htpasswd` file
 - ▶ available as part of the Apache HTTP Server
 - ▶ `docker run -it --rm httpd htpasswd -Bn myuser`
- ▶ 2.) Run the Docker registry with `.htpasswd` authentication
 - ▶ <https://distribution.github.io/distribution/about/configuration/>

Docker, Kubernetes & AWS

AWS: Crash course

AWS: Crash course

- ▶ In this chapter, we will have to explore quite a bit of theory
- ▶ Later throughout this course, we will be working with AWS quite a bit
- ▶ **And for this, it is essential to explore important AWS concepts ahead:**
 - ▶ Region vs. Availability Zone
 - ▶ Using the AWS command line
 - ▶ IAM, policy, ...
- ▶ **Otherwise:**
 - ▶ The chapters later will be difficult to fully understand / follow along
- ▶ **But also here, I want to provide you with a hands-on example:**
 - ▶ We will see how we can allow an EC2 instance to access and control other EC2 instances
 - ▶ But this access will be limited to specific other EC2 instances
 - ▶ This could be a part of an important monitoring script for our EC2 cloud

Docker, Kubernetes & AWS

Terminology:

Region, Availability Zone

What is a region?

- ▶ **Definition:**

- ▶ Geographic area hosting multiple, isolated data centers

- ▶ **Purpose:**

- ▶ Redundancy & fault tolerance
 - ▶ Data sovereignty compliance
 - ▶ Latency optimization

- ▶ **Example:**

- ▶ **us-east-1:**

- ▶ Located in US East (North Virginia)

- ▶ **eu-central-1:**

- ▶ Located in Frankfurt (Germany, EU)

- ▶ **ap-southeast-2:**

- ▶ Located in Sydney (Australia)
- ▶ ap: Asia-Pacific

What is an availability zone?

- ▶ **Each region:**

- ▶ Has multiple, isolated locations
- ▶ These locations are known as availability zones (AZ)
- ▶ These availability zones can be imagined as separate data centers, in which our application can be run
- ▶ The availability zone is referred to by an additional letter, just after the region

- ▶ **Example:**

- ▶ `us-east-1a`:
 - ▶ Region: `us-east-1`
 - ▶ Availability zone: `a`
- ▶ `us-east-1b`:
 - ▶ Region: `us-east-1`
 - ▶ Availability zone: `b`

Docker, Kubernetes & AWS

What is a virtual server?

A virtual server

- ▶ **A virtual server:**

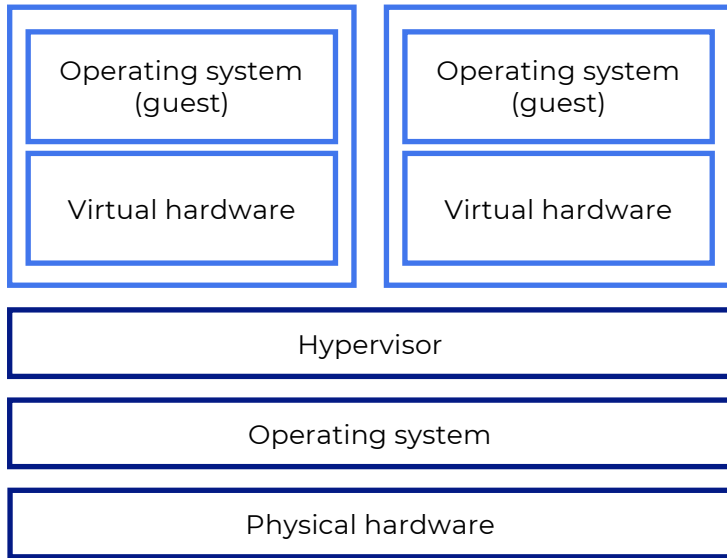
- ▶ A server, that is run on virtual hardware
- ▶ A single physical server can run multiple virtual servers
- ▶ The virtual machines are isolated from each other

- ▶ **In a data center:**

- ▶ Allows for software-based scheduling
- ▶ And better resource optimization

- ▶ **EC2:**

- ▶ We already rented a virtual server!
- ▶ EC2 allows us to rent a virtual server in AWS



Docker, Kubernetes & AWS

Accessing EC2: Through the command line

AWS command line

- ▶ We now want to access AWS through the command line
- ▶ For now, it's just another way to access AWS...
- ▶ ... but later, we will have to use the command line for certain things
- ▶ **AWS command line:**
 - ▶ Allows us to perform anything we can do through the AWS Dashboard
 - ▶ But we can do even more!
- ▶ **We can find additional information here:**
 - ▶ <https://aws.amazon.com/cli/>
- ▶ In the next lectures, we will install this command line on your system
- ▶ This is required, as we will need it later throughout this chapter / this course

Docker, Kubernetes & AWS

What is: IAM

IAM: Identity and Access Management

- ▶ **IAM:** Identity and Access Management
- ▶ **IAM users:**
 - ▶ Represent individual people or services that need access to AWS resources
 - ▶ Users can have their own credentials (password, access keys,...) and permissions
- ▶ **Root User:**
 - ▶ The AWS Root User is the account owner
 - ▶ This user has full access to all resources and services
 - ▶ It's recommended to limit the use of the root account
- ▶ **IAM Groups (not explored of this course):**
 - ▶ Collections of IAM users
 - ▶ Allows us to assign permissions to a group instead of users individually
- ▶ **IAM Policies (we'll get to these later):**
 - ▶ Documents that define permissions
 - ▶ They use JSON to specify what actions are allowed or denied for AWS resources

Docker, Kubernetes & AWS

The structure of an ARN

The structure of an ARN

▶ Example ARN:

▶ `arn:aws:ec2:eu-west-3:650251731599:instance/i-00dbb530af0f9aded`

▶ `arn:aws:iam::650251731599:user/MacBookProCLI`

▶ Format of an ARN:

▶ `arn:partition:service:region:account-id:resource-type:resource-id`

▶ `arn:partition:service:region:account-id:resource-type/resource-id`

▶ `arn:partition:service:region:account-id:resource-id`

▶ **Partition:** Where the service is located (aws, aws-cn, aws-us-gov)

▶ **Service:** The service that identifies the AWS product

▶ **Region:** The region, in which this resource is running

▶ **Account id:** The account id of the AWS root user

▶ **Resource type:** The type of resource (here: instance)

▶ **Resource id:** The resource identifier

Docker, Kubernetes & AWS

Terminology: IAM Overview (users, groups, policies)

IAM: Permission management

- ▶ **IAM:** Identity and Access Management
- ▶ **IAM users:**
 - ▶ Represent individual people or services that need access to AWS resources
 - ▶ Users can have their own credentials (password, access keys,...) and permissions
- ▶ **Root User:**
 - ▶ The AWS Root User is the account owner
 - ▶ This user has full access to all resources and services
 - ▶ It's recommended to limit the use of the root account for daily administrative tasks
- ▶ **IAM Groups:**
 - ▶ Collections of IAM users
 - ▶ Allows us to assign permissions to a group instead of users individually
- ▶ **IAM Policies:**
 - ▶ Documents that define permissions
 - ▶ They use JSON to specify what actions are allowed or denied for AWS resources

Heads-up

- ▶ IAM and permissions can be a rather complex topic
- ▶ However, we will explore this step by step
- ▶ Through this, it will become simple to understand
- ▶ If you've never worked with AWS / IAM before, be sure to not skip any lectures - they depend on each other

Docker, Kubernetes & AWS

IAM: Creating a second user

IAM: Creating a secondary user

- ▶ Let's say we wanted to grant read-only access to our EC2 instances to a shell script
- ▶ This could enable us to automate the monitoring of the status of our instance(s)
- ▶ **This access needs to be limited:**
 - ▶ It should be restricted to EC2
 - ▶ In addition, the access should be read-only
- ▶ **For this:**
 - ▶ We can create a separate user, and attach policies to it
 - ▶ We will then use this user to access information from EC2
 - ▶ Let's explore this!

Docker, Kubernetes & AWS

Operating as the secondary user in the CLI

Operating as the secondary user in the CLI

- ▶ We now want to fetch information about our EC2 servers through the CLI
- ▶ For this, we want to use the restricted user we created earlier
- ▶ **For this, we need to explore the following environment variables:**
 - ▶ **AWS_ACCESS_KEY_ID:**
 - ▶ The access key used to access AWS
 - ▶ **AWS_SECRET_ACCESS_KEY:**
 - ▶ The secret key used to access AWS
 - ▶ **AWS_DEFAULT_REGION:**
 - ▶ The region used by default
- ▶ **After that, we can just call the aws program:**
 - ▶ `aws ec2 describe-instances`

Docker, Kubernetes & AWS

IAM: Customizing the policy of a user (inline policy, resources)

Customizing a policy

- ▶ **The goal now:**

- ▶ Let's say the shell script should also be able to change the state of a specific instance
- ▶ It thus should be allowed to stop or start a specific EC2 instance... but only this one specific EC2 instance, not any other
- ▶ How can we grant it this permission?

- ▶ **The solution:**

- ▶ We can further customize the policy to only grant this access to one specific instance (or specific instances)

Docker, Kubernetes & AWS

IAM: Creating a managed (custom) policy

Creating a managed (custom) policy

- ▶ **So far:**

- ▶ We had used an inline policy
- ▶ This policy is directly attached to a user
- ▶ And can thus only be used by this user

- ▶ **We now want to create a general policy:**

- ▶ This policy can be re-used by multiple users
(or *roles*, but more on these later in this chapter)
- ▶ **Let's explore this!**

Docker, Kubernetes & AWS

Example:

Running aws on an EC2 instance

The problem so far

- ▶ **So far:**

- ▶ We would have to store the API key on the instance
(access key, secret access key)
- ▶ This instance would then be able to perform actions on behalf
of this user

- ▶ **The problems:**

- ▶ The API key is fixed and never changes
- ▶ Storing API keys on many different instances would be rather
tedious and difficult
- ▶ How about we automate everything?

Docker, Kubernetes & AWS

IAM: Roles

Using roles

► Roles:

- Allow us to automate the provisioning of policies to resources in the AWS cloud
- A resource (such as our EC2 instance) can automatically assume a specific role... after which it can then use it to access AWS resources

► Meaning:

- If configured properly, our EC2 instance no longer needs to store any AWS tokens / API keys - it can automatically fetch them!

► In addition:

- Tokens / API-Keys are automatically managed for us (and renewed regularly)
- We don't have to deal with it anymore
- And we can easily revoke policies / roles



Docker, Kubernetes & AWS

Getting started with git & GitHub

What is Git?



- ▶ **git is a version control system**

- ▶ Tracks changes in source code during software development
- ▶ Allows easy restoration to previous versions
- ▶ Enables parallel work through multiple branches, which can be merged

- ▶ **Suitable for small to very large projects**

- ▶ Essential tool for software developers
- ▶ Created by Linus Torvalds in 2005



Installation guide

- ▶ **macOS:**

- ▶ We can use homebrew to easily install git:

- ▶ `brew install git`

- ▶ **Linux:**

- ▶ It's best to use the corresponding package manager of our distribution

- ▶ For example, apt on Ubuntu / Debian:

- ▶ `sudo apt install git`

- ▶ **Windows:**

- ▶ We can install git directly:

- ▶ <https://git-scm.com/download/win>

- ▶ **Once git is installed:**

- ▶ Let's confirm we can use it:

- `git --version`



Docker, Kubernetes & AWS

Initializing a first Git repository

Initializing a new Git repository

- ▶ **To initialize a new Git repository, we can use the command `git init`**
 - ▶ A hidden `.git` directory will be created
 - ▶ It will be used to track changes and versions of our project
- ▶ **To ignore files:**
 - ▶ We can create a `.gitignore` file to specify which files or directories should be ignored (similar to a `.dockerignore` file when building docker images)
- ▶ **Typical entries:**
 - ▶ `.env` to ignore environment files
 - ▶ `node_modules/` to ignore installed Node.js dependencies
 - ▶ `dist/` to ignore build output directories
 - ▶ `venv/` to ignore virtual environment directories
 - ▶ `.DS_Store` to ignore macOS directory metadata files
 - ▶ `.Thumbs.db` to ignore Windows directory metadata files



Docker, Kubernetes & AWS

Basic git commands

Basic git commands

- ▶ **Git works by creating with 3 main areas:**

- ▶ **Working directory**

- ▶ This is the area in which we modify files
 - ▶ Changes here are not yet tracked, until we stage them

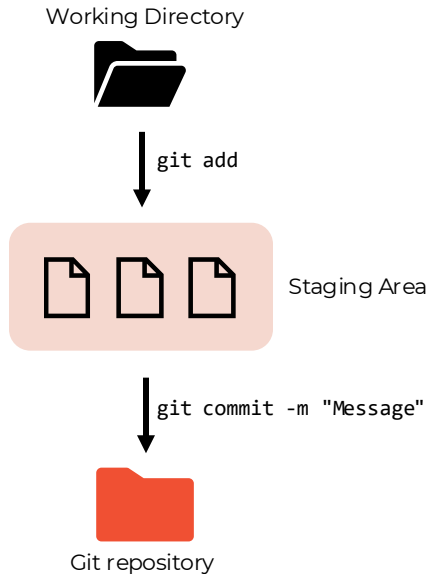
- ▶ **Staging area**

- ▶ A place where you can prepare changes
 - ▶ To stage file, we can use `git add`

- ▶ **Repository**

- ▶ The local git repository tracks the project history (managed through the `.git` folder)
 - ▶ Staged changes from the staging area are moved here using `git commit -m "Commit message"`

- ▶ `git status` displays a summary of which files are staged, unstaged, and untracked

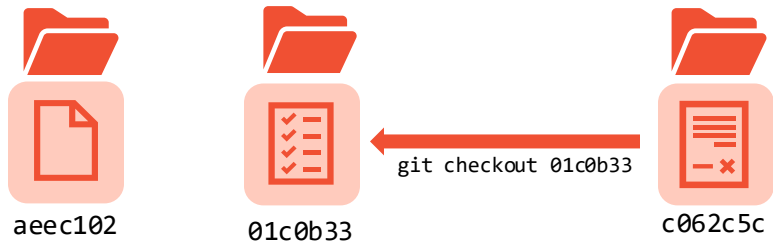


Docker Deployment Bootcamp

Git: Restoring a previous state

Restoring a previous state

- ▶ Each commit has a unique hash that we can use to identify it
- ▶ **To display a list of commits (and their hashes):**
 - ▶ `git log`
 - ▶ Short form: `git log --oneline`
- ▶ **To restore the directory to a previous state:**
 - ▶ `git checkout commit_hash`
- ▶ **To restore files to a specific state from a commit or branch:**
 - ▶ `git checkout commit_hash -- filename`

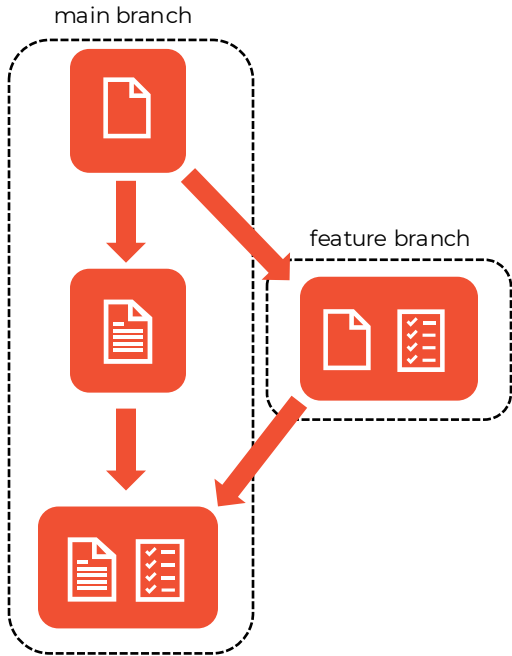


Docker Deployment Bootcamp

Git: Branches

Branches in Git

- ▶ **An important feature in Git are branches:**
 - ▶ Enable parallel development
 - ▶ Isolate individual features and bugfixes
- ▶ **Create a new branch:**
 - ▶ `git branch branch_name`
- ▶ **Switch to a branch:**
 - ▶ `git checkout branch_name`
- ▶ **Create a new branch and immediately switch to it:**
 - ▶ `git checkout -b branch_name`
- ▶ **List all branches in the repository:**
 - ▶ `git branch`
- ▶ **Merge a branch into the current branch:**
 - ▶ `git merge feature-branch`





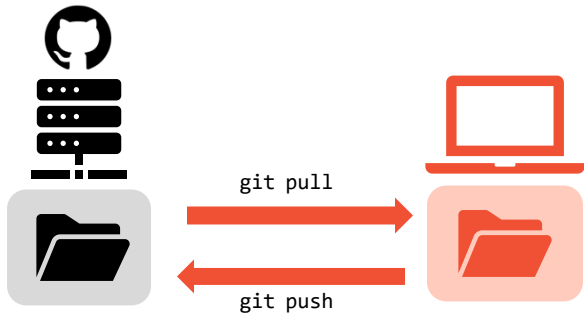
Docker Deployment Bootcamp

What is GitHub?



What is GitHub?

- ▶ A web-based platform for Git repositories
- ▶ Allows to store and manage your Git repositories online
- ▶ Collaborate with others on projects
 - ▶ E.g., track tasks, bugs, and feature requests
- ▶ Used by many open-source projects



Docker, Kubernetes & AWS

Hosting images on ECR / Deploying to AppRunner

Overview

- ▶ **In this chapter:**

- ▶ We want to explore how we can host our images in Amazon ECR
- ▶ ECR: Elastic Container Registry
- ▶ Allows us to store images in the Amazon cloud

- ▶ **In addition:**

- ▶ We will explore how we can automatically deploy changes to AppRunner
- ▶ And we will explore how we can automatically create images based on a GitHub repository

- ▶ **This will create a fully automated deployment setup:**



Docker, Kubernetes & AWS

What is ECR / Creating a repository

Amazon Elastic Container Registry



Amazon ECR

- ▶ **What is Amazon ECR?**

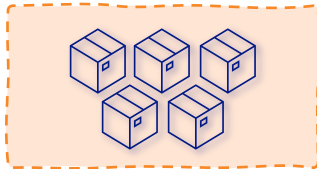
- ▶ Amazon ECR is a container registry service
- ▶ AWS handles the underlying infrastructure for hosting our registry

- ▶ **Benefits:**

- ▶ It is tightly integrated within the AWS ecosystem:
 - ▶ Automatically scales
 - ▶ Integrates into AWS Identity and Access Management (IAM) for permission control
 - ▶ Integrated into AppRunner, ECS, EKS,...

- ▶ **Reduces the complexity of handling our own container registry**

- ▶ **Let's explore ECR!**



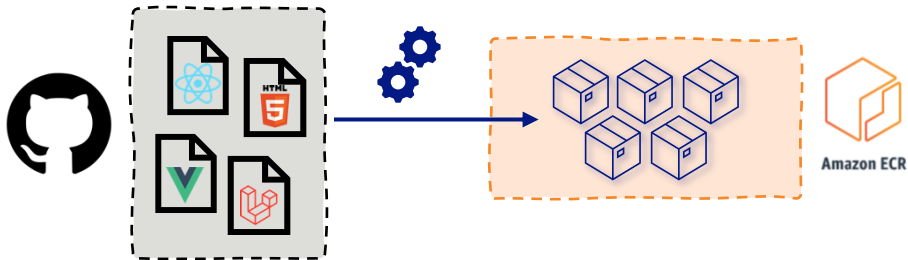
Docker, Kubernetes & AWS

Part 2:

Automatic deployment

Automating image builds

- ▶ So far, we had to build and upload the image manually (or through a shell script)
- ▶ This can be time-consuming, or we might forget it
- ▶ **What we want to explore:**
 - ▶ Let's say we had a GitHub repository to hold our source code
 - ▶ Can AWS automatically create the image for ECR from that?
 - ▶ This would significantly streamline our development workflow!



Docker, Kubernetes & AWS

Advanced Dockerfiles: Building for production

Overview

- ▶ **So far:**

- ▶ We have been able to build images through our own Dockerfile
- ▶ This was enough to allow some basic deployment

- ▶ **In this chapter:**

- ▶ We want to continue exploring Dockerfiles
- ▶ And we will explore concepts that are important for advanced deployment

- ▶ **Reducing the size of our image:**

- ▶ *This will allow us to deploy and update our application faster*
- ▶ Alpine Linux
- ▶ Multi-Stage-Builds:
 - ▶ How to efficiently deploy a ReactJS application

- ▶ **How to monitor the health of a container:**

- ▶ Adding a HEALTHCHECK

Docker, Kubernetes & AWS

The Linux distribution: Alpine Linux

Alpine Linux



- ▶ **Alpine Linux:**

- ▶ A very small Linux distribution (just a few MB)
- ▶ Often used as the base image for many images
- ▶ Good alternative to Debian

- ▶ **However:**

- ▶ Alpine uses a different C library than most other Linux distributions:
 - ▶ Alpine: musl
 - ▶ Most other Linux distributions: glibc (GNU C Library)

- ▶ **Let's explore Alpine Linux!**

Docker, Kubernetes & AWS

Alpine Linux (part 2)

musl. vs. glibc

- ▶ **musl and glibc:**

- ▶ Implementations of the C standard library
- ▶ It provides various functionality for other programs:
 - ▶ Opening and reading files
 - ▶ Reading input, printing output (printf)
 - ▶ Reading and converting different encodings
(for example: iconv to convert to / from unicode and UTF-8)
 - ▶ Formatting of date
 - ▶ Regular expressions
 - ▶ Network communication

- ▶ **glibc:**

- ▶ Defacto standard implementation on most Linux systems

- ▶ **musl:**

- ▶ Small, modern implementation of the C standard library

1st problem with musl

- ▶ **Usually:**

- ▶ Programs are optimized for glibc
- ▶ And most programmers don't assume that their code will be compiled with musl
- ▶ Differences or implementation details might not be accounted for

- ▶ **Example:**

- ▶ Performance differences:
 - ▶ <https://news.ycombinator.com/item?id=27379482>
- ▶ Or strftime once worked differently on musl vs. glibc systems:
 - ▶ This even affected Python applications!
 - ▶ <https://bugs.python.org/issue34672>

2nd problem with musl

- ▶ **When using Alpine Linux (musl):**

- ▶ Code that runs on a Linux that is based on musl, must have been compiled for musl
- ▶ When using a distribution specific package manager:
 - ▶ This is usually not much of a problem
 - ▶ All binary packages are already provided in the right form
- ▶ But when using non-distribution package managers, this can lead to problems

2nd problem with musl

- ▶ **Let's say we would run a Python container:**

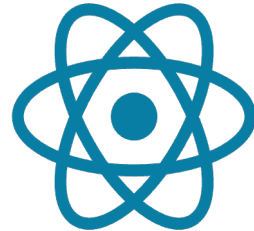
- ▶ `python:alpine` (based on Alpine)
- ▶ If we now want to use pip (package manager for python) to install `scikit-learn`:
 - ▶ `pip install scikit-learn`
- ▶ The pip repository provides the following:
 - ▶ The source code to build the tool "scikit-learn"
 - ▶ And a pre-compiled version of `scikit-learn`

- ▶ **However:**

- ▶ The pre-compiled version is usually only provided for glibc systems
- ▶ Thus, Alpine Linux will have to compile `scikit-learn` from scratch

- ▶ **But if we were to use a different Python image:**

- ▶ `python:slim` (based on Debian)
- ▶ If we tried to install pandas here, most likely pip could use the pre-compiled `.whl` binary to install it => significantly faster, no build tools required!



Docker, Kubernetes & AWS

Multi-stage builds

Multi-stage builds

- ▶ **Problem:**

- ▶ Sometimes, for example when we need to compile our application, our image will become rather large

- ▶ **The reason:**

- ▶ We need to install the compiler with all the dependencies
 - ▶ And even when uninstalling them later, they might still be contained in the image (in a previous layer)

- ▶ **So how can we solve the following problem:**

- ▶ Large compile step
 - ▶ But a small, runtime environment

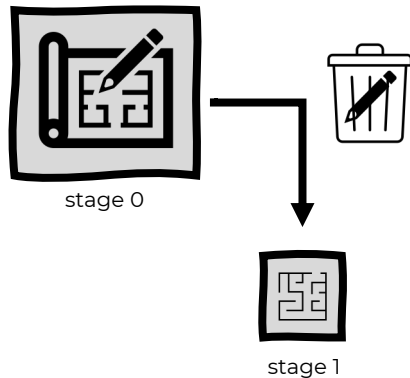
Multi-stage builds

- ▶ **Solution:**

- ▶ Multi-stage builds!
- ▶ This allows us to have a build-stage, and an execution stage

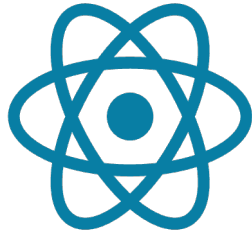
- ▶ **Idea:**

- ▶ We create all necessary artefacts (such as executable files,...) in a temporary container
- ▶ We then copy those into a smaller image (such as into a container based on Alpine Linux)
- ▶ This image will then only contain the components that are needed during execution (but no container)



Multi-stage builds

- ▶ How can we create a multi-stage build?
- ▶ We can just use multiple `FROM` commands within the same `Dockerfile`
- ▶ **We can then use the `COPY` command to copy files:**
 - ▶ We just need to specify an additional parameter: `--from`
 - ▶ `COPY --from=0 [old path] [new path]`
- ▶ **We can also name the building phases with `AS`:**
 - ▶ `FROM node:20 AS builder`
 - ▶ `[...]`
 - ▶ `FROM nginx:alpine AS runtime`
 - ▶ `COPY --from=builder [path in builder image] [path in new image]`



Docker, Kubernetes & AWS

Adding a HEALTHCHECK

Adding a HEALTHCHECK

- ▶ Sometimes, we want docker to ensure that the image started properly
- ▶ **For this, we can add a HEALTHCHECK:**
 - ▶ HEALTHCHECK [options] CMD [CMD]
 - ▶ The command should exit with success (exit code: 0) if everything is okay
 - ▶ And it should exit with an error (exit code: 1) if some an error occurred
 - ▶ **Example:**
 - ▶ HEALTHCHECK CMD curl -f <http://localhost> || exit 1
 - ▶ **We can customize the default options:**
 - ▶ HEALTHCHECK --interval=30s --timeout=30s --start-period=0s --start-interval=5s --retries=3 CMD curl -f <http://localhost> || exit 1
- ▶ **A container with a healthcheck:**
 - ▶ Will have an additional status, initially set to *"starting"*
 - ▶ Once the healthcheck has passed, it will be *"healthy"*
 - ▶ After a certain number of failed health checks, this status will become *"unhealthy"*

Docker Deployment Bootcamp

Networking on AWS

Networking in AWS

- ▶ **In this chapter, we want to explore networking in AWS:**

- ▶ VPCs
- ▶ Internet Gateways
- ▶ Network ACLs
- ▶ Security groups

- ▶ **This chapter is essential:**

- ▶ AWS will only tell you that the communication to another service has failed
- ▶ It will be up to you to figure out the problem
- ▶ This chapter may be a bit theoretical - but it's the foundation we need to launch multi-service applications on AWS
- ▶ Otherwise, we'd spend endless hours on debugging and researching when an error occurs
- ▶ This chapter will give you a step-by-step guide!

Docker, Kubernetes & AWS

Networking: What is a VPC?

What is a VPC?

- ▶ **A VPC** (Virtual Private Cloud):

- ▶ You can imagine this as a network - just virtual, and in the cloud
- ▶ Resources (such as EC2 servers) within a VPC can communicate directly to each other
- ▶ AWS creates a default VPC for each region... but you can also create additional ones

- ▶ **Virtual:**

- ▶ The whole networking is being emulated by AWS
- ▶ To us, it just behaves like we got an individual network, just for our resources (servers)

- ▶ **Private Cloud:**

- ▶ Your instances are in a separate network
- ▶ You will not have machines from other people in your VPC
- ▶ Thus, a VPC creates a designated, private area for your machines

Docker, Kubernetes & AWS

Networking: From VPC to subnets

What is a subnet?

- ▶ **A subnet:**

- ▶ A segment of the IP address range, in which we can place our servers (or other resources)
- ▶ A subnet can only span a single availability zone

- ▶ **By default:**

- ▶ The VPC is configured so that all subnets can communicate to nodes in other subnets... but we could change that (route table)

VPC

IP range (172.31.0.0/16):

- 172.31.0.0 to 172.31.255.255

Subnet 1

AZ: eu-west-3a

IP range (172.31.0.0/20):

- 172.31.0.0 - 172.31.15.255

EC2 instance 1

EC2 instance 2

**Additional resources /
services**

Subnet 2

AZ: eu-west-3b

IP range (172.31.16.0/20):

- 172.31.16.0 to 172.31.31.255

EC2 instance 3

EC2 instance 4

**Additional resources /
services**

Subnet 3

AZ: eu-west-3c

IP range (172.31.32.0/20):

- 172.31.32.0 to 172.31.47.255

EC2 instance 5

EC2 instance 6

**Additional resources /
services**

Simplified diagram. An AZ could also host multiple subnets. Also, an EC2 instance could have multiple network interfaces, and through that, be part in multiple subnets at the same time,...

Docker, Kubernetes & AWS

Let's create a custom VPC!

Public vs. private subnets

- ▶ **Public subnet:**

- ▶ Has a route to an Internet Gateway
- ▶ This allows resources (EC2,...) to access the internet

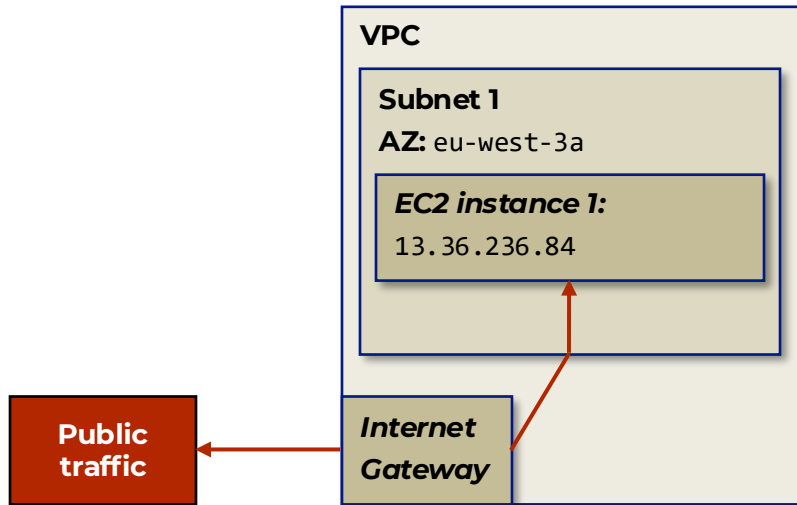
- ▶ **Private subnet:**

- ▶ Does not have a direct route to the Internet Gateway.
- ▶ Resources are private (even if they have a public IP address)

Docker, Kubernetes & AWS

Internet Gateways

How do instances connect to the public internet?



Docker, Kubernetes & AWS

Network ACLs

Network ACL

- ▶ Network Access Control List
- ▶ Optional firewall feature
- ▶ **Allows us to define rules on a subnet level:**
 - ▶ **Inbound rules:**
 - ▶ Allows us to configure which traffic can enter a subnet
 - ▶ **Outbound rules:**
 - ▶ Allows us to configure which traffic can leave a subnet
 - ▶ **Important:**
 - ▶ Network ACLs are *stateless*
 - ▶ **Stateless means:**
 - ▶ If we want to allow an inbound connection...
we must explicitly allow the return traffic (with an outbound rule)
- ▶ **Default configuration:**
 - ▶ Allow all traffic

Docker, Kubernetes & AWS

Terminology: Security group

What is a security group?

- ▶ **Security group:**

- ▶ A virtual firewall
- ▶ Controls inbound and outbound traffic to and from our AWS resources (such as EC2 instances)

- ▶ **Inbound rules:**

- ▶ Control which incoming traffic is allowed to reach our resource

- ▶ **Outbound rules:**

- ▶ Control which outgoing traffic is allowed from our resource

- ▶ **Stateful:**

- ▶ If we allow inbound traffic to an instance, the response traffic is automatically allowed back out (regardless of outbound rules)

- ▶ **An individual instance:**

- ▶ Can be associated with multiple security groups
- ▶ Rules from all associated security groups are aggregated

Docker Deployment Bootcamp

Debugging networking issues

Important

- ▶ To access another instance / resource in AWS, several conditions must be fulfilled
- ▶ **Especially the following:**
 - ▶ **Route to host:**
 - ▶ We must be able to connect to the other instance:
 - ▶ We must be in the same VPC
 - ▶ The VPC / route table / ... must be configured properly
 - ▶ The Network ACL must allow this connection
 - ▶ **Security group (most common issue):**
 - ▶ The security group(s) on the outgoing side must allow the connection
 - ▶ The security group(s) on the receiving side must allow the connection
 - ▶ **For internet access (incoming and outgoing):**
 - ▶ Public IP + Subnet needs to have an Internet Gateway + Internet Gateway is in Route Table
 - ▶ *(or we would have to configure a NAT Gateway, not part of this course)*
 - ▶ **Permissions (IAM):**
 - ▶ When accessing an AWS service, we must be allowed to access it
(from a permission perspective)

Docker, Kubernetes & AWS

Deployment of a full-stack web application

Deployment of a full-stack web app



► In this chapter:

- We will deploy a full-stack app to AWS:

► Frontend:

- React

► Backend:

- Node.js
- MongoDB

► The setup:

- Frontend will be built automatically and will be deployed to S3 (Simple Storage Service)
- Backend will be built automatically and will be deployed to AppRunner
- Frontend & backend will be served through a CloudFront CDN



Optional chapter

- ▶ **This chapter is completely optional**
- ▶ We will learn things about deployment into production...
- ▶ ... but won't learn anything new about Docker or container
- ▶ If you want to learn Docker (without deployment on AWS) as fast as possible... feel free to skip this chapter!

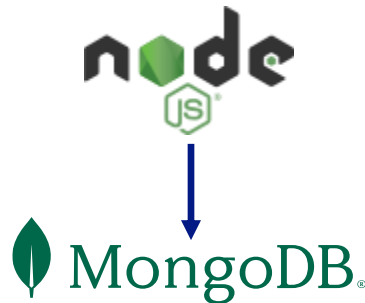
Docker, Kubernetes & AWS

Overview:

Deploying the backend

Deploying the backend

- ▶ So far, we were able to deploy stateless services to AppRunner
- ▶ **However, the backend consists out of 2 components:**
 - ▶ **Node.js application:**
 - ▶ Serves our actual backend
 - ▶ Provides the interface to the frontend
 - ▶ Stateless, doesn't store any data
 - ▶ **MongoDB:**
 - ▶ Accessed by the Node.js application
 - ▶ Provides the database to store our data
 - ▶ Stateful service!
- ▶ **The idea:**
 - ▶ We'll deploy the Node.js backend with AWS AppRunner
 - ▶ And let AWS manage our MongoDB for us:
 - ▶ *Amazon DocumentDB*



Amazon DocumentDB (with MongoDB compatibility)

Enables you to quickly and easily insert, index, query, and perform aggregations over JSON data without translations

The next two lectures

► The next two lecture:

- We will deploy the Node.js backend to AppRunner
- This is something that we've done already previously (deployment to AppRunner)
- The next 2 lectures are meant to be skipped by most students... but still included for the sake of completeness
- Feel free to continue with enabling Amazon DocumentDB!

Docker, Kubernetes & AWS

Deploying the backend: Enabling DocumentDB

Amazon DocumentDB

- ▶ **Overview:**

- ▶ Fully managed document database service
- ▶ Designed to be compatible with MongoDB workloads
- ▶ Ideal for storing, querying, and indexing JSON data

- ▶ **Compatibility with MongoDB:**

- ▶ DocumentDB is compatible with MongoDB
- ▶ Thus, we don't need to rewrite any code, and can just treat it as if it was a mongodb

- ▶ **Pricing: Heads-up!**

- ▶ **DocumentDB can incur significant costs!**
- ▶ **Be sure to check the pricing page**
- ▶ **And turn off the database, once you no longer need it!**

Docker, Kubernetes & AWS

Deploying the frontend

Deploying the frontend

- ▶ We now could also use an AppRunner to serve the frontend
- ▶ **But technically speaking...**
 - ▶ ... the frontend doesn't contain any code that needs to get executed
 - ▶ ... it's just a bunch of files that need to be made available
- ▶ **Idea:**
 - ▶ We compile the frontend into static files...
 - ▶ ... which we then store on S3 (Simple Storage Service)
 - ▶ And serve the files through a CloudFront CDN!

What is S3?

- ▶ **S3: Simple Storage Service**

- ▶ Scalable object storage service
- ▶ Designed for high durability, availability, and performance
- ▶ Integrates well into other AWS services
(such as CodeBuild, CloudFront,...)

- ▶ **Let's create a bucket!**

Docker, Kubernetes & AWS

Deploying the frontend: Configuring CloudFront

CloudFront

- ▶ **CloudFront CDN:**

- ▶ Content Delivery Network
- ▶ When a user requests an object, they are automatically routed to the nearest edge location
- ▶ Files can be cached, and re-used to serve the next request from the same geographic region
- ▶ This allows fast delivery of our assets

- ▶ **In addition:**

- ▶ We will be able to configure quite a bit
- ▶ Which will allow us to connect our API to the same domain...
- ▶ ... and even allow us to configure multiple domains:
 - ▶ www.docker-test.click
 - ▶ docker-test.click

Docker, Kubernetes & AWS

Configuring CloudFront: Enabling the backend

Configuring CloudFront

- ▶ **The idea:**

- ▶ */api*

- ▶ All requests that start with `/api` will be fulfilled by the Node.js backend (AppRunner)

- ▶ *All other paths:*

- ▶ All other requests will be served from S3
 - ▶ This will allow us to keep serving all our static files from S3

Docker Deployment Bootcamp

Deleting the setup

Deleting the setup

- ▶ **To delete the setup:**

- ▶ **We need to remove the application:**

- ▶ CloudFront CDN
 - ▶ Amazon DocumentDB cluster / instance
 - ▶ S3 bucket

- ▶ **We need to delete the build pipeline:**

- ▶ AWS CodeBuild
 - ▶ Image on ECR

- ▶ **We could also delete the domain configuration:**

- ▶ Hosted Zone on Route53 (but we will need it later anyway)

- ▶ **In addition, we might also want to remove all logs:**

- ▶ CloudWatch

Docker, Kubernetes & AWS

Updating containers / Volumes in docker

Overview: Volumes in docker

► The goal of this chapter:

- We want to run a database (MySQL / MariaDB) in a docker container
- However, in the future, we want to be able to seamlessly update this database to a new version
- How can we achieve this?

► Roadmap:

- How do updates work in docker?
- How do volumes allow us to store data outside of a container?
- How can we perform an update of our database system (and maintain the data stored in it)?



Docker, Kubernetes & AWS

How to update a container?

How do update a container?

- ▶ **Short answer:** We don't!
- ▶ We never update an existing container
- ▶ **In docker:**
 - ▶ We just create a new container
 - ▶ And throw away the old one!
- ▶ **Why is that?**

Docker, Kubernetes & AWS

Volumes in docker

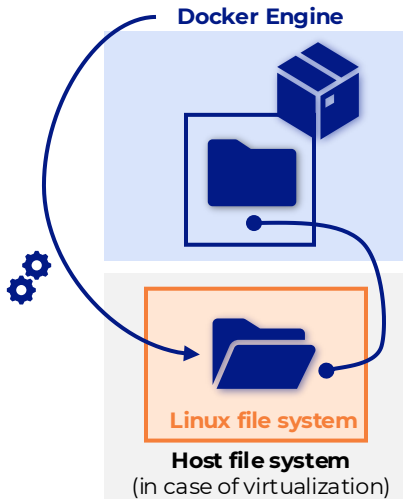
Volumes in docker

► What is a volume?

- It's a mechanism for mapping container data to a (hidden) directory on the host
- A volume can be linked to multiple containers
- Especially useful for databases

► Volumes:

- Are managed directly by Docker Engine and thus, efficient
- On Linux, volumes are usually stored under `/var/lib/docker/volumes`
- On Windows/macOS, they are stored within the VM that is running docker
- **No matter the OS:** We should never access it directly!
- We should only access it through `docker volume`



How to create & manage a volume?

- ▶ We can create a new volume through docker `volume create`:
 - ▶ `docker volume create nginx-vol`
- ▶ **To show all volumes:**
 - ▶ `docker volume ls`
- ▶ **To show information about an individual volume:**
 - ▶ `docker volume inspect nginx-vol`
- ▶ **We can also delete a volume:**
 - ▶ `docker volume rm nginx-vol`
 - ▶ This will only work if the volume is not in use
- ▶ **Or auto-delete all unused volumes:**
 - ▶ `docker volume prune`
- ▶ **Important:**
 - ▶ It's not possible to rename a volume
 - ▶ We would have to create a new volume, and copy all files into it

Docker, Kubernetes & AWS

How to connect a container with a volume

How to connect a container with a volume (--mount)

- ▶ **Just as with the bind mounts, we can use --mount to connect a volume:**

- ▶ `--mount type=volume,source=[volume-name],destination=[container-path]`

- ▶ **source:** The name of the volume, that we want to link

- ▶ **destination:** The path to the directory in the container, that we want to connect the volume to

- ▶ **Important:**

- ▶ If the volume does not exist so far, it will be created automatically

How to connect a container with a volume (-v)

- ▶ **But we can also just use -v instead:**

- ▶ -v [volume name]:[directory in the container]

- ▶ **Important:**

- ▶ This is very similar to the way of creating a bind mount

- ▶ **The only difference:**

- ▶ A directory (bind mount) would've started with "/" instead of the volume name

- ▶ A volume does not start with "/"

- ▶ If the volume does not exist so far, it will be created automatically

- ▶ When creating a bind mount, we need to be especially careful to not accidentally create a volume instead!

- ▶ **Multiple volumes:**

- ▶ We can also connect multiple volumes

- ▶ For this, we can just specify -v multiple times

Docker, Kubernetes & AWS

Automatically created volumes

Automatically created volumes



- ▶ Some containers will automatically create volumes for us
- ▶ This allows data to be stored outside of the container - and allows an easy update!
- ▶ **For example, the database management systems MariaDB or MySQL:**
 - ▶ `docker run -it -e MARIADB_ROOT_PASSWORD=password123 mariadb`
 - ▶ For running MariaDB, we need to set the root password through an environment variable, using `-e` (or `--env`)
 - ▶ **If we then run a docker inspect on this container:**
 - ▶ `docker inspect -f "{{ .Mounts }}" [container name / id]`
 - ▶ **This volume also appears in the list of volumes:**
 - ▶ `docker volume ls`
 - ▶ **It would be a bit better to create our own named volume:**
 - ▶ Naming the volume ourselves ensures we can easily reuse the volume
 - ▶ `docker run -it -v mariadb-vol:/var/lib/mysql -e MARIADB_ROOT_PASSWORD=password123 mariadb`

Docker Deployment Bootcamp

Dockerfiles and Volumes

Dockerfiles and Volumes

- ▶ Through a Dockerfile, we can also enforce that a directory will always be placed on a volume:
 - ▶ `VOLUME [folder]`
 - ▶ **Example:**
 - ▶ `VOLUME /usr/share/nginx/html`
- ▶ This could be useful for our own applications, if we'd manage the data ourselves
- ▶ **However:**
 - ▶ Once set, a `VOLUME` directive in a Dockerfile can't be undone

Docker, Kubernetes & AWS

How do we update the DB?

How do we update the database?

► The update process:

- We just create a new container with the new version
- And map / copy all the required data into it!
- This is why we stored the actual data on a separate volume!

► Heads-up:

- MariaDB is not backward-compatible
- We might also want to set the environment variable `MARIADB_AUTO_UPGRADE=1`
- Once the data is upgraded, we can't downgrade it (usually)



The plan of attack

- ▶ **First, we will run an outdated version of MariaDB:**

- ▶ This container will be connected to a custom volume
- ▶ We will connect to the database, and insert some data into it
- ▶ After this, we will turn this container off

- ▶ **After this, we will launch the latest version of MariaDB:**

- ▶ This container will be connected to the same volume
- ▶ We will see that the data is still available!

Docker, Kubernetes & AWS

Networking in Docker

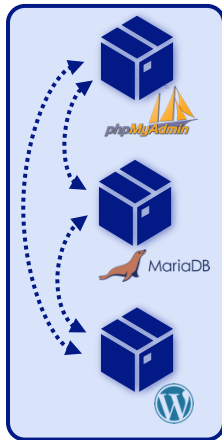
The goals of this chapter

► Questions:

- How can we connect multiple Docker containers to each other? Such as a webserver and a database?
- How can we create a network that allows multiple containers to communicate with each other?
- How can we isolate a container for sensitive tasks?
- What are *bridge* networks and which other network types can we use?

► The goal in this chapter:

- You can launch multi-container setups
- WordPress + MariaDB + phpMyAdmin

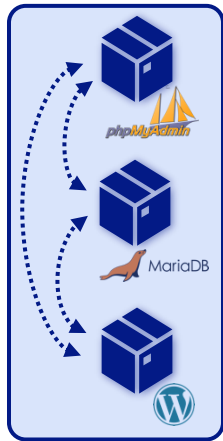


Docker, Kubernetes & AWS

Networking in docker

Communication between containers

- ▶ In Docker, we should run each service in an isolated container
- ▶ But usually, our application will consist of multiple services
- ▶ **Example:**
 - ▶ 1 container for MySQL / MariaDB
 - ▶ 1 container for phpMyAdmin
(management software for MySQL / MariaDB)
 - ▶ 1 container for WordPress
- ▶ How can we connect multiple containers?
- ▶ **The solution: We can create a network!**
 - ▶ Docker networks allow containers to communicate securely and efficiently by simulating a traditional network
 - ▶ This seamless interaction between multiple containers is what makes containers incredibly scalable and powerful



Default networks in docker

- ▶ Docker comes with some predefined networks
- ▶ **We can inspect them through the following command:**
 - ▶ `docker network ls`
 - ▶ The default networks are bridge, host, and none
 - ▶ Later, we will also be able to add custom networks
- ▶ **Each network:**
 - ▶ Has a unique name
 - ▶ And a unique network ID
- ▶ **We can inspect the networks with the following command:**
 - ▶ `docker network inspect [network-name/id]`
 - ▶ `docker network inspect bridge`

Docker, Kubernetes & AWS

Network type: bridge

Network type: bridge

- ▶ **Bridge network:**

- ▶ Enables communication between standalone containers
- ▶ But keeps them isolated from the external world

- ▶ **How it works:**

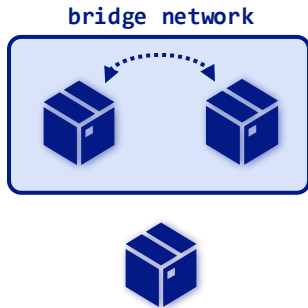
- ▶ Docker establishes a private subnet for containers
- ▶ Typically, within the IP range of `172.17.0.0/16`
- ▶ Containers must be executed on the same host (as in our current setup)

- ▶ **This is the default network:**

- ▶ If no specific network is assigned at launch, containers will be connected to this network automatically

- ▶ **Important:**

- ▶ Of course, we can still publish container ports to enable external communication (using the `-p` option)



Docker, Kubernetes & AWS

Network types:

host & none

Network types: host

► Host network:

- The container's networking is directly integrated with the host
- There's no network isolation!
- Containers are accessible via the IP address of the host
- Ports that containers open are directly accessible on the host system
- We can only have a single host network per host

► Important:

- Publishing ports (with the `-p` option) will no longer work
- This network type is only supported on Linux hosts
- **On macOS / Windows:**
 - Docker runs inside a virtual machine
 - Thus, the host network mode is not supported



192.168.136.133

Network types: none

- ▶ **None network:**

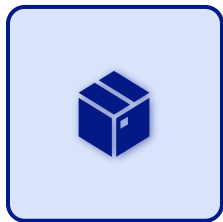
- ▶ The networking capabilities of a container are disabled
- ▶ No network interface is created, except the loopback interface (localhost or 127.0.0.1)

- ▶ **This means:**

- ▶ We can't access this container from the external world
 - ▶ We can't connect from this container to the internet
 - ▶ Port mappings are not applicable either

- ▶ **Important**

- ▶ We cannot create this network manually
 - ▶ This network is already built-in within docker

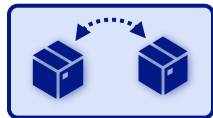
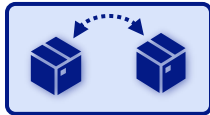


Docker, Kubernetes & AWS

How to create a custom network?

Custom bridge networks

- ▶ We can also create our own, custom bridge network
- ▶ This allows us to create containers that can only communicate to each other
- ▶ **Why would we want to do this?**
 - ▶ Isolates specific groups of containers from others on the system
 - ▶ Reduces the attack surface, and thus leads to a more secure deployment
- ▶ **We can do this through docker network:**
 - ▶ `docker network create [network-name]`
- ▶ **After this, we can attach this network to a container:**
 - ▶ `docker container run --network [network-name]`
- ▶ **Of course, we can also remove networks:**
 - ▶ To delete a specific network:
 - ▶ `docker network rm my-network`
 - ▶ To delete all unused networks:
 - ▶ `docker network prune`



How to assign a network manually

- ▶ **We can also connect a container to an existing network:**

- ▶ `docker network connect [network-name/id] [container-name/id]`

- ▶ **Example:**

- ▶ `docker network connect my-blog my-mariadb`

- ▶ This will connect the container `my-mariadb` to the network `my-network`

- ▶ **We can also disconnect a container from a network:**

- ▶ `docker network disconnect bridge my-mariadb`

- ▶ The container `my-mariadb` is now no longer in the default network `bridge`

Docker, Kubernetes & AWS

Working with hostnames

Custom network vs. bridge

- ▶ A custom network works slightly different than the default bridge network
- ▶ **In the default bridge network:**
 - ▶ Containers can access each other using their IP addresses
- ▶ **Within a custom network:**
 - ▶ Similarly, containers can access each other through their container IP addresses
 - ▶ Additionally, a custom network enables DNS resolution of container names to their respective IP addresses
 - ▶ This allows containers to communicate by name
 - ▶ **Let's explore this in an example!**



Docker, Kubernetes & AWS

MongoDB & mongo-express

Task: MongoDB & mongo-express

► Project:

- We want to launch a database with a software to manage it
- In this project, we want to launch a **MongoDB** database
- And be able to access it through a **mongo-express** interface
- This should happen within an isolated, custom network in docker

► Images used:

► MongoDB:

- https://hub.docker.com/_/mongo

► mongo-express:

- https://hub.docker.com/_/mongo-express



Task: MongoDB & mongo-express

► Task (part 1):

- Create a bridge network called `mongo-net` and start a MongoDB container with the name `my-mongo` in it
- At the same time, connect the directory `/data/db` in the container directly to a volume `mongo-vol`

► Tips:

- You can run MongoDB in the background (detached mode)
- You can also inspect the container more closely:
 - It runs on a Debian
 - You can just launch a `bash` in a running container

Task: MongoDB & mongo-express

► Task (part 2):

- Now, we want to start a `mongo-express` container

► For this, we need to ensure:

- That this container is attached to the `mongo-net` network
- That we forward a port from the host to port `8081` of `mongo-express`

► Additional tips:

- For `mongo-express` to be able to connect to MongoDB, you must provide the correct environment variables
- You can find those in the documentation on DockerHub
- Finally, open `mongo-express` on a browser from your host

Docker, Kubernetes & AWS

Docker Compose: **Managing multi-container apps**

Docker Compose: Managing multiple containers

- ▶ **In this chapter you will learn:**

- ▶ How can we use Docker Compose to orchestrate multiple containers at the same time?

- ▶ **How can we launch a multi-container setup?**

- ▶ MySQL
- ▶ WordPress

- ▶ **We will discuss:**

- ▶ How do we write the necessary configuration file?
 - ▶ What is YAML?
 - ▶ How is the `compose.yaml` structured?
 - ▶ What are the most important components?
 - ▶ How does the command `docker compose work`?

Docker, Kubernetes & AWS

What is Docker Compose?

How to handle multi-container setups?

► The problem:

- So far, deploying applications that require multiple containers is a complex and tedious process
 - We must launch each container manually through `docker run`
 - But we still had to connect them manually
- **Of course, we could put it all into a single container:**
 - But this would contradict the philosophy of Docker
 - Docker encourages splitting apps into smaller, independent parts (separation of concerns)
 - We want to stick to this principle to ensure maintainability
- **Isn't there a better way?**
 - Docker Compose!

`docker run` 

`docker run` 

`docker run` 

`docker run` 

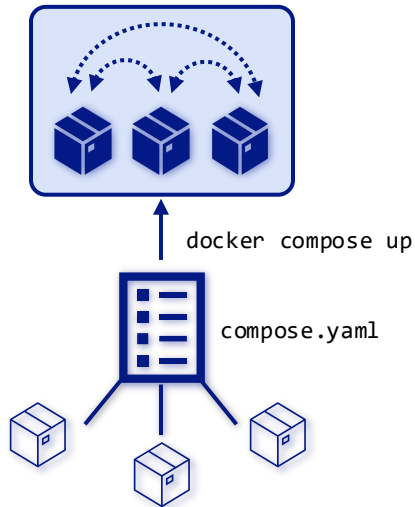
`docker run` 

`docker run` 

How to handle multi-container setups?

► The solution:

- We create a configuration file to manage multiple containers
 - Usually called `compose.yaml`
 - There we define the services using Docker images (or build contexts from `Dockerfiles`)
 - We also specify the application's networks and volumes
- **Once we created this file, we can run:**
 - `docker compose up`
 - Docker Compose will automatically create a private network for us - and launch all specified containers in it
- **Thus:**
 - We can transform multiple `docker run` commands into a single `compose.yaml`





Docker, Kubernetes & AWS

What is YAML?

What is YAML?

- ▶ YAML stands for: **Y**AML **a**in't **M**arkup **L**anguage
- ▶ Data format designed for readability and easy data interchange
 - ▶ Every valid JSON file is also a valid YAML document
- ▶ Commonly used for configuration files
 - ▶ You can use it in almost all programming languages (if there's a library to read it)
- ▶ **We only need a few elements of the YAML Syntax**
 - ▶ YAML uses indentation to indicate structure
 - ▶ Key-value pairs for data representation
 - ▶ Lists are indicated with a hyphen (-)



Docker, Kubernetes & AWS

Configuring WordPress + MariaDB

Configuring WordPress + MariaDB

Element	Function
image	Specifies which Docker image we want to use
environment	Sets environment variables for the container
ports	Allows us to expose a port to the host (important, no whitespace allowed: - 80:80)
restart	Defines the restart policy for the container (e.g., always, on-failure)
volumes	Mounts host directories or volumes (volumes must also be listed as a top-level property volumes)

Docker, Kubernetes & AWS

The command: docker compose

Important docker compose commands

Command	Function
<code>docker compose up [--detach]</code>	Launches multiple containers
<code>docker compose down</code>	Stops the containers, removes the network (volumes will remain)
<code>docker compose stop</code>	Temporarily stops the containers
<code>docker compose start</code>	Starts the containers
<code>docker compose pause</code>	To pause the containers
<code>docker compose unpause</code>	To unpause the containers
<code>docker compose rm</code>	Remove the containers

Docker, Kubernetes & AWS

Accessing environment variables from a compose.yaml

Using environment variables

- ▶ **The goal:**

- ▶ Can we get rid of the passwords in our config file?
- ▶ This would allow us to commit the config file to a code repository (GitHub)... and not have to worry about sharing our password

- ▶ **Within a compose .yaml, we can access environment variables:**

- ▶ `services:`

- `db:`

- `environment:`

- `MARIADB_PASSWORD: ${DB_PASSWORD}`

- ▶ In addition, Docker Compose reads a `.env` file in the same folder:
 - ▶ In this file, we can then configure all passwords
 - ▶ This file would be excluded from our git repository (`.gitignore`)
 - ▶ `DB_PASSWORD=password`

Docker, Kubernetes & AWS

Docker compose: Adding a healthcheck

Docker compose: Adding a healthcheck

- ▶ In our config file, we can also add a property to configure the healthcheck of a container
- ▶ **This will allow us:**
 - ▶ To only launch the WordPress container, if the database has started successfully (`depends_on`)
 - ▶ To automatically re-launch unhealthy containers (through: `autoheal`)

Docker, Kubernetes & AWS

Docker compose: Deploying a custom NodeJS app

Deploying a custom NodeJS app

- ▶ **In this chapter:**

- ▶ We want to deploy a custom NodeJS application (from a `Dockerfile`)

- ▶ **However, our app has a few problems:**

- ▶ It can crash, freeze or just use a lot of CPU resources

- ▶ This serves as an example of a real-world application that contains a few bugs

- ▶ **The challenge:**

- ▶ How can we still deploy this application?

- ▶ **The solution:**

- ▶ Multiple NodeJS instances running to serve the requests

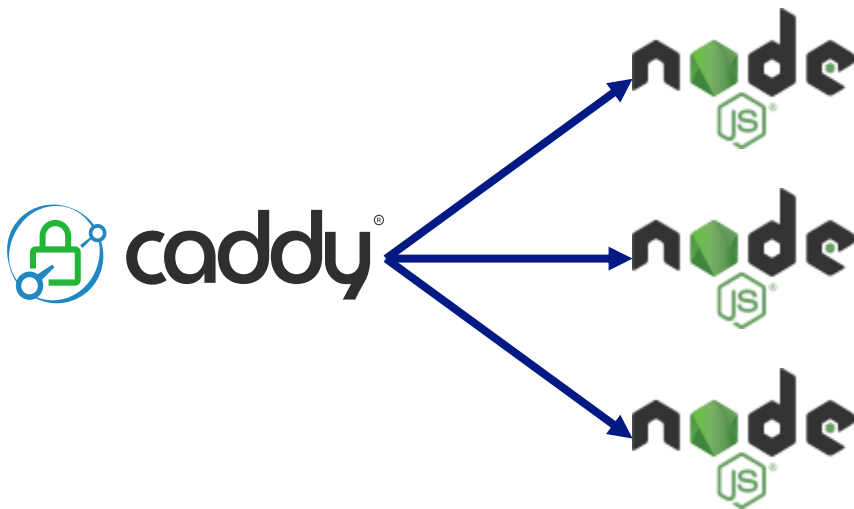
- ▶ 1 Caddy as a load balancer:

- ▶ It needs to be able to deal with a unhealthy instances

- ▶ We will also explore how to benchmark our setup

- ▶ And how to run this setup in production (with `https` certificate)

The architecture of our setup





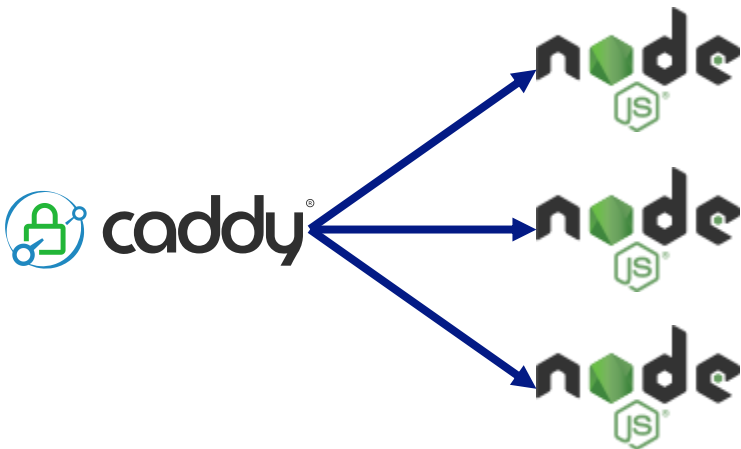
Docker, Kubernetes & AWS

Adding a reverse proxy / load balancer (Caddy)

Caddy as a reverse proxy / load balancer

- ▶ **The idea:**

- ▶ We add a single Caddy as a reverse proxy in front of our NodeJS application(s)
- ▶ This allows us to easily route traffic to all our instances



Docker, Kubernetes & AWS

Dealing with a crash

Dealing with a crash

- ▶ **Crash:**

- ▶ The process crashes
- ▶ The docker container exits / quits
- ▶ And it will be removed from the docker compose network
- ▶ Through this, even the load balancer will detect a crash

- ▶ **How to deal with a crash?**

- ▶ We can just attach a restart policy to the corresponding service
- ▶ This will ensure that this replica will be launched again

- ▶ **Let's have a look at it!**

Docker, Kubernetes & AWS

Overview:

Dealing with a frozen instance

Dealing with a frozen instance

- ▶ **Freeze:**

- ▶ The process becomes unresponsive
- ▶ But it is still running
- ▶ How do we differentiate between an unresponsive process, and a process that just happens to do some heavy work for a few seconds?
- ▶ A freeze is significantly harder to detect

- ▶ **Idea:**

- ▶ **Caddy:**

- ▶ If an instance doesn't reply within a few seconds, we can just try to query another instance
 - ▶ Even if an instance is frozen, the request will then go through (hopefully)

- ▶ **Docker:**

- ▶ We can add a healthcheck + autohealing!

Docker, Kubernetes & AWS

Exercise: Deploying this application on your own server

Exercise: Deploying this on a server

► Exercise:

- Deploy this application on your own server
- Be sure that this application restarts once the server reboots

► The steps:

- 1) If you have purchased a domain, ensure that the DNS records point to the server you're deploying this application to
- 2) Upload the application to the server
(maybe without the `./bad-app/node_modules` folder)
- 3) Launch the application on the server, be sure nothing else is running on port 80!
- 4) Be sure that port 80 is exposed to port 80 on the server
- 5) If you want to enable https (own domain required):
 - Ensure that enough time has passed since step 1
 - Ensure that port 443 is exposed for the service running the caddy
 - The Caddyfile should explicitly allow-list the domains for https

Docker, Kubernetes & AWS

Deployment on ECS

Deployment on ECS

- ▶ **In this chapter:**

- ▶ We want to explore how we can deploy an application to Amazon ECS (Elastic Container Service)

- ▶ **ECS offers significantly more features than AppRunner:**

- ▶ Offers significantly more control over the configuration and deployment of our application
 - ▶ This makes it more difficult to setup than AppRunner

- ▶ **But we're getting way more flexibility:**

- ▶ We can deploy multi-container services
 - ▶ We can connect containers to each other
 - ▶ We can easily mount external file systems

- ▶ **Overall:**

- ▶ Better suited for managing large and complex applications

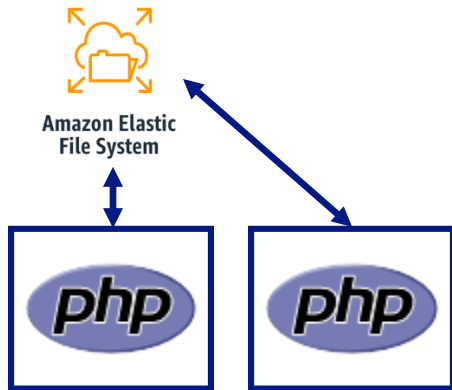
The project for this chapter

► In this chapter:

- We want to deploy an image upload form (in PHP, no database connection)
- Those files must be stored in a centralized way
- If we stored them within an individual container:
 - They would not replicate to other replicas
 - They would get lost during scaling

► The idea:

- We deploy a container with PHP on ECS
- But the actual files are stored within an EFS
 - EFS is a file system, that we can attach to our containers
 - More on it later!
- We will also configure a load balancer to route traffic to all our PHP instances!



The second project

- ▶ We will also see how we can deploy a WordPress blog
- ▶ For this, we need a similar setup
- ▶ **But we do need a database:**
 - ▶ We will explore how to create a database in the AWS cloud
 - ▶ And then give our WordPress sufficient permissions to access it!



Docker, Kubernetes & AWS

AWS ECS:

Deploying a php container (part 1)

ECS: Deploying a container

- ▶ We now want to start the deployment of the php container
- ▶ **For this, we must:**
 - ▶ **Create a task definition**
(this describes the task that should be run)
 - ▶ **Create a cluster**
(this is where the task will be run)
 - ▶ **Create a service in our ECS cluster**
(telling the cluster to run a task)

ECS: Creating a task definition

- ▶ **A task definition:**

- ▶ Defines the task that we want to run
- ▶ Immutable, once we create a task definition, we can't change it anymore...
 - ▶ ... but we can create a new revision of it

- ▶ **We need to configure:**

- ▶ **On which hardware the task can be run**

- ▶ **In our case:**

- ▶ We will deploy the task to AWS Fargate
 - ▶ Amazon will automatically manage the server (= Docker host) for us
 - ▶ Otherwise, we would have to do more configuration

- ▶ **What should get executed:**

- ▶ The image, options for execution,...

- ▶ **Additional configuration options (storage, etc.)**

Task vs. container

- ▶ A task can consist out of multiple containers
- ▶ Let's say we had a task that needed a local Redis instance
- ▶ In this case, this tasks would consist out of 2 containers:
 - ▶ PHP
 - ▶ Redis
- ▶ When scaling, this whole task is scaled horizontally
- ▶ Usually, we would not want this for Redis!
- ▶ In this course, we will focus on single-container tasks



Docker, Kubernetes & AWS

AWS ECS:

Deploying a container (part 2)

ECS: Creating a cluster

- ▶ **We now want to create a cluster:**

- ▶ For this, we need to specify which hardware this cluster provides
- ▶ In our case, we will run our cluster exclusively on AWS Fargate
- ▶ We will then be able to run all task definitions, that allow deployment on Fargate

- ▶ **Let's have a look at this!**

ECS: Creating a service

- ▶ Once we created the cluster, we want to create a service
- ▶ This will allow us to then actually deploy our tasks
- ▶ **A task:**
 - ▶ An individual stack of containers that we want to deploy
 - ▶ It must be defined ahead of time (through a task definition)
 - ▶ This enables us to run the same task definition multiple times
 - ▶ If a task crashes, or stops... the whole task is stopped
- ▶ **A service:**
 - ▶ A desired end state that should be reached
 - ▶ Will automatically create the tasks needed to reach this end state
 - ▶ If a task crashes, a service will automatically try to deploy another one
 - ▶ We can even update a service to run a different task definition (or revision)...
 - ▶ ... which will then seamlessly manage the tasks to fulfill it

Docker, Kubernetes & AWS

What is EFS?

What is EFS?

- ▶ **EFS: Elastic File System**

- ▶ Scalable file system solution for multiple EC2 instances
- ▶ The same file system can be mounted by multiple instances at the same time
- ▶ This is important, as it allows us to easily replicate our container
- ▶ We don't have to worry about storage limits, it can easily scale
- ▶ It is available within the VPC that we created it in
(but could also be made available to other VPCs)

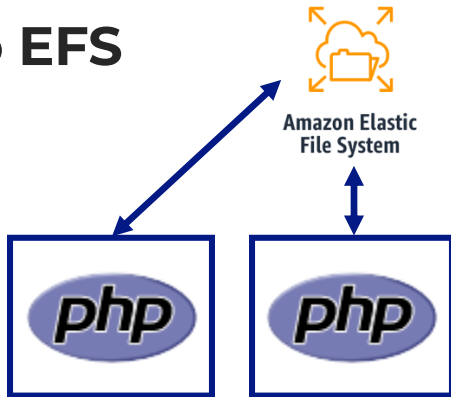
- ▶ **Let's create an EFS volume!**

Docker, Kubernetes & AWS

AWS TransferFamily: Uploading our application to EFS

Uploading our application to EFS

- ▶ We need to get our application (php code) into our containers
- ▶ **The best way would be to create our own image:**
 - ▶ We would derive this from the php base image
 - ▶ And then just add our own code to it
- ▶ **Here however, we will follow a different approach:**
 - ▶ We will store the complete application on EFS (including php files)
 - ▶ And just mount this volume into the php containers
 - ▶ This is a little bit less efficient, as php files are on a network mount
 - ▶ But easier for our project here - I don't have to show the whole setup with ECR again
 - ▶ In addition... it allows us to explore how to access EFS directly (from our computer)



AWS TransferFamily

- ▶ For accessing EFS from our own computer, we need a way to mount it
- ▶ We can't access it directly, as our own computer is not within the VPC of the EFS volume
- ▶ We thus need to put a server in between, for managing access
- ▶ **AWS TransferFamily:**
 - ▶ A managed service that allow us to access our EFS volume from outside
 - ▶ It can expose a SFTP server for us
 - ▶ Which then exposes the EFS volume to us
 - ▶ **Important, heads-up:**
 - ▶ As of recording, AWS TransferFamily is surprisingly expensive... about \$0.30 per hour
 - ▶ It may not sound like a lot... but this is \$216 per month!
 - ▶ **Thus, be sure to turn it off as soon as you don't need it anymore!**

Docker, Kubernetes & AWS

AWS ECS: Accessing EFS from a task

Mounting an EFS volume

- ▶ **To mount an EFS volume:**

- ▶ We need to update the **task definition**, to connect to the EFS volume
- ▶ In addition, we need to mount this volume into the **container**

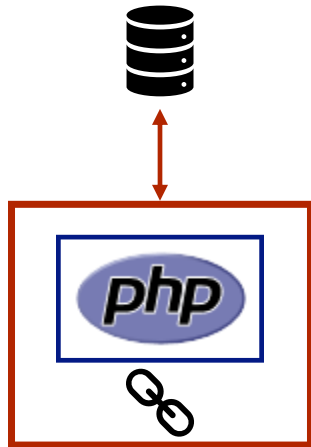
- ▶ **However:**

- ▶ **It must be possible to access EFS from our ECS task:**

- ▶ The security groups of EFS / ECS must allow this
- ▶ To change the networking configuration, we will have to create a new service

- ▶ **EFS must allow the access:**

- ▶ Luckily, we don't have to change anything here
- ▶ **Let's just explore this!**



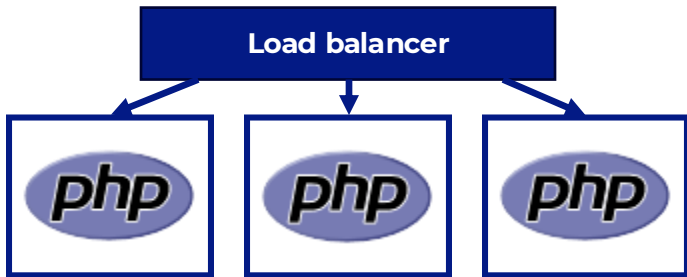
Docker, Kubernetes & AWS

AWS ECS:

Adding a load balancer

Adding a load balancer

- ▶ When creating the service, we can easily attach a load balancer to it
- ▶ In this course, we will focus on an application load balancer
- ▶ **It works on the application layer (http / https):**
 - ▶ The load balancer "*understands*" the meaning of http / https
 - ▶ Things can get rewritten / changed (such as caching headers, ...)
 - ▶ Our application does not need to implement anything about https - the load balancer can encrypt the traffic for us



Load balancer & target group

- ▶ **How does the load balancer know where it can send requests to?**
- ▶ **This is defined through a target group**
- ▶ **What is a target group?**
 - ▶ It bundles multiple "things" together that the load balancer could send traffic to
 - ▶ Luckily for us, the ECS service will configure the target group for us
 - ▶ And even keep it up to date!



Docker, Kubernetes & AWS

AWS ECS:

Configuring a domain

AWS ECS: Enabling a domain

- ▶ We have now already added a load balancer to our service
- ▶ We can now point a domain to this load balancer
- ▶ For this, we just need to configure the proper DNS entries
- ▶ **We have already seen how to configure the main domains:**
 - ▶ www.docker-test.click
 - ▶ docker-test.click
- ▶ **Thus, we will now configure a subdomain:**
 - ▶ memories.docker-test.click

Docker, Kubernetes & AWS

AWS ECS: Enabling https

AWS ECS: Enabling https

- ▶ We now want to enable https, so that our website is reachable through an encrypted connection
- ▶ **For this:**
 - ▶ We first need to generate the SSL certificate
 - ▶ After this, we can then enable https for our load balancer

Docker, Kubernetes & AWS

Project ECS: Overview, Hosting a WordPress

Project 2: Hosting a WordPress

- ▶ We now want to host a WordPress on our ECS cluster
- ▶ **For this, we need to configure a database:**
 - ▶ WordPress requires MySQL / MariaDB
 - ▶ However, we can't really host it on ECS
 - ▶ ECS is ideal for services, that can get scaled / replicated easily
 - ▶ **This is not the case for a database:**
 - ▶ Adding another replica is significantly more difficult
 - ▶ We would have to configure a MySQL / MariaDB cluster
- ▶ **However, we can use a managed service instead:**
 - ▶ Amazon RDS (Relational Database Service)

Docker, Kubernetes & AWS

Docker: Retrospective

Docker: Retrospective

- ▶ **You now know everything you need to know about docker:**

- ▶ Docker on the command line
- ▶ Dockerfiles
- ▶ Networking in Docker
- ▶ Managing multiple containers with docker compose

- ▶ **You can also deploy containers in various forms:**

- ▶ Your own server
- ▶ AppRunner
- ▶ Amazon ECS
- ▶ You can configure a load balancer, and offload assets to S3

- ▶ **This is a lot, congratulations so far already!**

- ▶ **If you're not interested in Kubernetes:**

- ▶ This chapter is the end of this course

Practice test

- ▶ **Practice test:**

- ▶ In the next lecture, you'll find a practice test that you can use to test your docker skills
- ▶ Feel free to do it now, but there's no need to – you can also do it at the end of this course

Docker, Kubernetes & AWS

Kubernetes



What is Kubernetes (K8s)?

► **Kubernetes:**

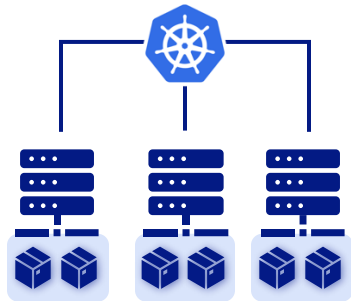
- Kubernetes is a platform for automating the deployment, scaling, and operating of application containers across clusters of hosts
- The name "Kubernetes" means "helmsman" or "pilot" in Greek

► **Main idea:**

- Its goal is to provide a framework for running distributed systems
- It manages the lifecycle of containerized applications and services (deployment, scaling, maintenance,...)
- K8s allows us to specify the desired infrastructure for our cluster, providing the flexibility to deploy on different cloud providers

► **Development & Open Source:**

- Kubernetes was developed by Google
- It has been open source since 2015, and now it is maintained by the Cloud Native Computing Foundation (CNCF)



What Kubernetes is not

- ▶ Does not deploy source code or build applications
- ▶ Does not manage physical machines
- ▶ Does not provide application-level services

Key Points

- ▶ Kubernetes encompasses a broad range of topics
- ▶ This course provides a concise introduction
- ▶ **Goal (in this course):**
 - ▶ Learn the basics to effectively communicate with cloud administrators about Kubernetes
 - ▶ Deploy a "simple" WordPress setup, first locally, and then in the AWS cloud

Outlook



- ▶ **Today, in this chapter:**

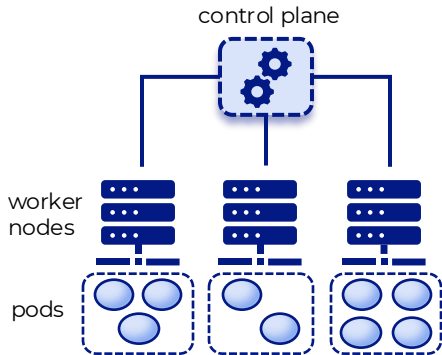
- ▶ We will launch a first, simple Kubernetes cluster (locally)
- ▶ **We will explore the basics about Kubernetes:**
 - ▶ What is Kubernetes?
 - ▶ How does the cluster work?
 - ▶ What is the control plane?
 - ▶ How to run and access the Kubernetes Dashboard
 - ▶ We will launch the first container in the Kubernetes cluster

Docker, Kubernetes & AWS

Kubernetes: Quick overview

Quick overview: Kubernetes

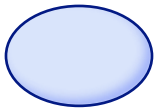
- ▶ A Kubernetes cluster consists out of many components
- ▶ **For now:**
 - ▶ **Control plane:**
 - ▶ Responsible for managing the cluster
 - ▶ Tells the worker nodes what to do
 - ▶ **Worker nodes:**
 - ▶ The physical machines running the cluster
 - ▶ A Kubernetes cluster must consist out of at least a single worker node
 - ▶ **Pods:**
 - ▶ The actual tasks running in our cluster
 - ▶ They contain the containers required for our application



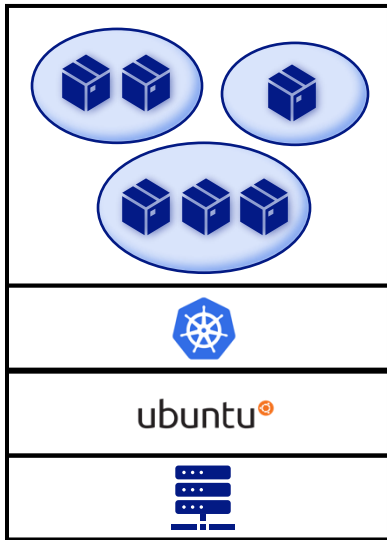
Docker, Kubernetes & AWS

Kubernetes: What is a pod?

Kubernetes Basics: Pods



- ▶ Interacts with the container engine at an abstraction level
- ▶ **Pods: Smallest deployable unit**
 - ▶ High-level perspective on container-based applications
 - ▶ A pod can contain one or multiple containers
 - ▶ Multiple pods can be placed on the same physical machine
 - ▶ A pod can't be sub-divided, it is either run on a single machine or not at all
 - ▶ Pods are ephemeral (temporary)
 - ▶ If a physical machine fails, the pod will not be restarted
 - ▶ To change the configuration, a new pod can be created



Docker, Kubernetes & AWS

Enabling Kubernetes, Managing the cluster: kubectl

Enabling Kubernetes

- ▶ **To use Kubernetes:**

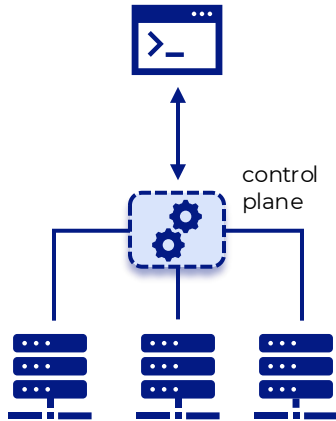
- ▶ We must launch a Kubernetes cluster on our local machine
- ▶ Luckily, this is already integrated in Docker Desktop
- ▶ We just need to enable it

- ▶ **Let's have a look at this!**

Managing the cluster: kubectl

► What is kubectl?

- Command-line tool to interact with Kubernetes clusters
- Allows us to deploy applications, inspect and manage cluster resources
- kubectl connects to the control plane of our cluster (*context*)
- We can use kubectl to either configure the cluster directly - or through `.yaml` files (just as with the `docker` command)
- **Before using kubectl, let's confirm it is connected to the right cluster:**
 - `kubectl config get-contexts`
 - `kubectl config use-context [context]`



First steps with kubectl

- ▶ **kubectl version**

- ▶ It will print out the client and the server version (among others)

- ▶ **To display the version information in YAML format:**

- ▶ `kubectl version --output=yaml`

- ▶ **To show help:**

- ▶ `kubectl version --help`

- ▶ **kubectl get [resource]:**

- ▶ Allows us to print information about K8s resources

- ▶ **kubectl get nodes:** Will print all nodes (state should be: ready)

- ▶ **kubectl get pods:**

- ▶ Allows us to print information about running pods

- ▶ We can add `-A` to show pods from all namespaces

- ▶ **kubectl get all -A:**

- ▶ Shows us an overview about what's happening on the cluster

Major difference!

- ▶ **In docker, the syntax has been:**

- ▶ `docker [resource] [action]`
- ▶ `docker container ls`

- ▶ **In kubectl, when accessing something, the order is:**

- ▶ `kubectl [action] [resource]`
- ▶ `kubectl get nodes`

Docker, Kubernetes & AWS

Launching a first pod

Launching a first pod

- ▶ We now want to run a first "thing" in our cluster
- ▶ **Let's just launch an ubuntu pod:**
 - ▶ `kubectl run [name] [options]`
 - ▶ `kubectl run my-ubuntu --image=ubuntu -it`
 - ▶ This will launch a pod with a single container, based on the ubuntu image
- ▶ **We can exit the pod with the exit command:**
 - ▶ `exit`
 - ▶ However, the pod will restart after that
(unlike with a docker run)
- ▶ **Important:**
 - ▶ The lifecycle of pods is managed independently from docker
 - ▶ Kubernetes will restart the pod
(a few times, depending on the restartPolicy)



The restart policy



- ▶ In Kubernetes, we can configure a restart policy for a pod
- ▶ **We can do this by specifying `--restart` for `kubectl run`:**
 - ▶ `kubectl run my-ubuntu2 --restart=Never --image=ubuntu -it`
 - ▶ If this pod terminates, it will not be restarted
- ▶ **Possible values:**
 - ▶ **Never:** Never restarts this pod
 - ▶ **Always:** This pod will always be restarted (*default*)
 - ▶ **OnFailure:** This pod will be restarted if the container exits with an error
- ▶ **Heads-up:**
 - ▶ Kubernetes doubles the wait time after each failed restart attempt, up to a maximum of 300s
- ▶ **Example:**
 - ▶ 10s delay -> 20s delay -> 40s delay -> ... -> up to 300s delay

To stop the pod

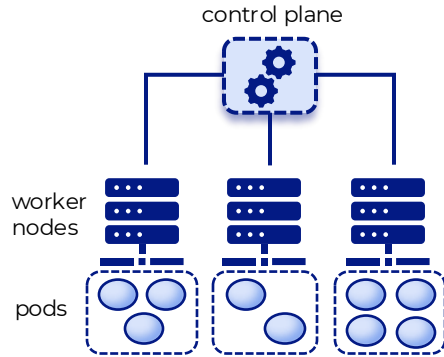
- ▶ **To stop the pod:**

- ▶ We can use `kubectl` for this
- ▶ `kubectl delete pod my-ubuntu`



Docker, Kubernetes & AWS

How does Kubernetes work?



Structure of the control plane

- ▶ The control plane consists of the following components:

- ▶ **kube-apiserver:**

- ▶ Frontend for node communication, provides the API
 - ▶ Sole entry point for cluster management (security)

- ▶ **etcd:**

- ▶ Stores configuration, application states, and cluster metadata

- ▶ **kube-scheduler:**

- ▶ Distributes pods based on resources

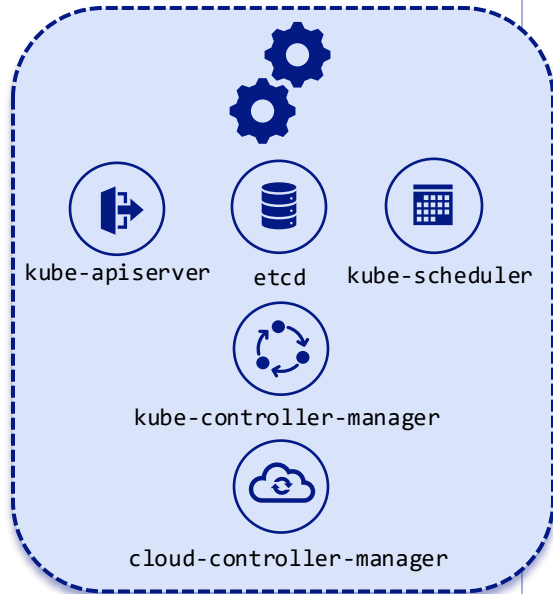
- ▶ **kube-controller-manager:**

- ▶ Monitors nodes and responds to changes in state

- ▶ **cloud-controller-manager:**

- ▶ Handles cloud-specific monitoring

- ▶ **The control plane can be split across multiple nodes**



The components of a node

► Node components:

► **kubelet:**

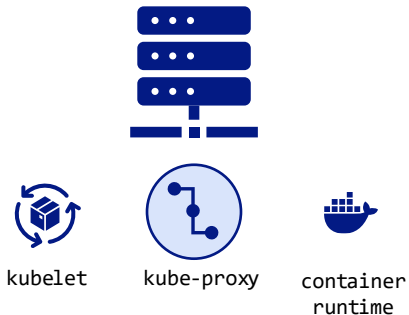
- Runs pods containing containers
- Executes according to requests from kube-scheduler

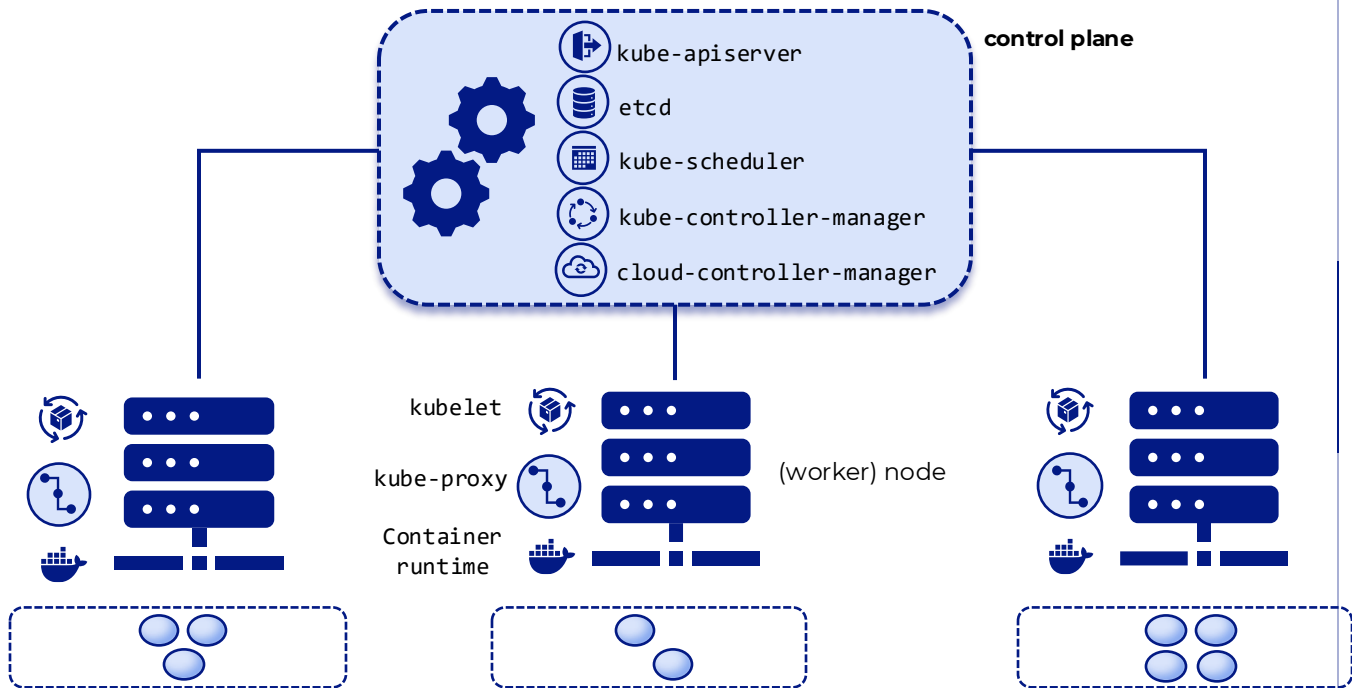
► **kube-proxy:**

- Implements network rules
- Manages connection forwarding to the host
- Forwards requests between pods and ensures efficient communication

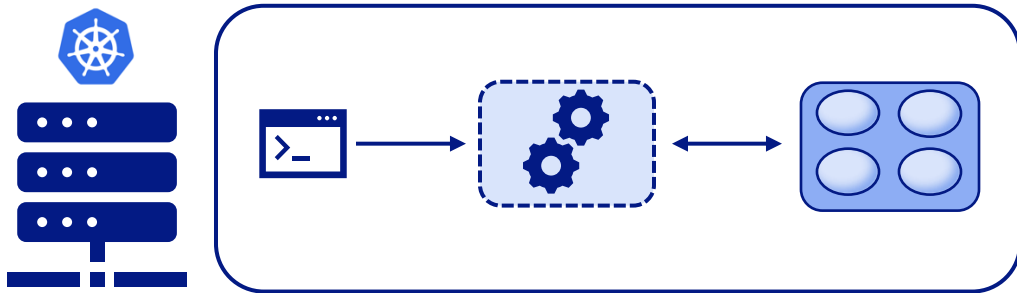
► **Container runtime:**

- For example: Docker





How does kubectl work





Docker, Kubernetes & AWS

Installing software: helm

Kubernetes package manager: Helm



- ▶ **Package manager for Kubernetes:**

- ▶ Manages Kubernetes applications through Helm charts
- ▶ A Helm chart packages all necessary components of a Kubernetes application

- ▶ **To use Helm:**

- ▶ The Kubernetes cluster must be up and running
- ▶ kubectl must be installed and configured to communicate to the cluster
- ▶ We must install helm on our local machine

- ▶ **Let's have a look at this!**

Docker, Kubernetes & AWS

Kubernetes: Dashboard

K8s: Dashboard

- ▶ **K8s Dashboard:** Graphical interface for K8s
 - ▶ Like *Docker Desktop*, but for our K8s cluster
 - ▶ Allows us to visually inspect the state of our cluster
 - ▶ We can even manage the cluster from it:
 - ▶ We can even launch a terminal in a pod!
- ▶ **However, to launch the dashboard:**
 - ▶ We first must install it (through `helm`), and launch it
 - ▶ It will then be running in the cluster - we can't access it yet
 - ▶ For this, we must expose its port (through a proxy)
- ▶ **After that, to login to the dashboard:**
 - ▶ Then, we need to create a user
 - ▶ And create an authentication token
 - ▶ This token, we can then use to access the dashboard

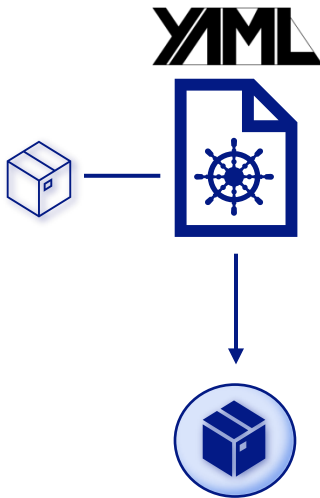


Docker, Kubernetes & AWS

Using `.yaml` for configuration

YAML: Deployment of a pod

- ▶ We can also deploy a pod through a `.yaml` file
- ▶ **The advantages:**
 - ▶ We could store the configuration in a `git` repository
 - ▶ This allows us to replicate the setup without executing CLI commands in a specific order
 - ▶ It also simplifies updates
- ▶ **In addition:**
 - ▶ Later, configuration will become increasingly complex
 - ▶ This would not be possible with simple CLI commands
- ▶ First, we must write the `.yaml` file
- ▶ **Then, we can apply it:**
 - ▶ `kubectl apply -f pod.yaml`



YAML file: Most important properties

- ▶ **apiVersion:**

- ▶ Relates to the version of the K8s API

- ▶ **kind:**

- ▶ The kind of object to deploy
- ▶ It must exist in the corresponding apiVersion
(e.g. Pod is always under v1)

- ▶ **metadata:**

- ▶ Used to define a name and labels to access this object

- ▶ **spec:**

- ▶ Describes the desired goal state (specification) for this object
- ▶ Properties depend on the type of object

- ▶ **status:**

- ▶ Describes the status of this object in the cluster
(only when asking the cluster to provide us the YAML back)

Docker, Kubernetes & AWS

Overview: Deploying applications

Overview: Deploying applications

► In this chapter:

- We want to learn the basics for deploying applications in a Kubernetes cluster
- We will launch WordPress containers and a MariaDB database
- At the end of this chapter, you will be able to launch persistent containers (workloads) in a Kubernetes cluster

► For this, we will explore:

- Workloads, different workload types
- Deployment / ReplicaSet for stateless applications
- StatefulSet for stateful applications
- How to set environment variables
- How to securely store environment variables (secrets)



Docker, Kubernetes & AWS

What is a workload?

Workload types in K8s

► Workload:

- A containerized application, that is being run in a K8s cluster
- Kubernetes offers various types of workloads
- They define, how the cluster executes the application

► Example workload types:

► ReplicaSet / Deployment:

- Used for stateless applications

► StatefulSet:

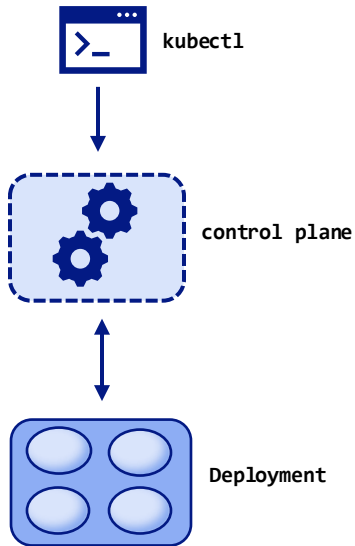
- For stateful applications

► DaemonSet:

- Will be run on every node within the cluster

► Job:

- A task that is just run a specific number of times
- Once it is finished, the job is complete

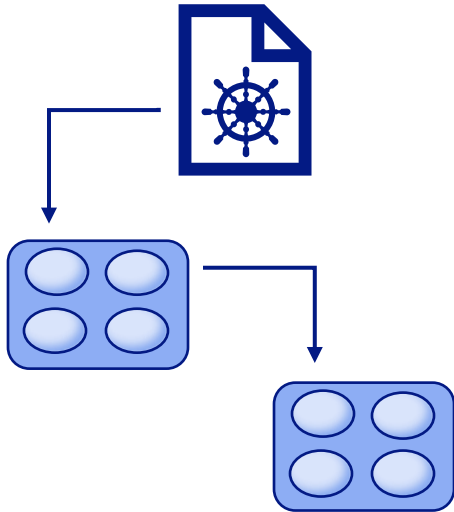


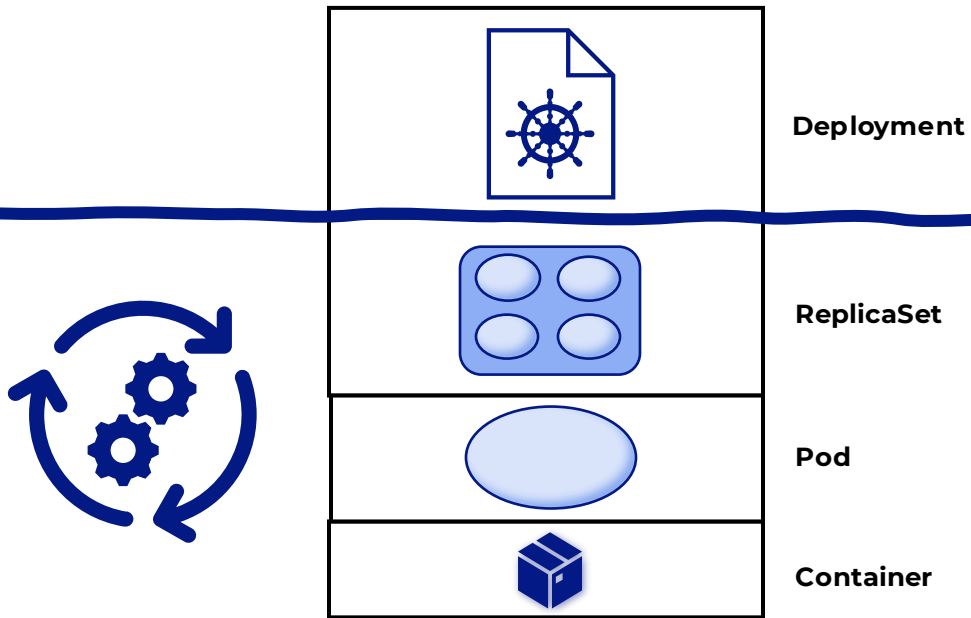
Docker, Kubernetes & AWS

Deploying a stateless application: ReplicaSet vs. Deployment

ReplicaSet vs. Deployment

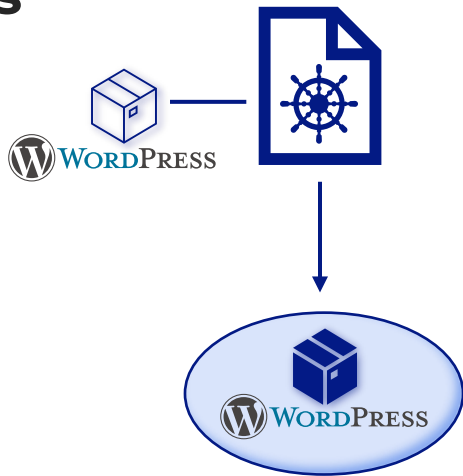
- ▶ We now want to execute a stateless application:
 - ▶ A simple WordPress web server
- ▶ **For this, we have 2 different options:**
 - ▶ **ReplicaSet:**
 - ▶ Ensures that our cluster executes a specified number of copies of this pod
 - ▶ **Deployment:**
 - ▶ Manages a ReplicaSet, but allows us to migrate over to a new ReplicaSet
 - ▶ This enables seamless updates of our application





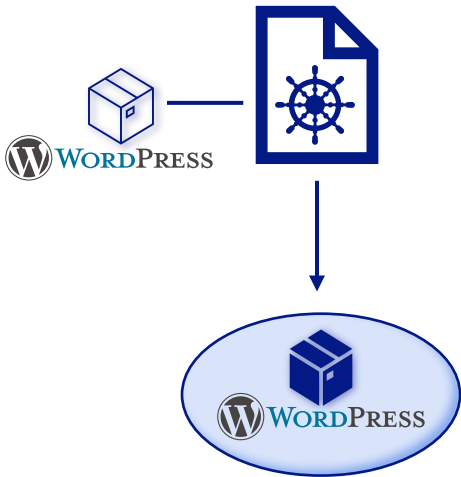
Deployment of a WordPress

- ▶ Let's now explore this by deploying WordPress
- ▶ **We can create a new deployment with:**
 - ▶ `kubectl create deployment [name] --image=[image]`
 - ▶ `kubectl create deployment wordpress-deployment --image=wordpress --replicas=3 --port=80`
 - ▶ The image specified by the `--image` parameter must be available through DockerHub
 - ▶ `--replicas` allows us to specify the number of replicas
 - ▶ `--port` is for documentation purposes (similar to EXPOSE in a Dockerfile)
- ▶ **Once created, we can inspect / edit the configuration:**
 - ▶ `kubectl edit deployment/wordpress-deployment`



How to access WordPress

- ▶ Now, WordPress is running within our Kubernetes cluster
- ▶ However, the Kubernetes network is isolated from our computer's network
- ▶ Kubernetes includes a networking stack that allows pods to communicate with each other
- ▶ **For now, we need to launch a port-forwarding proxy:**
 - ▶ `kubectl port-forward deployment/wordpress-deployment 8080:80`
 - ▶ It will accept incoming connections on port 8080
 - ▶ And forward them to the WordPress deployment
 - ▶ **Important:**
 - ▶ This functionality is only meant for development
 - ▶ We should not use `port-forward` in production
 - ▶ Don't worry about this for now; we will explore different mechanisms later

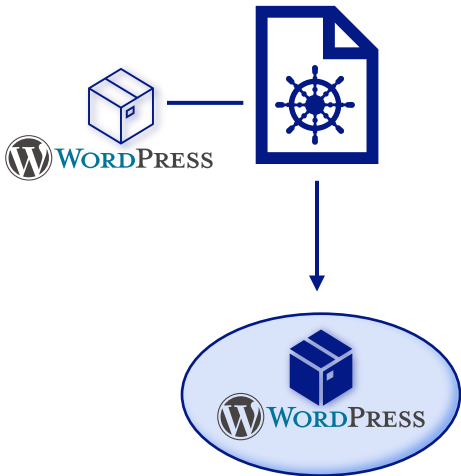


Docker, Kubernetes & AWS

Deploying WordPress (YAML)

Example: deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```



Docker, Kubernetes & AWS

Labels of a deployment

Labels of a deployment

- ▶ When specifying our deployment, we attached labels
- ▶ **Why do we need them?**
 - ▶ **They allow us to easily find our deployment:**
 - ▶ `kubectl get deployments -l app=wordpress`
 - ▶ Which labels we use, is entirely up to us
 - ▶ **We can even use filter by multiple labels:**
 - ▶ `kubectl get deployments -l app=wordpress,version=1.0`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
    version: '1.0'
```

Docker, Kubernetes & AWS

Labels for pods

Labels for the pods

► `spec.template.metadata.labels`:

- Allows us to specify labels for each pod
- **We can filter the pods by this label**
 - `kubectl get pods -l app=wordpress`

► `spec.selector.matchLabels`:

- It allows a deployment to find its pods
- **Important:** This property is immutable. It cannot be changed once a deployment is created
- **Why aren't the `spec.template.metadata.labels` enough?**
 - We must specify the labels for finding separately
 - The `matchLabels` should allow future updates
 - **Important:** As the `matchLabels` are immutable, we usually want to omit filtering by version number,...

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

Docker, Kubernetes & AWS

From Deployment to StatefulSet

From Deployment...

- ▶ **A deployment:**

- ▶ Is used to deploy stateless pods
- ▶ All pods can be launched at the same time

- ▶ **A pod in a deployment:**

- ▶ Is considered interchangeable to any other pod
(from the same deployment)
- ▶ When launched, it gets assigned a random identifier
- ▶ When scaling down, Kubernetes will choose randomly, which pod to terminate

... to StatefulSet

- ▶ **A StatefulSet:**

- ▶ Is used to deploy stateful pods

- ▶ **A pod in a StatefulSet:**

- ▶ Kubernetes maintains a sticky identity for each of its pods
 - ▶ They are created from the same spec - but are not interchangeable
 - ▶ Each pod is considered to perform a unique role

- ▶ **When scaling up:**

- ▶ Pods are created sequentially
 - ▶ Predecessors must be running and ready

- ▶ **When scaling down:**

- ▶ All its successors must be shutdown
 - ▶ Pods will be terminated in reverse order

Docker, Kubernetes & AWS

Environment variables

Environment variables

- ▶ **So far, the database failed to even launch:**

- ▶ To launch the database, we must set the MariaDB root password

- ▶ **For this, we must set an environment variable:**

- ▶ `spec.template.containers[0]`
 - ▶ `env:`
 - `name: MARIADB_ROOT_PASSWORD`
 - `value: "password"`

Docker, Kubernetes & AWS

Secrets

Secrets in Kubernetes

► A secret in Kubernetes:

- Object to store sensitive information securely
- **Examples:** passwords, OAuth tokens, TLS keys,...
- It decouples the sensitive information from the rest of the configuration
- The data is encoded in Base64 format

► Why do we need a secret?

- We could store the secret in a separate file... which does not get committed to the git repository
- This secret can then get managed separately

► In addition:

- We could implement more granular access control in Kubernetes
- Kubernetes can be configured to encrypt the data at rest
- **This is not part of this course!**

apiVersion: v1

kind: Secret

metadata:

name: my-mariadb-secret

type: Opaque

data:

username: YWRtaW4=

password: c2VjcmV0cGFzc3dvcmQ=

Using a secret

- ▶ **The easiest way to use a secret is to have it turned into an environment variable:**

- ▶ `env:`
 - `name: MARIADB_USER`
 - `valueFrom:`
 - `secretKeyRef:`
 - `name: my-mariadb-secret`
 - `key: username`

- ▶ *(there're also other ways for using a secret that are not part of this course)*

```
apiVersion: v1
kind: Secret
metadata:
  name: my-mariadb-secret
type: Opaque
data:
  username: YWRtaW4=
  password: c2VjcGV0cGFzc3dvcmQ=
```

Heads-up

- ▶ When deploying Kubernetes on the infrastructure of a cloud provider, you could also use the cloud providers secret mechanism
- ▶ **Example AWS:**
 - ▶ AWS Secrets Manager
 - ▶ This is independent of Kubernetes
 - ▶ You could choose to store all the secrets in there
 - ▶ In this case, AWS / IAM would be responsible for the permission management

Docker, Kubernetes & AWS

Using your own images, imagePullPolicy

Using your own images

- ▶ We can use Kubernetes with our own images
- ▶ **For this:**
 - ▶ All images must be available on all nodes in the cluster
- ▶ **In a single-node cluster:**
 - ▶ We can use all images usable by Docker locally
- ▶ **In a cloud setup:**
 - ▶ We should configure the cluster to pull images from our own, private repository
 - ▶ Cloud hosting providers usually provide a private repository for us (such Elastic Container Repository for AWS)

Setting the `imagePullPolicy`

- ▶ Kubernetes may automatically pull images again
- ▶ This can be configured through the `imagePullPolicy`
- ▶ **Possible values:**
 - ▶ **IfNotPresent:** Only pulls the image if it's not present on the local node
 - ▶ **Never:** Never pulls an image. If the image is not present, it will fail
 - ▶ **Always:**
 - ▶ Will always pull the image
 - ▶ Similar to a `docker pull` every time the container launches
 - ▶ Still utilizes docker's caching mechanism
- ▶ **By default:**
 - ▶ **IfNotPresent:** Usually used
 - ▶ **Always:**
 - ▶ Used when referring to the tag `:latest`, or using an image without a tag

Docker, Kubernetes & AWS

Overview: Deploying applications

Overview: Deploying applications

► In this chapter:

- We want to learn the basics for deploying applications in a Kubernetes cluster
- We will launch WordPress containers and a MariaDB database
- At the end of this chapter, you will be able to launch persistent containers (workloads) in a Kubernetes cluster

► For this, we will explore:

- Workloads, different workload types
- Deployment / ReplicaSet for stateless applications
- StatefulSet for stateful applications
- How to set environment variables
- How to securely store environment variables (secrets)



Docker, Kubernetes & AWS

What is a workload?

Workload types in K8s

► Workload:

- A containerized application, that is being run in a K8s cluster
- Kubernetes offers various types of workloads
- They define, how the cluster executes the application

► Example workload types:

► ReplicaSet / Deployment:

- Used for stateless applications

► StatefulSet:

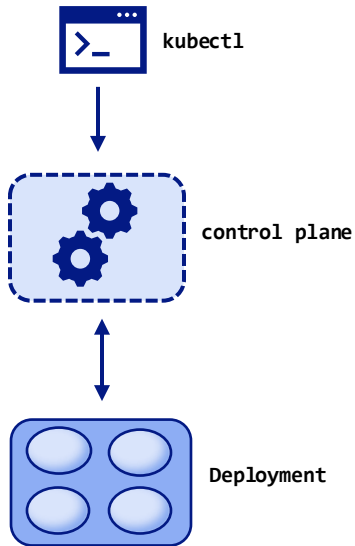
- For stateful applications

► DaemonSet:

- Will be run on every node within the cluster

► Job:

- A task that is just run a specific number of times
- Once it is finished, the job is complete

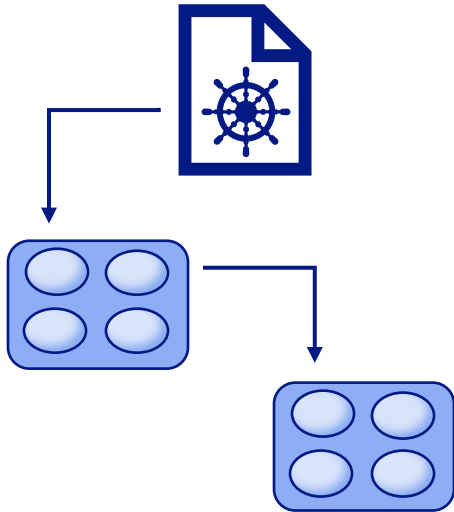


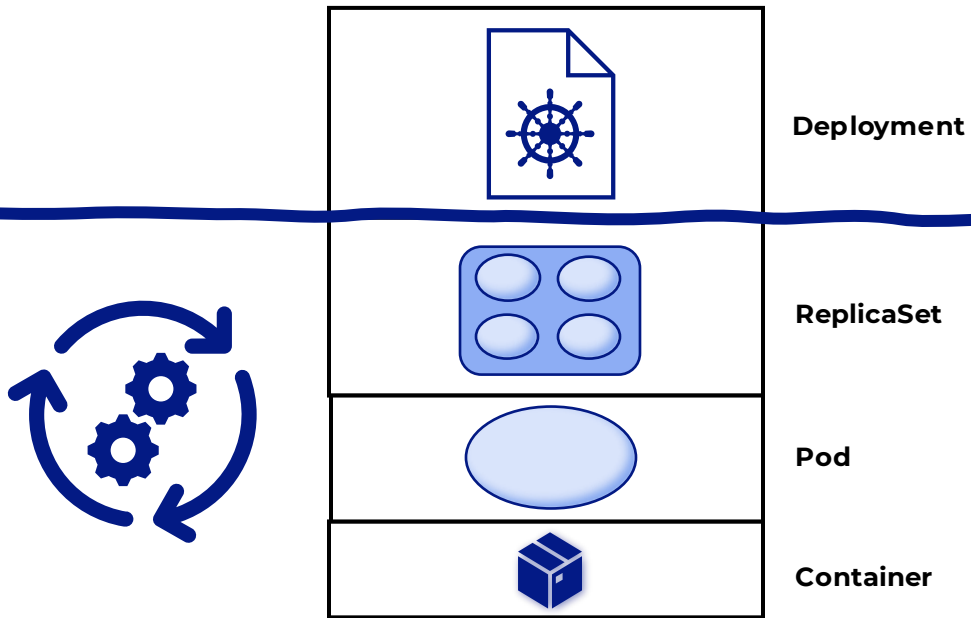
Docker, Kubernetes & AWS

Deploying a stateless application: ReplicaSet vs. Deployment

ReplicaSet vs. Deployment

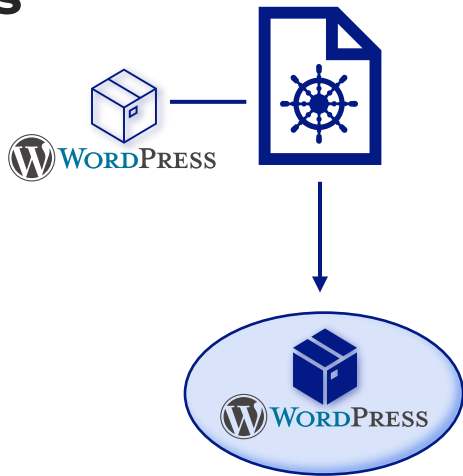
- ▶ We now want to execute a stateless application:
 - ▶ A simple WordPress web server
- ▶ **For this, we have 2 different options:**
 - ▶ **ReplicaSet:**
 - ▶ Ensures that our cluster executes a specified number of copies of this pod
 - ▶ **Deployment:**
 - ▶ Manages a ReplicaSet, but allows us to migrate over to a new ReplicaSet
 - ▶ This enables seamless updates of our application





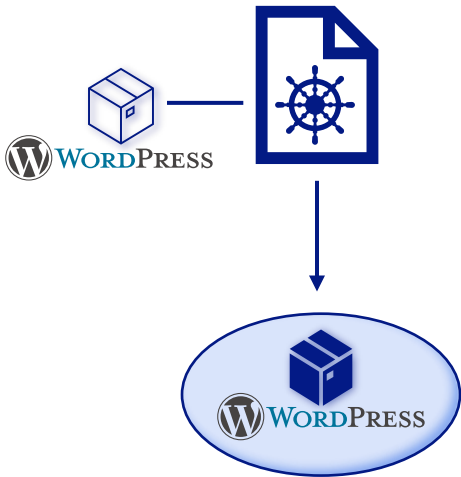
Deployment of a WordPress

- ▶ Let's now explore this by deploying WordPress
- ▶ **We can create a new deployment with:**
 - ▶ `kubectl create deployment [name] --image=[image]`
 - ▶ `kubectl create deployment wordpress-deployment --image=wordpress --replicas=3 --port=80`
 - ▶ The image specified by the `--image` parameter must be available through DockerHub
 - ▶ `--replicas` allows us to specify the number of replicas
 - ▶ `--port` is for documentation purposes (similar to EXPOSE in a Dockerfile)
- ▶ **Once created, we can inspect / edit the configuration:**
 - ▶ `kubectl edit deployment/wordpress-deployment`



How to access WordPress

- ▶ Now, WordPress is running within our Kubernetes cluster
- ▶ However, the Kubernetes network is isolated from our computer's network
- ▶ Kubernetes includes a networking stack that allows pods to communicate with each other
- ▶ **For now, we need to launch a port-forwarding proxy:**
 - ▶ `kubectl port-forward deployment/wordpress-deployment 8080:80`
 - ▶ It will accept incoming connections on port 8080
 - ▶ And forward them to the WordPress deployment
 - ▶ **Important:**
 - ▶ This functionality is only meant for development
 - ▶ We should not use `port-forward` in production
 - ▶ Don't worry about this for now; we will explore different mechanisms later

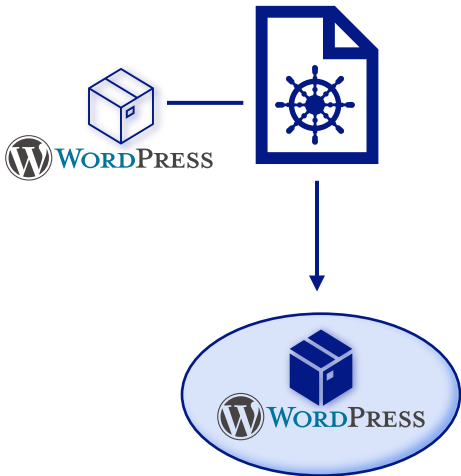


Docker, Kubernetes & AWS

Deploying WordPress (YAML)

Example: deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```



Docker, Kubernetes & AWS

Labels of a deployment

Labels of a deployment

- ▶ When specifying our deployment, we attached labels
- ▶ **Why do we need them?**
 - ▶ **They allow us to easily find our deployment:**
 - ▶ `kubectl get deployments -l app=wordpress`
 - ▶ Which labels we use, is entirely up to us
 - ▶ **We can even use filter by multiple labels:**
 - ▶ `kubectl get deployments -l app=wordpress,version=1.0`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
    version: '1.0'
```

Docker, Kubernetes & AWS

Labels for pods

Labels for the pods

► `spec.template.metadata.labels`:

- Allows us to specify labels for each pod
- **We can filter the pods by this label**
 - `kubectl get pods -l app=wordpress`

► `spec.selector.matchLabels`:

- It allows a deployment to find its pods
- **Important:** This property is immutable. It cannot be changed once a deployment is created
- **Why aren't the `spec.template.metadata.labels` enough?**
 - We must specify the labels for finding separately
 - The `matchLabels` should allow future updates
 - **Important:** As the `matchLabels` are immutable, we usually want to omit filtering by version number,...

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress-deployment
  labels:
    app: wordpress
spec:
  replicas: 3
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

Docker, Kubernetes & AWS

From Deployment to StatefulSet

From Deployment...

- ▶ **A deployment:**

- ▶ Is used to deploy stateless pods
- ▶ All pods can be launched at the same time

- ▶ **A pod in a deployment:**

- ▶ Is considered interchangeable to any other pod
(from the same deployment)
- ▶ When launched, it gets assigned a random identifier
- ▶ When scaling down, Kubernetes will choose randomly, which pod to terminate

... to StatefulSet

- ▶ **A StatefulSet:**

- ▶ Is used to deploy stateful pods

- ▶ **A pod in a StatefulSet:**

- ▶ Kubernetes maintains a sticky identity for each of its pods
 - ▶ They are created from the same spec - but are not interchangeable
 - ▶ Each pod is considered to perform a unique role

- ▶ **When scaling up:**

- ▶ Pods are created sequentially
 - ▶ Predecessors must be running and ready

- ▶ **When scaling down:**

- ▶ All its successors must be shutdown
 - ▶ Pods will be terminated in reverse order

Docker, Kubernetes & AWS

Environment variables

Environment variables

- ▶ **So far, the database failed to even launch:**

- ▶ To launch the database, we must set the MariaDB root password

- ▶ **For this, we must set an environment variable:**

- ▶ `spec.template.containers[0]`
 - ▶ `env:`
 - `name: MARIADB_ROOT_PASSWORD`
 - `value: "password"`

Docker, Kubernetes & AWS

Secrets

Secrets in Kubernetes

► A secret in Kubernetes:

- Object to store sensitive information securely
- **Examples:** passwords, OAuth tokens, TLS keys,...
- It decouples the sensitive information from the rest of the configuration
- The data is encoded in Base64 format

► Why do we need a secret?

- We could store the secret in a separate file... which does not get committed to the git repository
- This secret can then get managed separately

► In addition:

- We could implement more granular access control in Kubernetes
- Kubernetes can be configured to encrypt the data at rest
- **This is not part of this course!**

apiVersion: v1

kind: Secret

metadata:

name: my-mariadb-secret

type: Opaque

data:

username: YWRtaW4=

password: c2VjcmV0cGFzc3dvcmQ=

Using a secret

- ▶ **The easiest way to use a secret is to have it turned into an environment variable:**

- ▶ `env:`
 - `name: MARIADB_USER`
 - `valueFrom:`
 - `secretKeyRef:`
 - `name: my-mariadb-secret`
 - `key: username`

- ▶ *(there're also other ways for using a secret that are not part of this course)*

```
apiVersion: v1
kind: Secret
metadata:
  name: my-mariadb-secret
type: Opaque
data:
  username: YWRtaW4=
  password: c2VjcmV0cGFzc3dvcmQ=
```

Heads-up

- ▶ When deploying Kubernetes on the infrastructure of a cloud provider, you could also use the cloud providers secret mechanism
- ▶ **Example AWS:**
 - ▶ AWS Secrets Manager
 - ▶ This is independent of Kubernetes
 - ▶ You could choose to store all the secrets in there
 - ▶ In this case, AWS / IAM would be responsible for the permission management

Docker, Kubernetes & AWS

Using your own images, imagePullPolicy

Using your own images

- ▶ We can use Kubernetes with our own images
- ▶ **For this:**
 - ▶ All images must be available on all nodes in the cluster
- ▶ **In a single-node cluster:**
 - ▶ We can use all images usable by Docker locally
- ▶ **In a cloud setup:**
 - ▶ We should configure the cluster to pull images from our own, private repository
 - ▶ Cloud hosting providers usually provide a private repository for us (such Elastic Container Repository for AWS)

Setting the `imagePullPolicy`

- ▶ Kubernetes may automatically pull images again
- ▶ This can be configured through the `imagePullPolicy`
- ▶ **Possible values:**
 - ▶ **IfNotPresent:** Only pulls the image if it's not present on the local node
 - ▶ **Never:** Never pulls an image. If the image is not present, it will fail
 - ▶ **Always:**
 - ▶ Will always pull the image
 - ▶ Similar to a `docker pull` every time the container launches
 - ▶ Still utilizes docker's caching mechanism
- ▶ **By default:**
 - ▶ **IfNotPresent:** Usually used
 - ▶ **Always:**
 - ▶ Used when referring to the tag `:latest`, or using an image without a tag

Docker, Kubernetes & AWS

Networking in Kubernetes

Overview: Networking in Kubernetes

- ▶ **The problems now:**

- ▶ How could the WordPress instance connect to a database?
- ▶ How can we make WordPress more permanently accessible?

- ▶ **Or in general:**

- ▶ How does networking in Kubernetes work?
- ▶ How do IP addresses work within a Kubernetes cluster?
- ▶ How does DNS hostname resolution work?
- ▶ What is a service?
- ▶ Which kind of services does Kubernetes offer?

Docker, Kubernetes & AWS

Networking, IP addresses in Kubernetes

IP addresses within a Kubernetes cluster

- ▶ **In Kubernetes:**

- ▶ **Kubernetes deploys its own, separate network**

- ▶ Each pod gets its own unique IP address
 - ▶ It does not matter where the pod is run;
it will always have a Kubernetes-internal IP address
 - ▶ By default, each pod can access all other pods / services

- ▶ **As an application developer, this is amazing:**

- ▶ It allows us to think about a different pod as a different (physical) machine in a (physical) network
 - ▶ But the actual implementation is up to Kubernetes
 - ▶ Kubernetes can schedule the pod anywhere in the cluster
 - ▶ It may even decide to run it on the same host!

- ▶ **Let's launch a shell in one of our pods and see if we can ping the DB:**

- ▶ `kubectl exec -it [pod-name] -- /bin/bash`

10.1.0.128



Pod 1

10.1.0.129



Pod 2

Important

- ▶ **MySQL configuration:**

- ▶ MySQL is not configured to run in a cluster
- ▶ This means our database can only run on a single instance

- ▶ **Furthermore:**

- ▶ Furthermore, data might get lost if Kubernetes decides to spin up a new container for our database
- ▶ We should never use a Deployment for a stateful container!

Docker, Kubernetes & AWS

Networking in Kubernetes: Services

Networking in Kubernetes: Services

- ▶ **So far:**

- ▶ We had to connect to other pods through their IP address
- ▶ This IP address has been assigned on the fly
- ▶ It may even change in the future (after a system reboot,...)
- ▶ Thus, using an IP address would be more than just a bit inconvenient
- ▶ Isn't there a more robust way?

- ▶ **For this, we now want to explore services:**

- ▶ They allow us to easily connect to other pods in the cluster
- ▶ In case a pod is unavailable or stops, a service allows simple failover
- ▶ In addition, services will help us make pods publicly accessible

- ▶ **But let's have a look at them 😊**

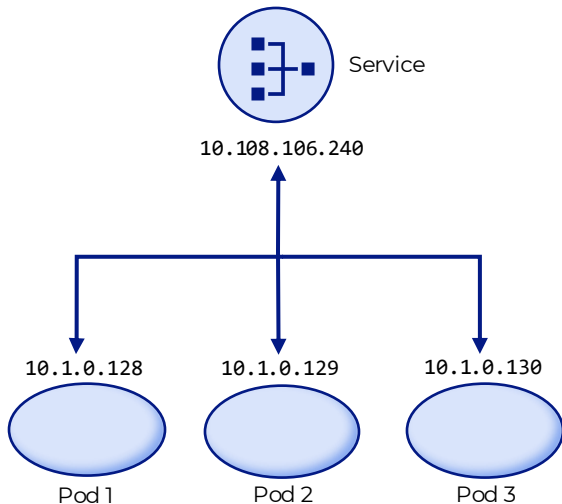
Services in Kubernetes

► Service:

- A mechanism, to make pods accessible (for now) from within a Kubernetes network
- It bundles pods and the access to them

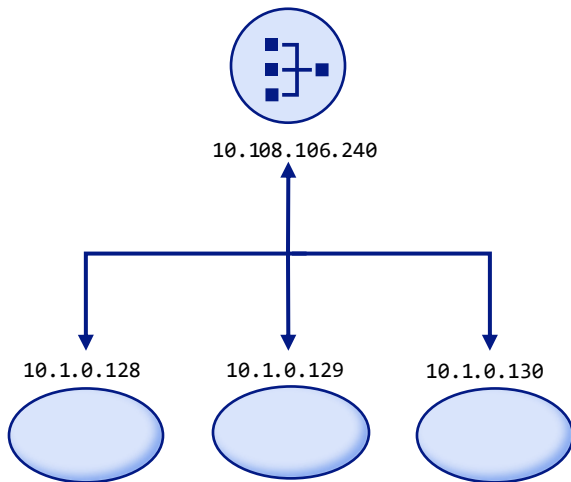
► Intuition:

- A service creates a new cluster-internal, IP address
- This address will remain the same even if the pods change
- **Here:** 10.108.106.240
- From within a pod, it looks like they can just connect to this IP address directly
- However, the service will forward this connection to one of the attached pods



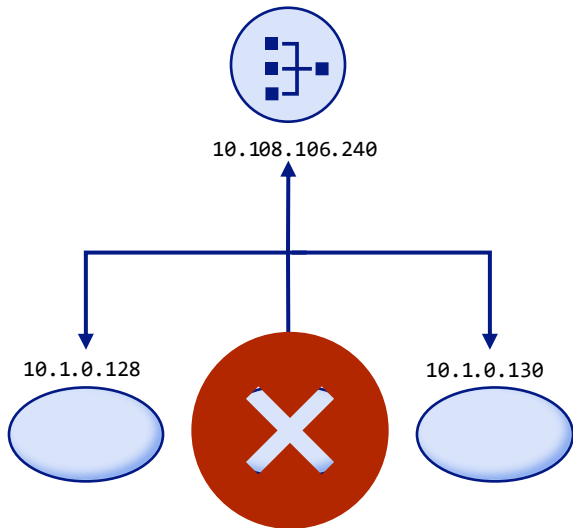
In addition: Automatic failover

- Services allow us to automatically deal with a crash or unavailability of an instance



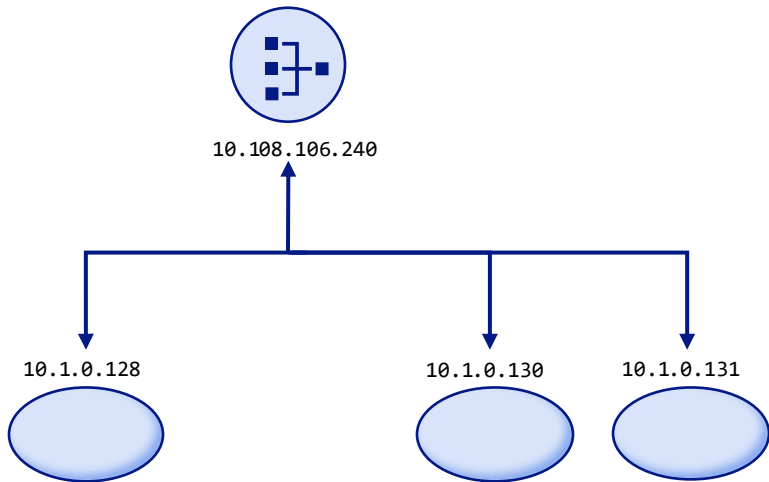
In addition: Automatic failover

- Services allow us to automatically deal with a crash or unavailability of an instance



In addition: Automatic failover

- Services allow us to automatically deal with a crash or unavailability of an instance



Docker, Kubernetes & AWS

Creating a service

Creating a service

- ▶ **We can either create it automatically:**
 - ▶ `kubectl expose deployment wordpress-deployment`
- ▶ **Or use YAML to define it:**
 - ▶ This gives us many more configuration options
 - ▶ The service has a selector attached to it, to find the pods where the traffic should be sent to
- ▶ **Once we have the service:**
 - ▶ We can then just use the IP address of the service
 - ▶ And get connected to (one of) the pods associated to it!

```
apiVersion: v1
kind: Service
metadata:
  name: mariadb-service
  labels:
    app: mariadb
spec:
  ports:
    - port: 3306
      protocol: TCP
  selector:
    app: mariadb
```

Docker, Kubernetes & AWS

Multiple configs in the same file

Multiple configs in a YAML file

- ▶ We can specify multiple entries in a single `.yaml` file
- ▶ **For example:**
 - ▶ We can specify first the `StatefulSet`...
 - ▶ ... and then its `Service`, all in a single YAML file
- ▶ This significantly simplifies updates to our configuration!

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mariadb-statefulset
  labels:
    app: mariadb
spec:
  [...]
---
apiVersion: v1
kind: Service
metadata:
  name: mariadb-service
  labels:
    app: mariadb
spec:
  [...]
```

Docker, Kubernetes & AWS

DNS in Kubernetes

DNS in Kubernetes

- ▶ **By default:**

- ▶ Services are reachable through their name
- ▶ This allows us to not have to work with IP addresses anymore

- ▶ **We can confirm this by executing a nslookup:**

- ▶ `apt update && apt install -y dnsutils`
- ▶ `nslookup mariadb-service`
- ▶ This now resolves to the IP address of the `mariadb-service`!

- ▶ **How does it work?**

- ▶ Kubernetes deploys its own DNS service (CoreDNS)
- ▶ This DNS service allows the resolution of hostnames to IP addresses

- ▶ **This means for us:**

- ▶ We now finally have a reliable way to access MariaDB...
- ▶ ... and we don't have to hard-code the IP address!

Docker, Kubernetes & AWS

Additional types of services

Type of services

- ▶ **Kubernetes offers various additional types of services**

- ▶ **The most important ones are:**

- ▶ **ClusterIP:**

- ▶ This is the default service type - we previously created a ClusterIP service
 - ▶ Used for cluster-internal connections

- ▶ **LoadBalancer:**

- ▶ Allows incoming connections
 - ▶ Utilizes a load balancer which needs to be provided by a cloud provider
 - ▶ Thus locally, we can't use this type

- ▶ **NodePort:**

- ▶ Does everything that a ClusterIP does...
 - ▶ ... but in addition, also opens a port on all our nodes
 - ▶ Which will expose this service to the public
 - ▶ This public port should be in the range between 30'000 and 32'767

Docker, Kubernetes & AWS

Service type: NodePort / Creating a service for the WordPress pods

Creating a NodePort

- ▶ Now, we want to create a service for exposing the WordPress instances to the public
- ▶ **These pods:**
 - ▶ Should be publicly accessible
 - ▶ For now, we're running Kubernetes locally
 - ▶ This means, we can only choose NodePort as a service
- ▶ **But how does a NodePort service work?**

NodePort service

- ▶ To create a NodePort service, we can just specify the type and set it to NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
  labels:
    app: wordpress
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30080
      protocol: TCP
  selector:
    app: wordpress
```

Docker, Kubernetes & AWS

The ports of a service

Why 3 different ports?

► port:

- This is the cluster-internal port for this service
- This is the port under which this service is available within the cluster
- In our case, this is port 80, so within the cluster, nodes can send requests to <http://wordpress-service> (on port 80)
- This traffic will then get forwarded to the individual WordPress containers

► targetPort:

- This is the port that the service will forward traffic to
- Our wordpress container is exposing port 80, thus, this is the targetPort
- If omitted, this is the same as the port of the service

► nodePort (only for NodePort services):

- Cluster-external port for this service
- This port will be exposed through the NodePort service
- On each node, this port gets opened directly on the host
- From outside the cluster, we can access this port to access the service

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
  labels:
    app: wordpress
spec:
  type: NodePort
  ports:
    - targetPort: 80
      port: 80
      nodePort: 30080
      protocol: TCP
  selector:
    app: wordpress
```

Docker, Kubernetes & AWS

Naming ports

Naming ports

- ▶ In Kubernetes, ports can also be named
- ▶ This allows us to more easily refer to ports by their name
- ▶ **Let's just explore this!**

Docker, Kubernetes & AWS

Kubernetes: Volumes

Kubernetes: Volumes

- ▶ **In this chapter:**

- ▶ We want to explore how to store data in our Kubernetes cluster
- ▶ For this, we must explore how volumes work and how we can configure them

- ▶ **At the end of this chapter:**

- ▶ WordPress will be able to install and store data persistently across all replicas
- ▶ MariaDB will be able to store actual data

- ▶ **Thus:**

- ▶ We will finally be able to go through the WordPress install!

Heads-up

- ▶ Storage in Kubernetes can be quite vendor-specific
- ▶ **On AWS:**
 - ▶ We might want to store WordPress data on an EFS volume
 - ▶ And the database on an EBS volume
- ▶ **In this chapter (for now):**
 - ▶ We will focus on how to store volumes in our local cluster
 - ▶ This allows us to explore how volumes work in general

Docker, Kubernetes & AWS

Kubernetes: Volume types

Main variants of volumes

- ▶ **Ephemeral volume:**

- ▶ Exists for the lifetime of a pod
- ▶ If this pod is destroyed:
 - ▶ The volume will be destroyed automatically

- ▶ **Persistent volume:**

- ▶ Will **not** be destroyed automatically
- ▶ The volume will persist, even if a pod is destroyed

- ▶ **Important:**

- ▶ Both variants can be implemented through different means, known as volume types
- ▶ We will only explore a few, simple volume types
- ▶ These types can get extended by cloud-specific plugins

Kubernetes vs. Docker volumes

- ▶ **Kubernetes volume:**

- ▶ Fully managed by Kubernetes
- ▶ We must specify it through the configuration

- ▶ **Docker volume:**

- ▶ Kubernetes may internally use docker to run the containers

- ▶ **Docker also supports volumes:**

- ▶ But those volumes are completely independent of Kubernetes
- ▶ Kubernetes doesn't know anything about these volumes
- ▶ **This means:** If docker implicitly creates a volume, this volume will just persist within docker

- ▶ **TLDR: Kubernetes will never manage docker volumes for us!**

Docker, Kubernetes & AWS

Kubernetes: Using a first volume

Using a first volume

- ▶ **To use a volume:**

- ▶ We assign a volume to the pod
- ▶ We mount this volume into a container in this pod

- ▶ **Here, we are using an ephemeral volume:**

- ▶ **emptyDir:**

- ▶ A temporary directory
- ▶ Shares a pod's lifetime
- ▶ Storage is provided locally from the kubelet

- ▶ **This means:**

- ▶ Data will be lost if we launch another pod!

```
apiVersion: v1
kind: Pod
metadata:
  name: my-ubuntu-pod
spec:
  containers:
    - image: ubuntu
      name: my-ubuntu
      command: ["sleep", "infinity"]
      volumeMounts:
        - mountPath: /app
          name: app-volume
  volumes:
    - name: app-volume
      emptyDir: {}
```

Docker, Kubernetes & AWS

Ephemeral volume: hostPath

The volume type: hostPath

► Let's now try another type of volume:

► hostPath:

- Mounts a file / directory from the host into a pod
- Data is tied to the specific node running the pod
- **If the pod is rescheduled and run on a different node:**
 - The data will not be transferred
 - This can lead to potential data loss
- **Thus:**
 - Suitable for sharing specific host files or directories, usually only for temporary data
 - This data may (or may not) persist beyond the pod's lifecycle
- To use a hostPath volume, we must adjust the YAML accordingly

```
apiVersion: v1
kind: Pod
metadata:
  name: my-ubuntu-pod
spec:
  containers:
    - image: ubuntu
      name: my-ubuntu
      command: ["sleep", "infinity"]
      volumeMounts:
        - mountPath: /app
          name: app-volume
  volumes:
    - name: app-volume
      hostPath:
        path: /path/on/host
```

Docker, Kubernetes & AWS

PV, PVCs, StorageClass

PV, PVCs, StorageClass

- ▶ We now want to explore a completely different way of handling volumes in Kubernetes
- ▶ This way will give us significantly more control over volumes, and is required to store files separately from each container
- ▶ **For this, we will explore the following terms:**
 - ▶ StorageClass
 - ▶ PersistentVolume (PV)
 - ▶ PersistentVolumeClaim (PVC)

Kubernetes: StorageClass

- ▶ **A StorageClass:**

- ▶ Provides a way to define different classes of storage in a Kubernetes cluster
- ▶ Specifies parameters for provisioning storage resources
- ▶ Most properties are not relevant for statically provisioned volumes

- ▶ **Only relevant for us:**

- ▶ The name of the StorageClass
- ▶ Which StorageClass is the default one?
- ▶ Potentially, additional Parameters

- ▶ **Provisioner:**

- ▶ We can see this as a "driver" for this StorageClass

Docker, Kubernetes & AWS

Persistent volumes (PV)

Persistent volume

- ▶ **A persistent volume:**

- ▶ Provides a consistent way to handle storage
- ▶ You can imagine a PV as a possible place for storing a PVC

- ▶ **Or, in other words:**

- ▶ A PV provides a storage location to Kubernetes
- ▶ Kubernetes may (or may not) use it as a place to store data
(= assign a PVC to it)

Persistent volume

- ▶ Let's now create a persistent volume
- ▶ For this, we can use YAML again
- ▶ **Important properties:**
 - ▶ **spec.capacity.storage:**
 - ▶ Sets the maximum storage capacity that the PV can provide
 - ▶ **spec.accessModes:**
 - ▶ Specifies the supported access modes
 - ▶ More on those later
 - ▶ **spec.storeClassName:**
 - ▶ Indicates which StorageClass should be used
 - ▶ **spec.hostPath, spec.hostPath.path:**
 - ▶ We are creating a PV that is directly stored in the filesystem of the host
 - ▶ The folder can be configured through `spec.hostPath.path`

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: hostpath
  hostPath:
    path: /mnt/data
```

Docker, Kubernetes & AWS

Persistent volumes claims (PVC)

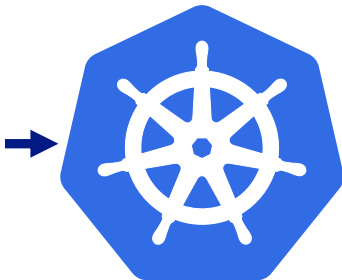
Persistent volume claims (PVC)

- ▶ PVCs allow users to request storage - without knowing the details of the underlying storage
- ▶ **In other words:**
 - ▶ A PVC fills the "bucket" created by a PV
 - ▶ Users specify the size and access capabilities... and Kubernetes automatically finds appropriate PVs to store the data
- ▶ This gives Kubernetes lots of flexibility on where to store the actual data
- ▶ **Important:**
 - ▶ A PV can only be used by a single PVC!

Matching PVCs to PVs

Persistent Volume Claim

```
storageClassName: hostpath
resources:
  requests:
    storage: 10Gi
```



PV 1:

storageClassName: hostpath
(no PVC attached yet)



PV 2:

storageClassName: hostpath
(already used by another PVC)



PV 3:

storageClassName: hostpath
(no PVC attached yet)



PV 4:

storageClassName: efs-sc
(no PVC attached yet)

- ▶ **This PVC can only be created on PV 1 or PV 3:**
 - ▶ They are not used, and their storageClassName matches
- ▶ **It will never be placed on PV 2:**
 - ▶ PV2 is already used by another PVC
- ▶ **It will never be placed on PV 4:**
 - ▶ The storageClassName does not match

How to create a PVC

- To create a PVC, we can use the YAML syntax:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: local-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: hostpath
```

Docker, Kubernetes & AWS

Using a PVC from a pod

Using a PVC from a pod

- ▶ **To use a Persistent Volume Claim (PVC) in a pod, we need to:**
 - ▶ Create a pod specification
 - ▶ Add a volume to the pod
 - ▶ And mount this volume into the container
- ▶ The PVC then provides the pod with access to persistent storage
- ▶ **Let's explore this!**

```
apiVersion: v1
kind: Pod
metadata:
  name: my-ubuntu-pod
spec:
  containers:
    - image: ubuntu
      name: my-ubuntu
      command: ["sleep", "infinity"]
      volumeMounts:
        - mountPath: /data
          name: app-volume
  volumes:
    - name: app-volume
      persistentVolumeClaim:
        claimName: local-pvc
```

Docker, Kubernetes & AWS

PVCs in a StatefulSet

PVC in a StatefulSet

► When using a PVC in a StatefulSet:

- We can choose to create a separate PVC for each pod in the StatefulSet
- For this, we can specify a `volumeClaimTemplate` in the spec of the StatefulSet:

```
spec:
  volumeClaimTemplates:
  - metadata:
      name: mariadb-data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: hostpath
      resources:
        requests:
          storage: 1Gi
```

Docker, Kubernetes & AWS

Access Modes

Access Modes

- ▶ PVs must declare their supported access modes
- ▶ PVCs must declare their required access mode
- ▶ **Access mode:**
 - ▶ Specifies the relationship between the pods and the storage layer
 - ▶ Dictates how the pods can read and write data to the storage
- ▶ **Available access modes include:**
 - ▶ **ReadWriteOnce:**
 - ▶ Volume can be mounted by a single node
 - ▶ Commonly used for storage that doesn't support simultaneous read-write access from multiple nodes
 - ▶ **ReadWriteOncePod:** Volume can be mounted as read-write by a single pod
 - ▶ **ReadWriteMany:**
 - ▶ Volume can be mounted as read-write by many nodes
 - ▶ Used for shared storage solutions
 - ▶ **ReadOnlyMany:** Volume can be mounted as read-only by many nodes

Outlook: Why do we need access modes

- ▶ Different storage providers may support different access modes
- ▶ **Amazon EFS (Elastic File System):**
 - ▶ Shared filesystem, based on NFS (Network File System)
 - ▶ Multiple nodes connect to a central file server
 - ▶ Managing this causes some performance overhead
- ▶ **Amazon EBS (Elastic Block Storage):**
 - ▶ A block storage in the cloud
 - ▶ **Block storage:**
 - ▶ Data is stored in a fixed-size block, fast random-access, high performance
 - ▶ Data is accessed in a binary way
 - ▶ Your internal SSD / HDD is a block storage (and your OS builds its filesystem on top)
 - ▶ You can imagine this as an SSD, that is accessed through a network
 - ▶ It can only be connected to a single node at a time
 - ▶ **But because of that:** Less overhead (=faster)!

Docker, Kubernetes & AWS

Kubernetes

Kubernetes

- ▶ You now have a solid foundation about Kubernetes
- ▶ **You know about:**
 - ▶ Volumes
 - ▶ Networking
 - ▶ Services
 - ▶ And how it all works together
- ▶ **And especially:**
 - ▶ You have managed a complete setup in your local Kubernetes cluster!
- ▶ **You are now able to:**
 - ▶ Discuss Kubernetes deployment issues with others
 - ▶ Perform changes to the configuration of a Kubernetes cluster

Kubernetes: Outlook

- ▶ **In the next lecture:**

- ▶ You can test your knowledge with a practice test
- ▶ This practice test is optional:
 - ▶ You can do it now... or at the end of this course

- ▶ **In the next chapter:**

- ▶ We will explore AWS EKS (Elastic Kubernetes Service)
- ▶ If you don't follow along with AWS, you can skip this chapter... and you are now at the end of this course



Docker, Kubernetes & AWS

Kubernetes on AWS:

Elastic Kubernetes Service (EKS)

EKS: Overview

- ▶ **In this chapter:**

- ▶ We will launch a Kubernetes cluster in the AWS cloud

- ▶ **In this chapter:**

- ▶ You will run an nginx web server, including a load balancer

- ▶ **After this, we will then deploy a complete WordPress setup:**

- ▶ MySQL as a database
 - ▶ WordPress for the blog
 - ▶ All running from within the Kubernetes cluster

- ▶ **However, the whole setup is rather complex:**

- ▶ It's best to run the EKS cluster in its own, separate VPC
 - ▶ This VPC must be properly connected to the internet
 - ▶ The firewall must be configured properly (security groups,...)
 - ▶ Thus, we'll utilize an additional, official tool to perform the whole setup for us (eksctl)

How to manage costs

- ▶ **We see 3 different paths for this chapter:**

- ▶ **If costs are not an issue:**

- ▶ Just watch the videos, and keep the EKS cluster running over multiple days

- ▶ **[Recommended] To minimize costs:**

- ▶ You can also watch the videos first, without following along
 - ▶ Once you've familiarized yourself with EKS, you can then set aside a whole day to create your own EKS cluster
 - ▶ At the end of the day, you'll delete the EKS cluster
 - ▶ This approach minimizes costs

- ▶ **Mix of both approaches:**

- ▶ You can dedicate a weekend to this chapter
 - ▶ And keep the EKS cluster running over 1 night
 - ▶ Then at least you won't be charged for several weeks of EKS time

Heads-up

- ▶ In this chapter, we will create a complete Kubernetes cluster in AWS
- ▶ Be sure to familiarize yourself with the pricing – from all chapters, this will be the most expensive chapter
- ▶ **In addition:**
 - ▶ We will need / utilize several other services from AWS
 - ▶ Elastic IP, VPC, Elastic Load Balancer,...
 - ▶ Additional charges will occur for this
- ▶ It's best to only run it for a few hours only!

Docker, Kubernetes & AWS

Installing eksctl

Installing eksctl

- ▶ **We could create the cluster directly in the AWS dashboard**
- ▶ **However:**
 - ▶ We would have to configure everything ourselves
 - ▶ We would have to setup the networking, networking rules, and much more
- ▶ **Let's now install eksctl:**
 - ▶ <https://eksctl.io/>
 - ▶ After this, we can check if eksctl has been installed correctly:
 - ▶ `eksctl help`
- ▶ **Heads-up:**
 - ▶ If installing eksctl doesn't work... you can also run it through docker
 - ▶ `docker run --rm -it -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube public.ecr.aws/eksctl/eksctl help`

Docker, Kubernetes & AWS

Using eksctl

Using eksctl: Creating the cluster

- ▶ **We can now easily create a cluster through eksctl:**

- ▶ `eksctl create cluster --full-ecr-access --alb-ingress-access --region eu-west-3`

- ▶ **This will create the full stack for us:**

- ▶ Separate VPC for just the EKS cluster
 - ▶ It will also enable access to ECR (`--full-ecr-access`)
 - ▶ And enable the settings to use a load balancer (`--alb-ingress-access`)

- ▶ **We could also add:**

- ▶ `--node-private-networking`
 - ▶ In this case, individual worker nodes won't get assigned a public IP
 - ▶ This reduces costs, and the attack surface
 - ▶ However, in this chapter we'll want to examine a `NodePort` service in more detail...
 - ▶ ... and for that, it will be helpful if nodes have a public IP
 - ▶ ... but for real-world projects, consider adding this option as well!

Creating the cluster with eksctl

- ▶ **Heads-up:**

- ▶ Configuring the cluster can take up to 15-30 minutes

- ▶ **In addition:**

- ▶ eksctl will configure kubectl for us
- ▶ We will be able to use kubectl to access the EKS cluster

- ▶ **To change this back:**

- ▶ `kubectl config get-contexts`
- ▶ `kubectl config set-context [context]`

Docker, Kubernetes & AWS

Launching an nginx

Launching an nginx

- ▶ **Let's launch a first deployment:**

- ▶ Deployment of the nginx web server...
- ▶ In combination with a NodePort service

- ▶ **Let's explore this!**

- ▶ **However:**

- ▶ We were not able to access nginx through port 30080

- ▶ **The reason:**

- ▶ The security groups of the node do not allow incoming connections on port 30080
- ▶ If we enable this, we can access nginx!



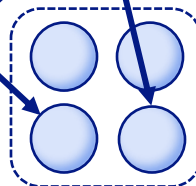
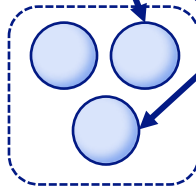
EKS cluster



NodePort
service

NodePort service
on node #1

NodePort service
on node #2

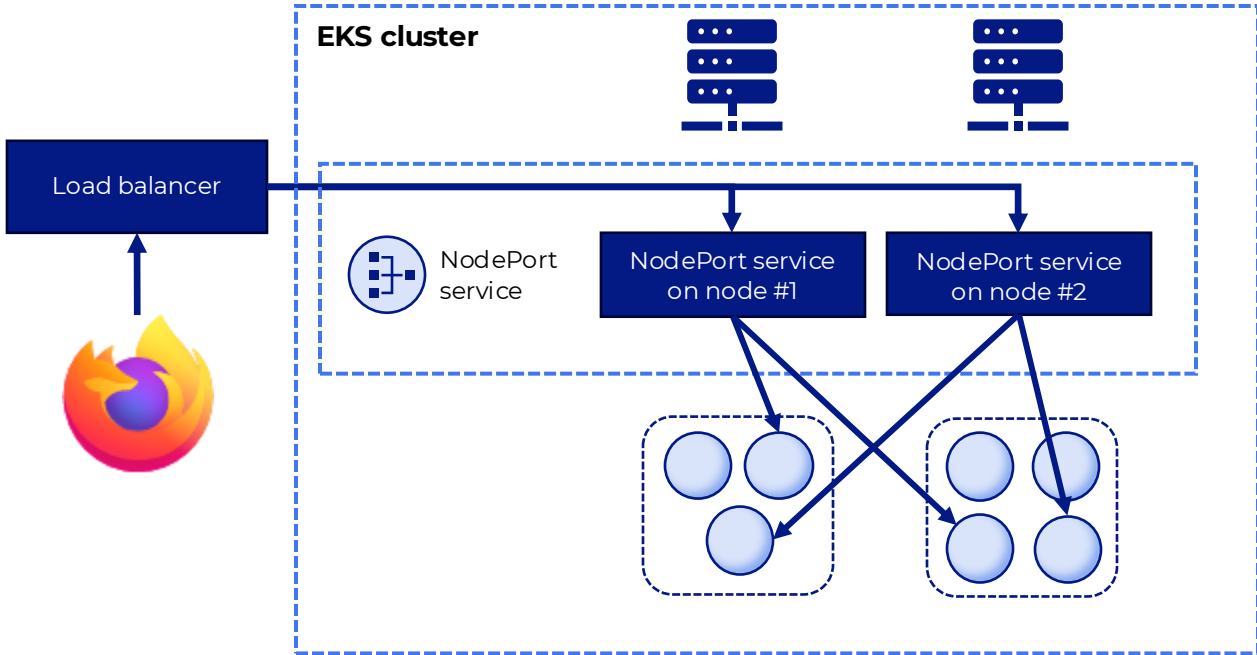


Docker, Kubernetes & AWS

Using a LoadBalancer (service)

Using a LoadBalancer

- ▶ In the previous chapter, we were not able to use a `LoadBalancer` service... this type of service is only available in a managed Kubernetes cluster
- ▶ This would be the generic / cross-platform way of creating a load balancer for a service
- ▶ **We're now running on a managed cloud (AWS):**
 - ▶ This means, we can now use a `LoadBalancer` service
 - ▶ Internally, it will also use a `NodePort` service (it builds on top of that)
 - ▶ ... but it will also create an EC2 Load Balancer for us!



Troubleshooting: LoadBalancer

- ▶ If the load balancer does not work, we need to ensure that all connections are allowed



- ▶ Must listen on port 80 / 443
 - ▶ Must allow incoming connections on port 80 / 443
 - ▶ Must be allowed to access the nodes in the EKS cluster
 - ▶ Must be in the same VPC as the nodes of the EKS cluster (default, no need to check)
- ▶ Must allow incoming requests from the load balancer (on the node port)

However

- ▶ While this approach works, this kind of load balancer is almost deprecated within AWS
- ▶ We should consider using a different approach for creating a load balancer!
- ▶ Let's just create a new, application load balancer ourselves!

Docker, Kubernetes & AWS

Configuring an application load balancer (manually)

Manually creating a load balancer

► The idea:

- Let's create an application load balancer manually...
- ... and point it to the `NodePort` service for our nginx

► Heads-up:

- The load balancer must be able to communicate to the nodes of the EKS cluster



- Must listen on 80 / 443
 - Must allow incoming connections on port 80 / 443
 - Must be allowed to access the nodes of the EKS cluster
 - Must be in the same VPC as the nodes of the EKS cluster
- Must allow incoming requests from the load balancer

Docker, Kubernetes & AWS

[Optional]: Automatically configuring a load balancer

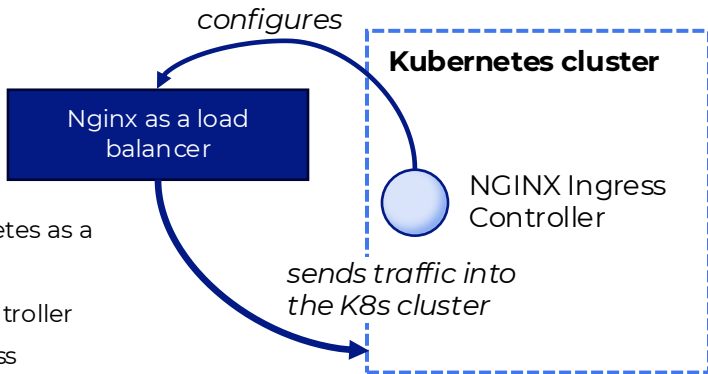
Automatically configuring a load balancer

- ▶ In the previous lecture, we had successfully configured an application load balancer manually
- ▶ **However, this approach could lead to a few problems:**
 - ▶ If we added additional nodes... the load balancer would not pick them up automatically
 - ▶ The load balancer had to be configured separately from the Kubernetes cluster
 - ▶ Can't we enable a more seamless configuration?

Kubernetes: Ingress

► Ingress in Kubernetes:

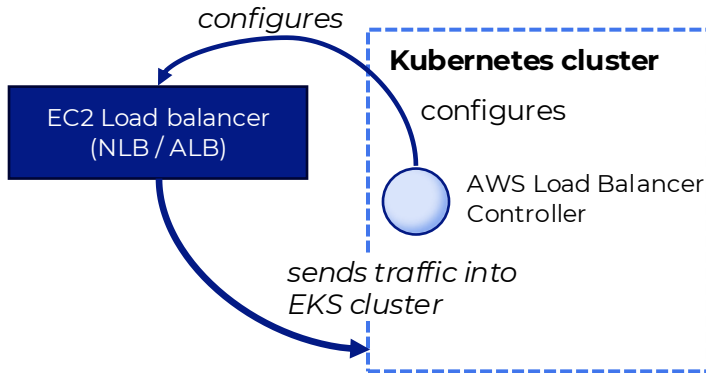
- A general mechanism to get traffic into a Kubernetes cluster
- **Most used (outside of AWS):**
 - NGINX Ingress Controller
 - Nginx runs externally (outside of Kubernetes as a load balancer
 - It just gets configured by the Ingress Controller
 - The Ingress Controller watches the ingress configuration of Kubernetes...
 - ... and if it detects a change of the ingress config...
 - ... it will re-configure the load balancer



Kubernetes: Ingress

► Within AWS / EKS:

- We usually use the AWS Load Balancer Controller
- It automatically provisions and configures EC2 load balancers:
 - Network Load Balancer (NLB)
 - Application Load Balancer (ALB)



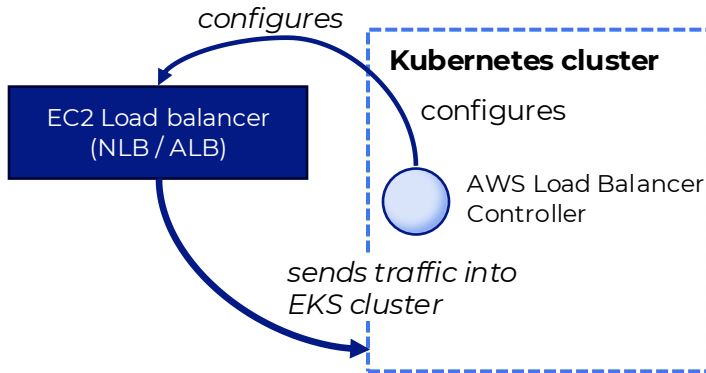
Docker, Kubernetes & AWS

[Optional]: Installing the AWS Load Balancer Controller

Kubernetes: Ingress

► Within AWS / EKS:

- We usually use the AWS Load Balancer Controller
- It automatically provisions and configures EC2 load balancers:
 - Network Load Balancer (NLB)
 - Application Load Balancer (ALB)



Installing the AWS load balancer Controller

- ▶ We pretty much must follow the guide here:
 - ▶ <https://kubernetes-sigs.github.io/aws-load-balancer-controller/v2.8/deploy/installation/>

Docker, Kubernetes & AWS

[Optional]: Enabling https for the automatically configured load balancer

Enabling https

► To enable https:

- We must have an https certificate in the Amazon Certificate Manager
- We must add an annotation to enable this certificate:

```
metadata:
  name: "nginx-ingress"
  namespace: default
  labels:
    app: nginx-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/certificate-arn: '[ARN of certificate]'
```

Docker, Kubernetes & AWS

[Optional]: **Directly routing traffic to pods**

Directly routing traffic to pods

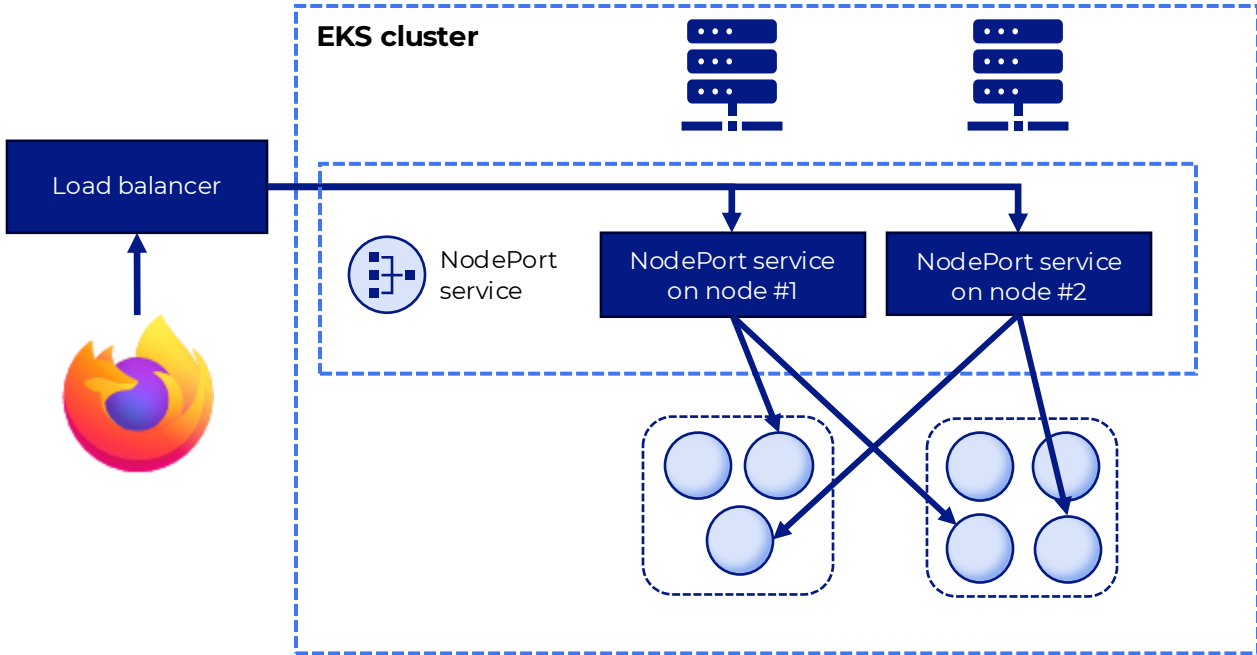
- ▶ **So far:**

- ▶ The load balancer had been rather inefficient
- ▶ Traffic was forwarded to an individual node...
- ▶ ... which would then forward it to the service...
- ▶ ... which would then forward it to any individual pod in the cluster

- ▶ **The result:**

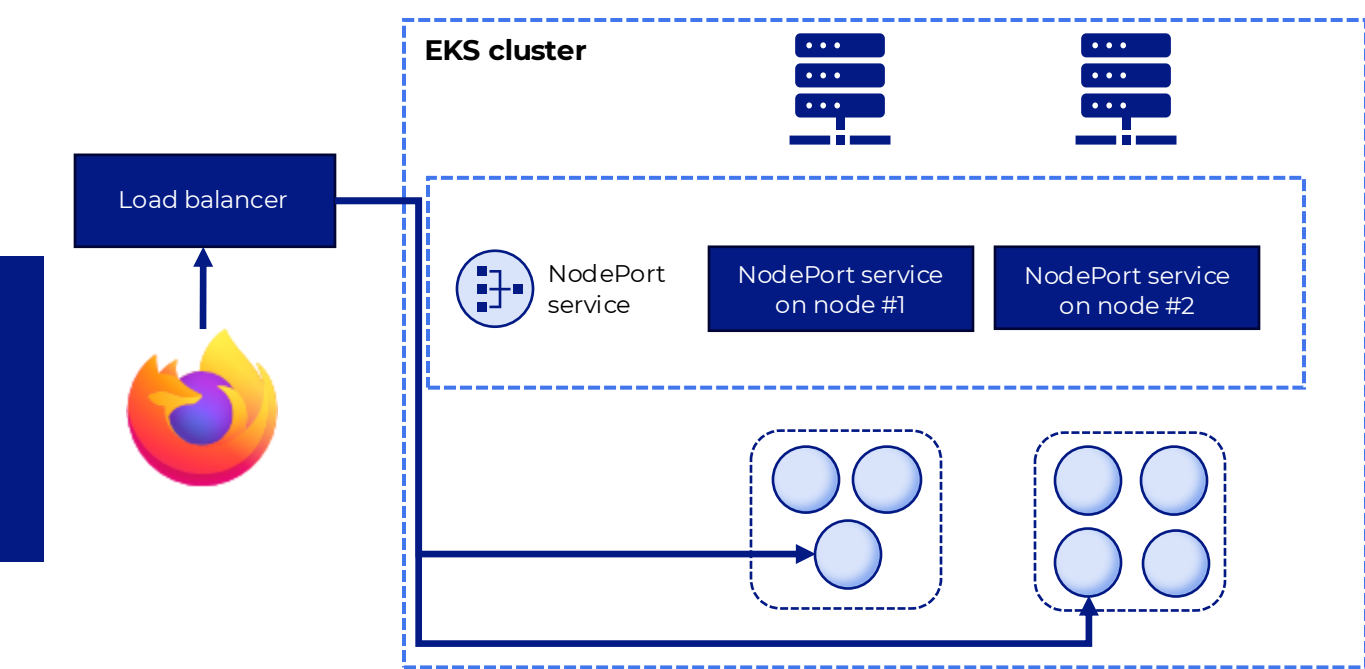
- ▶ Additional traffic within our cluster!

- ▶ **Let's explore the problem!**



Directly sending traffic to the pod

- ▶ When using AWS Load Balancer Controller, we can change it
- ▶ **The idea:**
 - ▶ The load balancer directly sends traffic to the right pod
 - ▶ The traffic can then skip the NodePort service completely
- ▶ **For this:**
 - ▶ The pods must be directly accessible through distinct IPs
- ▶ **Meaning:**
 - ▶ Kubernetes usually assigns individual, cluster-internal IPs to the pods
 - ▶ In this case, we couldn't forward traffic directly to a pod
 - ▶ But if we give pods IP addresses that are accessible from our VPC...
 - ▶ ... then the load balancer could send traffic directly to them!



How to enable this

- ▶ **To enable this:**

- ▶ **VPC CNI:**

- ▶ Container Network Interface
 - ▶ Pods will then share the same IP range as the VPC
 - ▶ This makes pods accessible from within the VPC of the cluster
 - ▶ We can enable this as a Kubernetes add-on

- ▶ **After this, we must configure the load balancer:**

- ▶ It should communicate directly to the pods
 - ▶ We need to add the following annotation to the ingress configuration:
 - ▶ `alb.ingress.kubernetes.io/target-type: ip`

Docker, Kubernetes & AWS

[Optional]: Using a custom image

[Optional]: Using a custom image

- ▶ Luckily, when creating the cluster, we had used the `eksctl` command with the parameter: `--full-ecr-access`
- ▶ This has configured the cluster, and it can now access the ECR

Docker, Kubernetes & AWS

EKS with storage: Overview

Overview: EKS with storage

- ▶ **In this chapter:**

- ▶ We want to deploy a full WordPress setup in EKS
- ▶ For this, we must explore 2 different ways of storing data:
 - ▶ EFS (Elastic File System)
 - ▶ EBS (Elastic Block Storage)

- ▶ **For this, we will explore:**

- ▶ How to access EFS from EKS
- ▶ How to access EBS from EKS:
 - ▶ We will explore how to limit our cluster to a single availability zone only

Docker, Kubernetes & AWS

Configuring storage: Elastic File System (EFS)

Configuring an EFS volume

- ▶ We now want to add an EFS volume to our EKS cluster
- ▶ **For this, we must:**
 - ▶ **1) Create and configure the EFS volume:**
 - ▶ Create the EFS volume in the VPC of the cluster
 - ▶ Allow incoming connections from the security group of the nodes of the EKS cluster
 - ▶ **2) Configure the cluster (EKS specific):**
 - ▶ Add the EFS driver to the cluster (and create the appropriate IAM role to facilitate this)
 - ▶ This driver will configure the EFS volume for our cluster
 - ▶ **3) Configure the cluster (normal Kubernetes):**
 - ▶ Create the PersistentVolume
 - ▶ Create the PersistentVolumeClaim
 - ▶ Adjust the WordPress configuration to use the PVC

Docker, Kubernetes & AWS

Outlook: Running a database in a cluster (EBS)

Elastic Block Storage (EBS)

- ▶ **EBS:**

- ▶ Block storage service for use with EC2 instances (and EKS)
- ▶ Designed for high availability and durability
- ▶ Offers consistent and low-latency performance

- ▶ **The downside:**

- ▶ It can only be used by a single EC2 instance at a time

- ▶ **This means:**

- ▶ In Kubernetes, this is equivalent to an access mode of `ReadWriteOnce`
- ▶ Kubernetes will then only use the volume on a single node then
- ▶ Which works for us, as we run the DB in a single pod anyway
- ▶ In addition, the EBS volume must be placed in the same availability zone (AZ)

- ▶ **Outlook:**

- ▶ Before we can use an EBS volume, we need to prepare our cluster for it...

Heads-up

- ▶ Kubernetes is best for stateless applications
- ▶ **This means:**
 - ▶ It is difficult to run a database in a scalable way in Kubernetes
 - ▶ We need to consider if we really want to self-host our database...
 - ▶ Instead, we should consider to use a managed database service from the AWS cloud
- ▶ **However:**
 - ▶ Here, we will run a database in a stateful way in EKS on AWS
 - ▶ This allows us to explore how stateful applications work on EKS

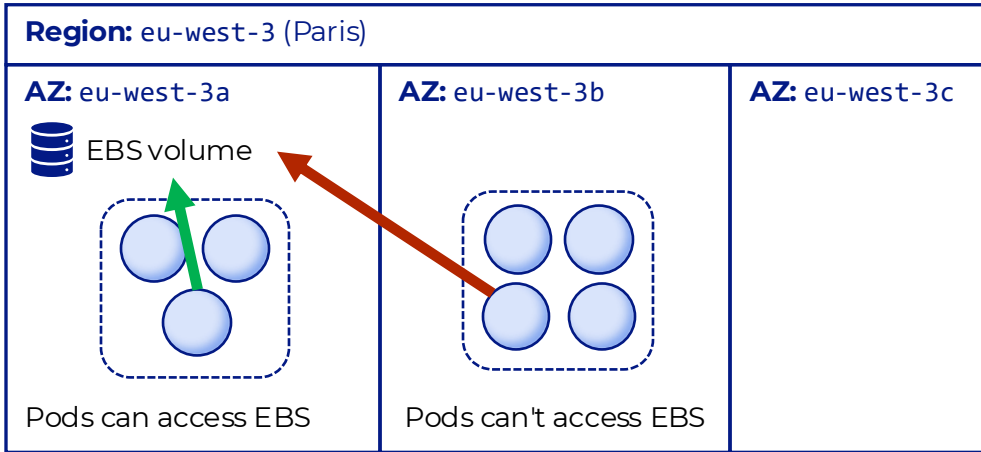
Docker, Kubernetes & AWS

Preparing for EBS: Outlook

Availability zones (AZ) and EBS

► The problem:

- An EBS volume can only be used by nodes that are placed in the same availability zone
- If the node was in a different availability zone... it can't access it!



Availability zones and EBS

- ▶ **How not to solve it:**

- ▶ A lot of tutorials use automatic provisioning for EBS volumes
- ▶ Through this, Kubernetes will automatically provision the volume in the correct AZ

- ▶ **However:**

- ▶ Let's say we had 2 nodes, one in eu-west-3a and one in eu-west-3b
- ▶ The EBS volume would then be provisioned in one of the AZ... let's say in eu-west-3a
- ▶ If now the node / pod crashed and got scheduled in eu-west-3b... it suddenly couldn't connect to the EBS volume anymore
- ▶ **This is not what we want!**

Availability zones and EBS

- ▶ **The easiest solution (we will take this path):**

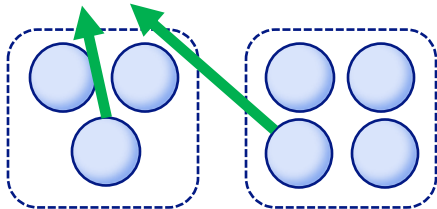
- ▶ We can configure the cluster to only run in a single availability zone, let's say in eu-west-3a...
- ▶ Then, we can always connect to an EBS volume in eu-west-3a

Region: eu-west-3 (Paris)

AZ: eu-west-3a



EBS volume



No matter on which node,
the EBS volume can be accessed

AZ: eu-west-3b

AZ: eu-west-3c

Availability zones and EBS

- ▶ **To achieve this:**

- ▶ We need to configure the autoscaling group of the node group, to only launch nodes in a single availability zone
- ▶ We need to perform an instance refresh on this autoscaling group to apply the changes

More complex solution

- ▶ **A more complex solution:**

- ▶ We could also ensure that we always got a node running in eu-west-3a (through a custom node group)
- ▶ And assign the database instance to only run on this node group
- ▶ Then the rest of the pods could also run in a different availability zone

- ▶ **If we wanted fail-over:**

- ▶ We of course need to repeat this process, for each availability zone...
and then create a separate replica of the `StatefulSet` for the database for each availability zone
- ▶ As you probably can imagine, this requires significantly more configuration from our side

Docker, Kubernetes & AWS

Creating an EBS volume

Configuring an EBS volume

- ▶ We now want to add an EBS volume to our EKS cluster
- ▶ **For this, we must:**
 - ▶ **Create and configure the EBS volume:**
 - ▶ Create the EBS volume in the AZ of the cluster
 - ▶ **Configure the cluster (EKS):**
 - ▶ Add the EBS driver to the cluster
(and create the appropriate IAM role to facilitate this)
 - ▶ **Configure the cluster (Kubernetes):**
 - ▶ Create the PersistentVolume
 - ▶ Create the PersistentVolumeClaim
 - ▶ Adjust the MariaDB configuration to use the PVC

Docker, Kubernetes & AWS

Deleting the cluster

Deleting the cluster

- ▶ **To delete the cluster:**

- ▶ We can do most of the deletion through the `eksctl` command:

- ▶ `eksctl delete cluster --region eu-west-3 adorable-monster-1726732797`

- ▶ **However, after that:** *(preferably in this order)*

- ▶ We must remove the EFS volumes
 - ▶ We must remove the EBS volumes
 - ▶ We can delete the IAM rules
 - ▶ We might have to manually remove the remaining load balancers
 - ▶ We might have to manually delete the NAT gateway
 - ▶ We might have to manually delete the Internet Gateway
 - ▶ We might want to release remaining Elastic IP addresses
 - ▶ We need to remove all remaining security groups
 - ▶ We must remove the VPC